COSC264 · TERM 4

Module 5

# Transport Layer Protocols

TCP and UDP

cosc264.up.railway.app

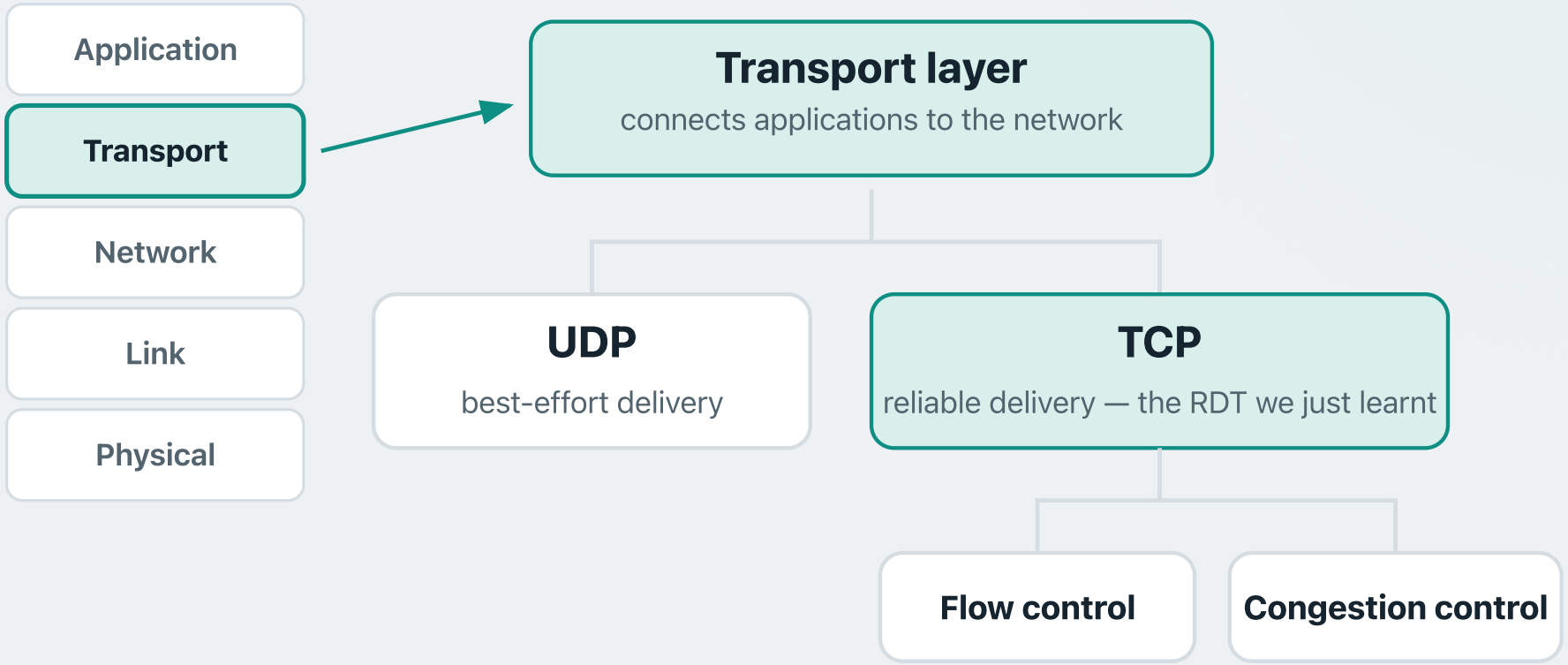
# Module 4 — what we covered

- Reliable Data Transfer (RDT) moves data reliably over an unreliable channel.
- From rdt 1.0 to rdt 3.0 we used checksums, acknowledgements (ACKs), sequence numbers, and a timer to detect errors and loss.
- RDT up to rdt 3.0 is **stop-and-wait**, which is inefficient, so we added **pipelining** — several packets in flight at once, capped by a **sender window**.
  - **Go-back-N** uses a sender window and **cumulative ACKs**; the receiver discards anything that arrives out of order.
  - **Selective repeat** improves on that with a **receiver buffer**, so out-of-order packets are kept rather than thrown away.

OUR ROUTE THROUGH THE TRANSPORT LAYER

# Module 5 roadmap

Recall: the Internet stack has five layers



END-TO-END ABSTRACTION

# What does the transport layer do?

Imagine a network application that runs as two processes, **process A** and **process B**, on separate devices.



END-TO-END ABSTRACTION

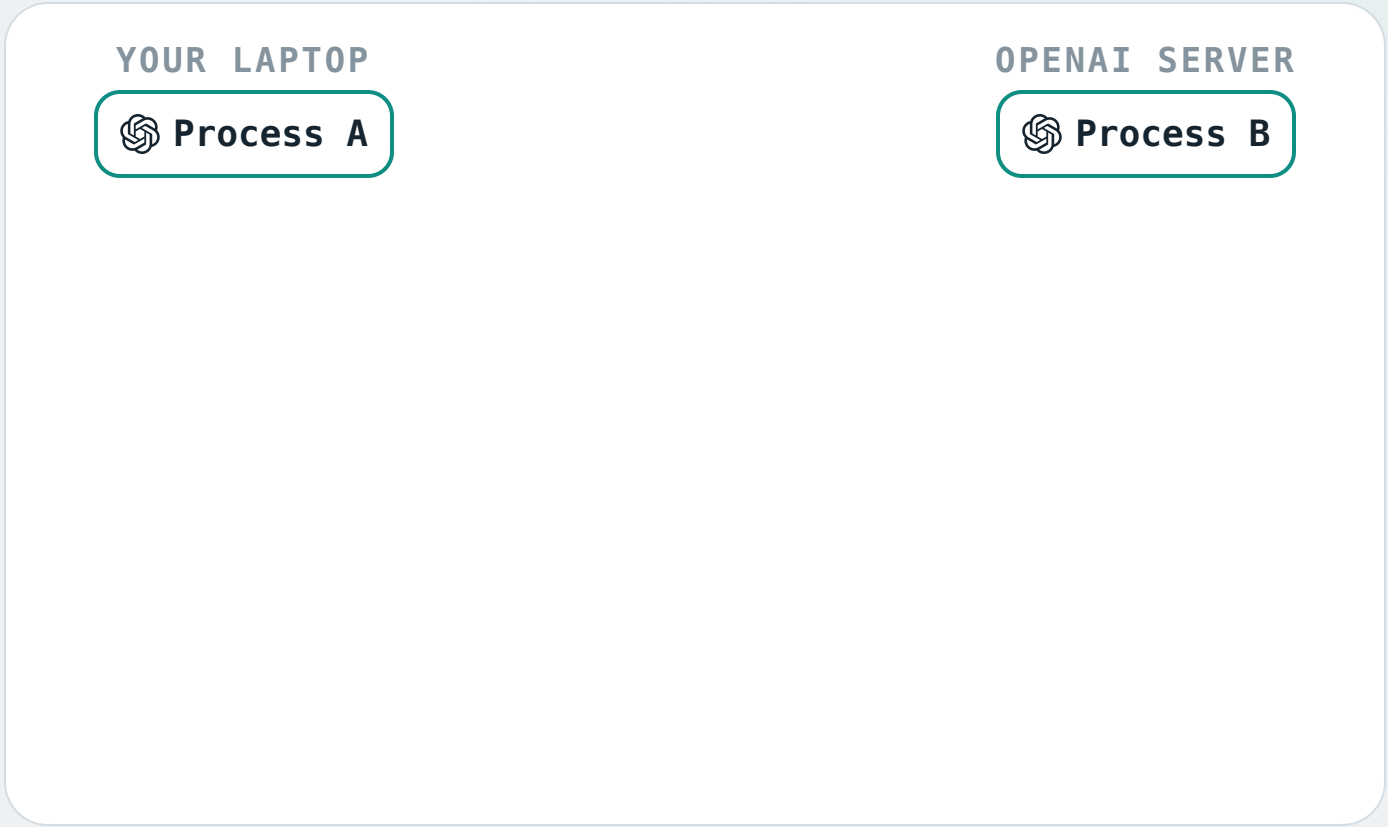
# What does the transport layer do?

Imagine a network application that runs as two processes, **process A** and **process B**, on separate devices.

e.g. process A is the **ChatGPT app** on your laptop, and process B runs on the **OpenAI server** that handles each request and sends back the response.

The two processes sit on separate devices over the network, not face to face.

Therefore, they need an **end-to-end communication channel** across the Internet.



END-TO-END ABSTRACTION

# What does the transport layer do?

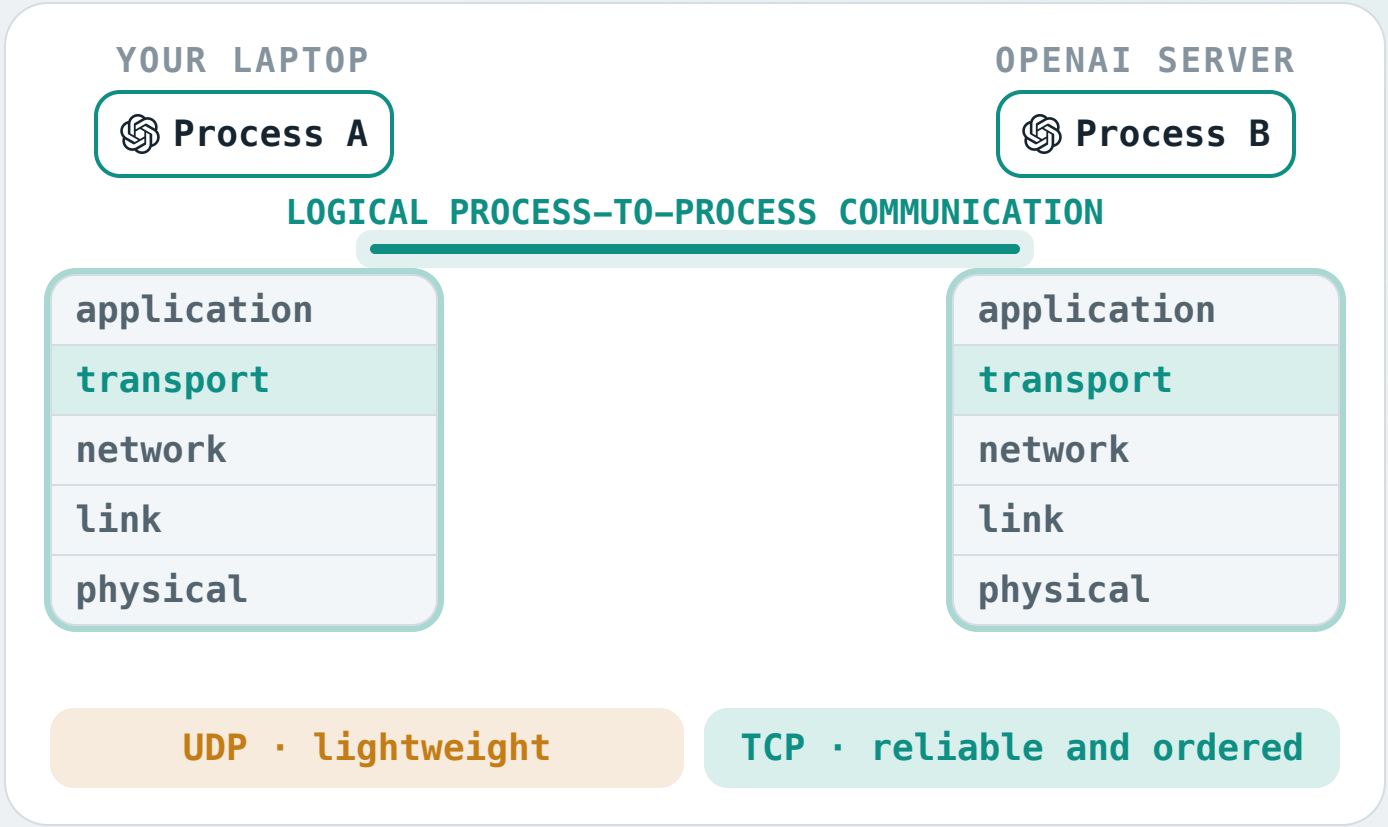
Therefore, they need an **end-to-end communication channel** across the Internet.

**Transport layer provides that channel.** It provides logical end-to-end communication between two processes.

**Abstraction.** The transport layer abstracts away the network's lower-layer mechanics, keeping them invisible to applications.

**Service choice.** Applications can choose UDP or TCP for different delivery needs.

UDP TCP



HOW THE ABSTRACTION IS DELIVERED

# Mechanisms utilized by transport-layer protocols

To provide this end-to-end abstraction, transport protocols use several mechanisms.

## Error detection

A checksum helps detect corruption in a received transport unit.

TCP + UDP

## Multiplexing and demultiplexing

Next up.

TCP + UDP

## Reliable data transfer (RDT)

Module 4.

TCP

## Flow and congestion control

Later in this module.

TCP

ONE HOST · MANY APPLICATION PROCESSES

# Processes reach the network through sockets

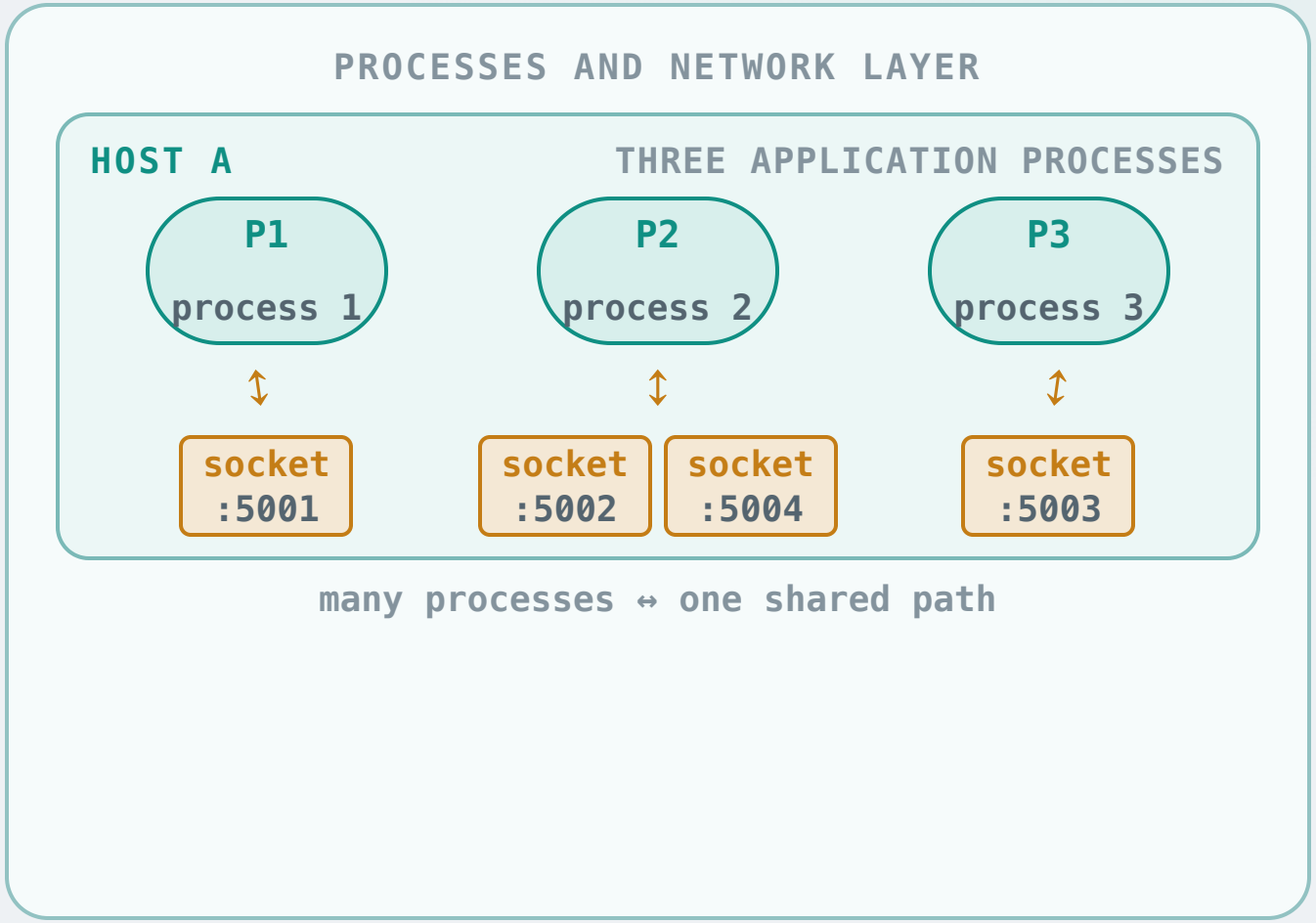
A single host can run many network applications over multiple processes. Each process may need to send and receive traffic through the network.

A process does not talk to the network directly. It sends and receives through a **socket**.

A socket is the transport layer's endpoint — the door a process writes its outgoing data into, and reads its incoming data out of.

Each socket is bound to a **port number** (16 bits, 0–65535) on its host.

A process may hold **more than one socket**.

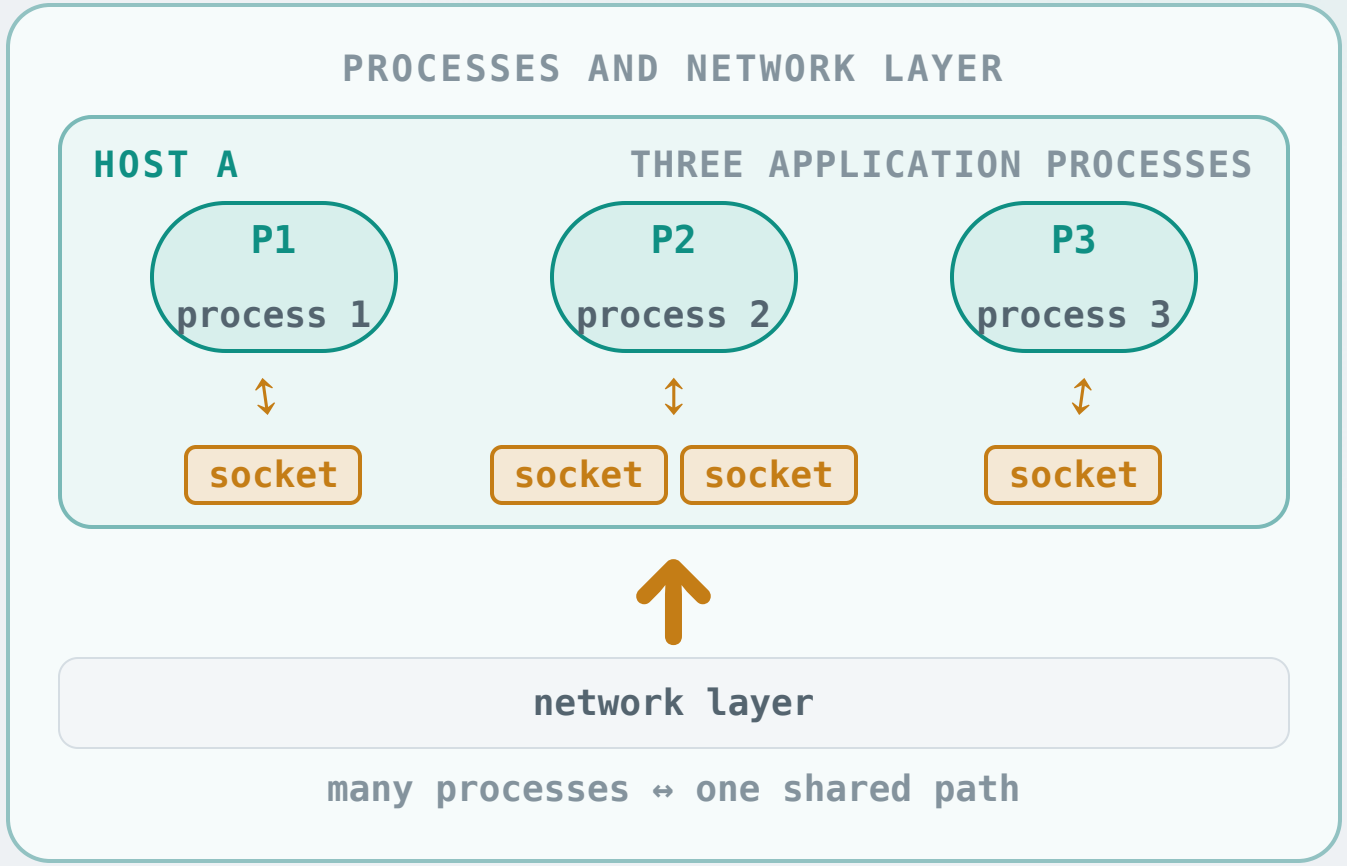


MANY SOCKETS · ONE SHARED PATH

# Why multiplexing is needed

The network layer — **IP** — only ever looks at **IP addresses**. To it a host is a single object at a single address; it has no view of the sockets inside.

So when a packet arrives, IP can deliver it to the **host** — but it cannot tell which **socket** the data inside should go to.



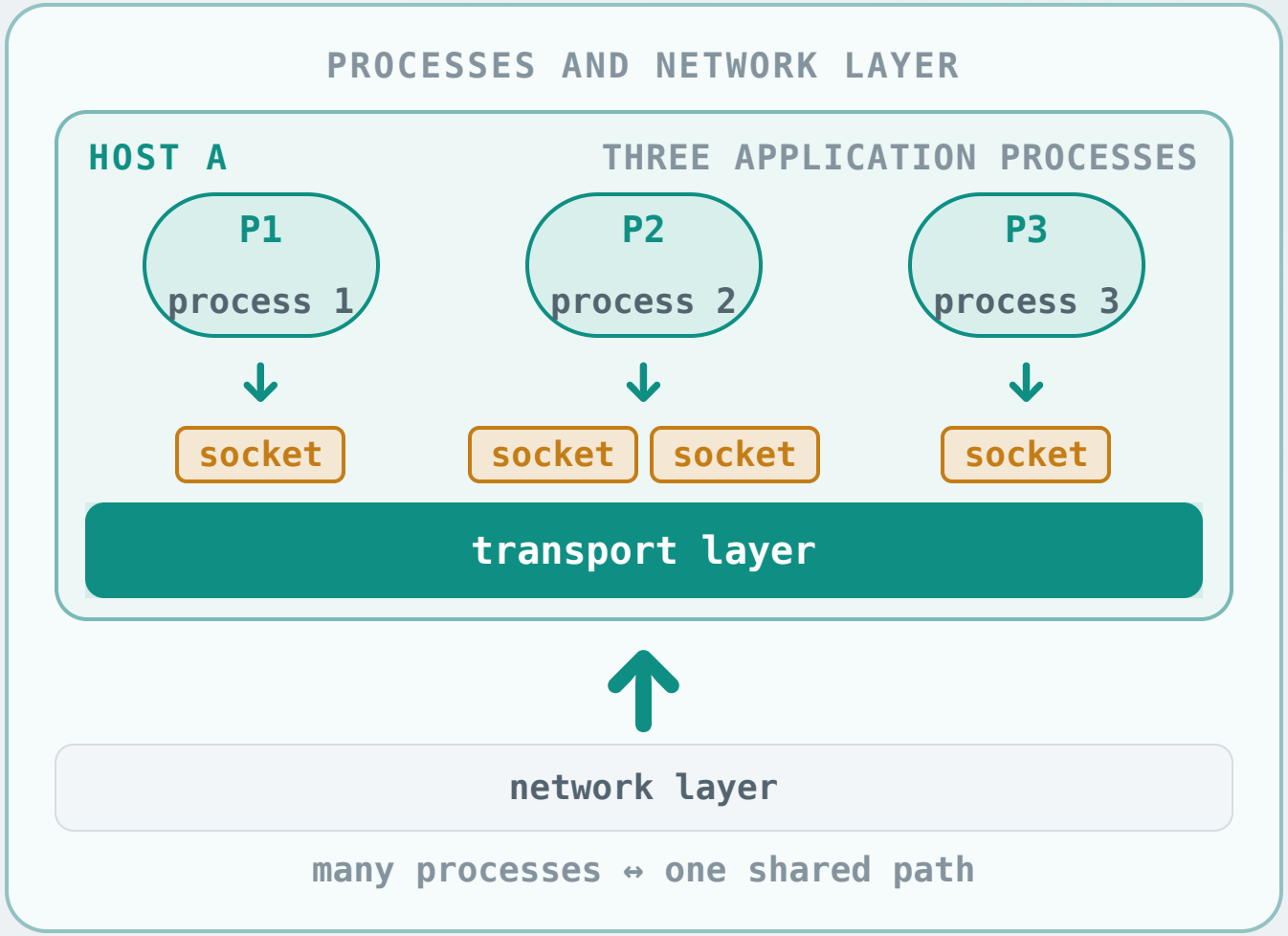
MANY SOCKETS · ONE SHARED PATH

# Why multiplexing is needed

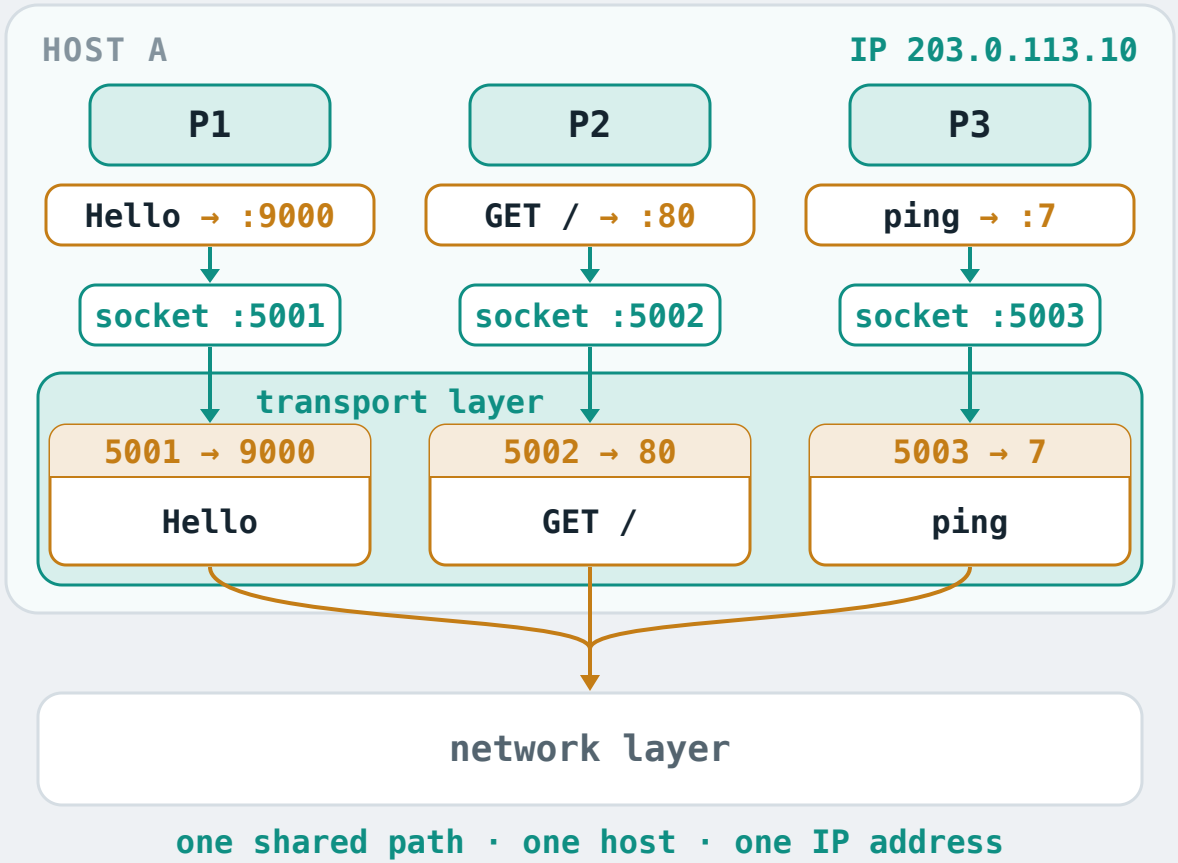
The network layer — **IP** — only ever looks at **IP addresses**. To it a host is a single object at a single address; it has no view of the sockets inside.

So when a packet arrives, IP can deliver it to the **host** — but it cannot tell which **socket** the data inside should go to.

The transport layer coordinates this traffic in both directions so each socket gets exactly its own data. This is known as **multiplexing/demultiplexing**.



# Multiplexing



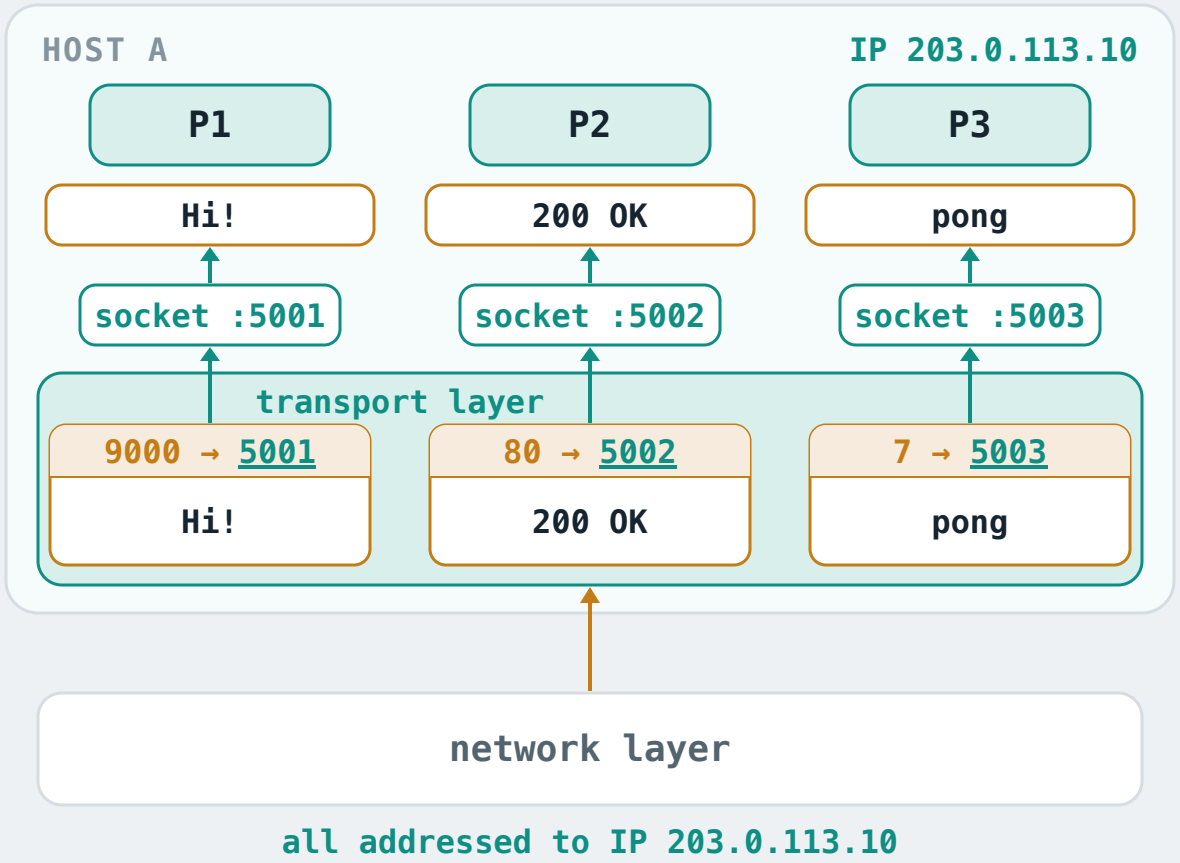

**Multiplexing** gathers data from many sockets and sends it down to the network layer.

**Each process writes to its own socket.** P1, P2 and P3 each hand their socket a message and its **destination port**, e.g. P1 sends "Hello" to port 9000. (We omit the destination IP address for simplicity.)

**Each message becomes a segment.** The transport layer adds a header carrying the source and destination ports.

**One shared path.** All segments go down to the same network layer, one after another — to IP, they come from a single host with a single IP address.

# Demultiplexing




**Demultiplexing** delivers each segment's data to the right socket.

**Segments arrive.** The network layer delivers them to this host, all addressed to its single IP address.

**Read the destination port.** The transport layer looks at each segment's header.

**Deliver to the matching socket.** It strips the header and hands the data to the socket with that port, e.g. "200 OK" goes to port 5002 (P2).

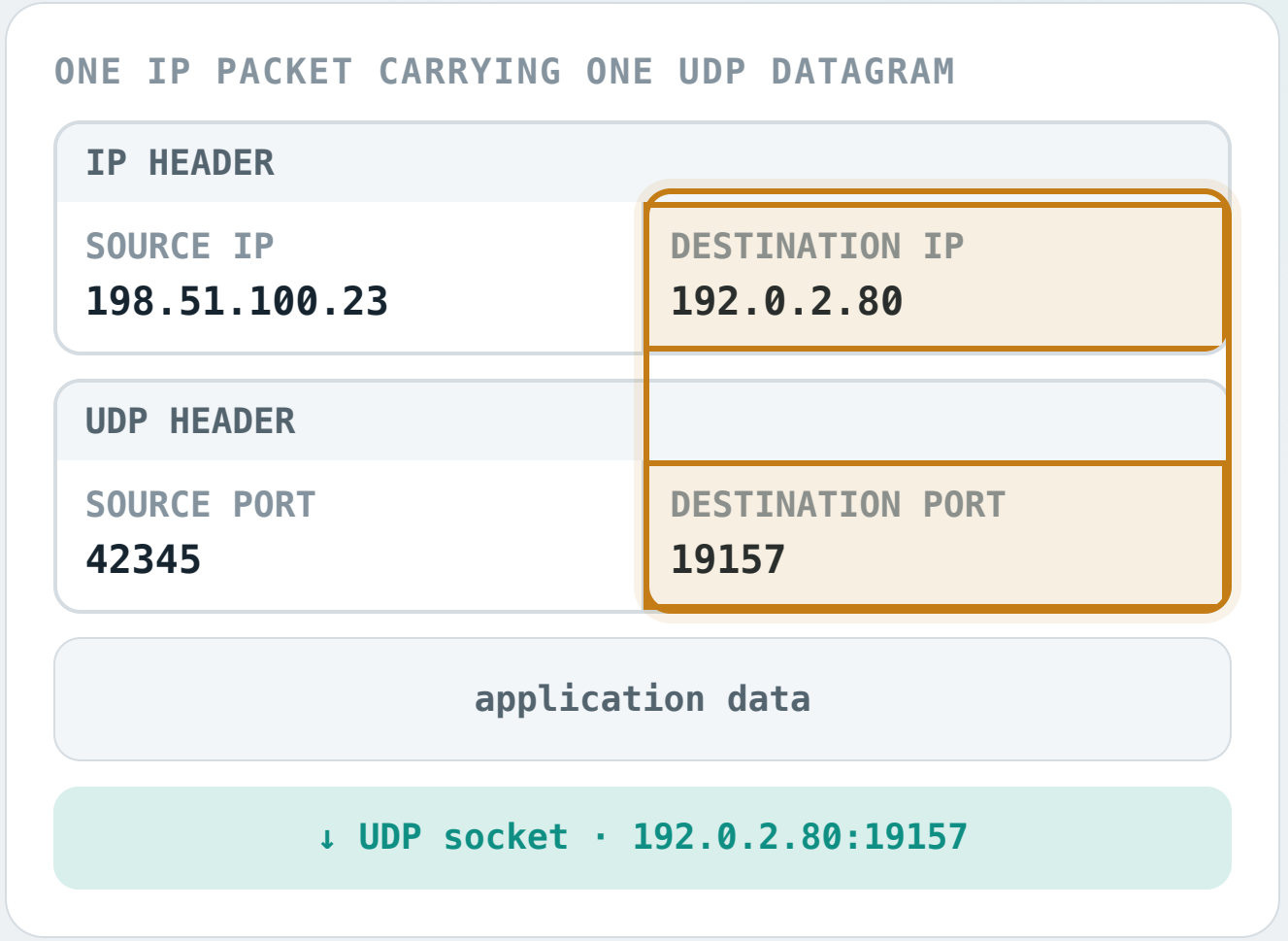
# UDP demultiplexing

Suppose the network layer uses the destination IP address to deliver the following IP packet to some host.

## How does UDP perform demultiplexing afterwards?

**UDP looks at two fields.** UDP uses the **destination IP address** in the IP header and the **destination port** in the **UDP header** to identify the correct socket.

For this packet, the identified socket is the one bound to **192.0.2.80 : 19157**.



# TCP demultiplexing

Suppose the network layer uses the destination IP address to deliver the following IP packet to some host.

**How does TCP perform demultiplexing afterwards?**

**TCP looks at four fields.** TCP uses the source IP address and destination IP address in the IP header, plus the source port and destination port in the TCP header, to identify the correct connected socket.

For this packet, the identified socket is the one bound to the pair **203.0.113.10 : 46428 ↔ 192.0.2.80 : 80**.

TCP CONNECTION IDENTITY

| Source endpoint | Destination endpoint |
|---|---|
| SOURCE IP ADDRESS<br>203.0.113.10 | DESTINATION IP ADDRESS<br>192.0.2.80 |
| SOURCE PORT<br>46428 | DESTINATION PORT<br>80 |

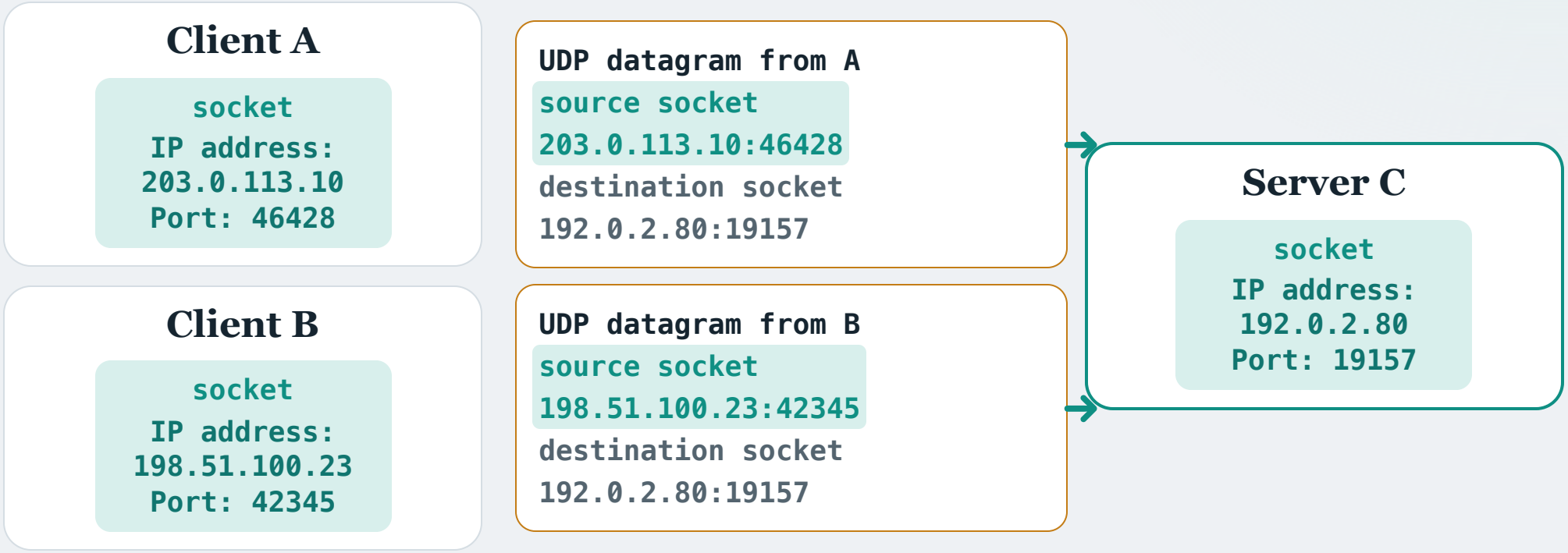
203.0.113.10:46428 ↔ 192.0.2.80:80

# UDP is unreliable and connectionless

- **Unreliable.** UDP does not guarantee delivery, ordering, or protection against duplicates.
- **Connectionless.** A sender does not set up a connection before sending.
- Why not? Because UDP identifies the socket by the **destination alone**. Put a destination IP address and port in the header, and UDP hands the datagram to that socket.
- It does not care where the datagram came from, so there is **no binding** between a particular sender and a particular receiver.

# Different sources, one socket

- Since UDP is connectionless, two different senders can send to the **same destination socket**, as depicted in this diagram.
- **But then how does the destination reply to each of them?**
- Every datagram also carries its **source IP address and port** — the return address the destination replies to.



FOUR FIELDS · EIGHT-BYTE HEADER

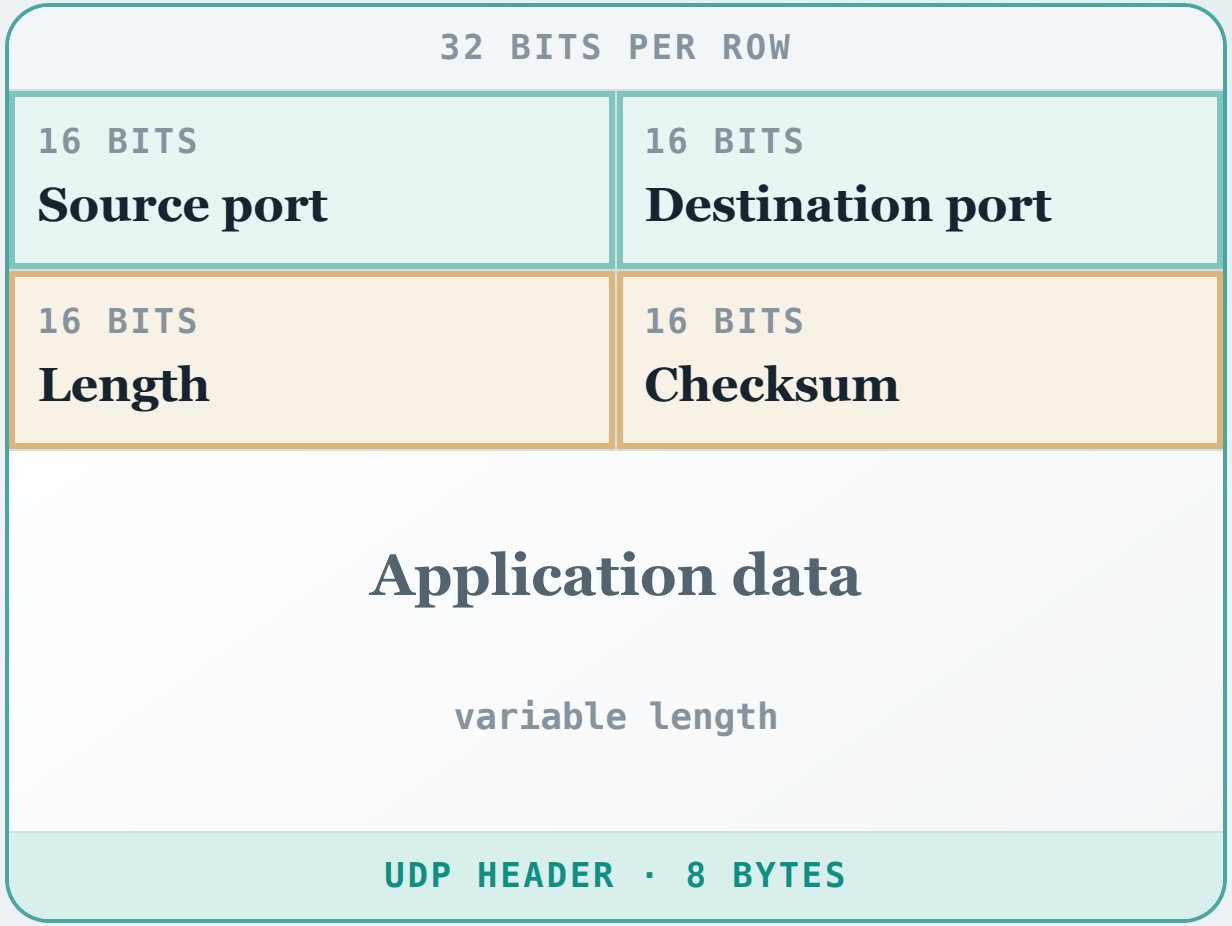
# Breaking down the UDP header

**Always 8 bytes.** Four 16-bit fields and nothing optional, so a UDP header is the same size on every datagram.

**Ports.** The 16-bit source and destination ports identify the sending and receiving application endpoints.

**Length.** The total number of bytes in the UDP header and application data; the minimum is eight.

**Checksum.** Covers the UDP header and the entire payload data to detect corruption.



THE BENEFIT OF A MINIMAL TRANSPORT SERVICE

# Why do some applications choose UDP?

1

## No setup delay

An application can send immediately because UDP does not establish a transport connection first.

2

## Minimal connection state

Since UDP is connectionless, it keeps no per-connection state — no buffers, sequence numbers or windows.

3

## Small header

Every UDP message adds only an eight-byte transport header.

4

## Application control

UDP provides no reliability, so the application decides when to send and builds whatever recovery it needs — rather than taking TCP's fixed mechanism.

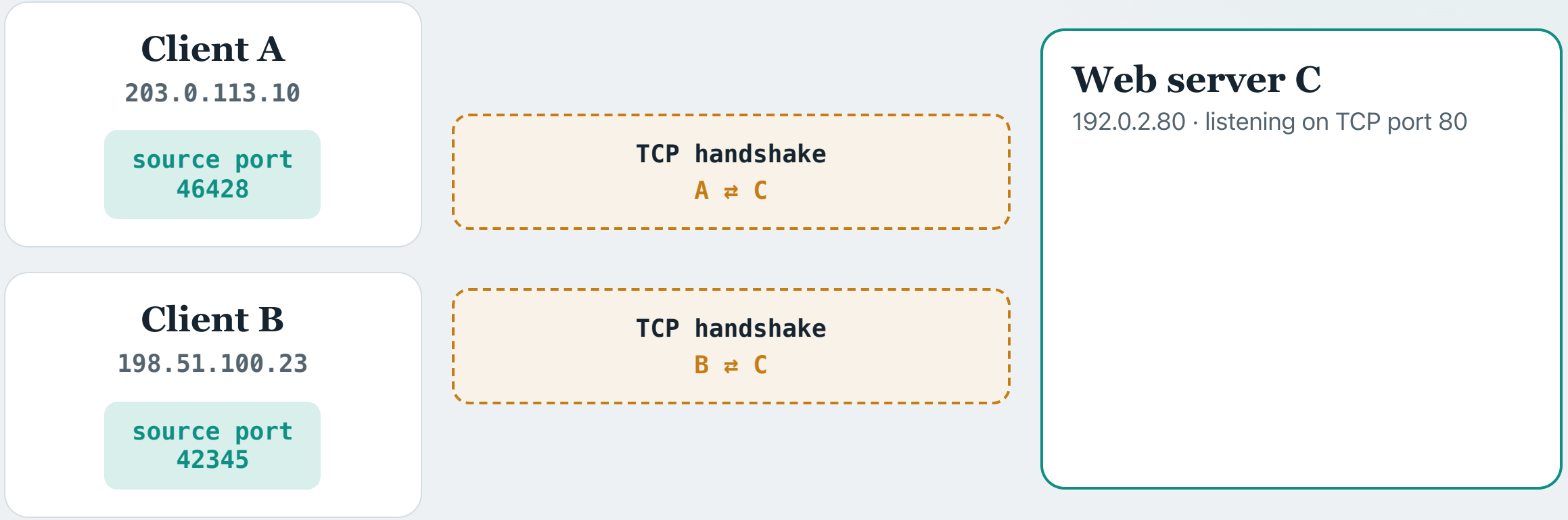
Common UDP examples include DNS lookups, real-time media, and online games.

# TCP is reliable and connection-oriented

- **Reliable.** TCP delivers the byte stream without loss, duplication or reordering — it has an RDT mechanism built in.
- **Connection-oriented.** TCP identifies a connection by **four** values: source IP address and port, destination IP address and port.
- So a sender must first run a **handshake** to set the connection up before it may send any data, and that connection ties one particular sender to one particular receiver.

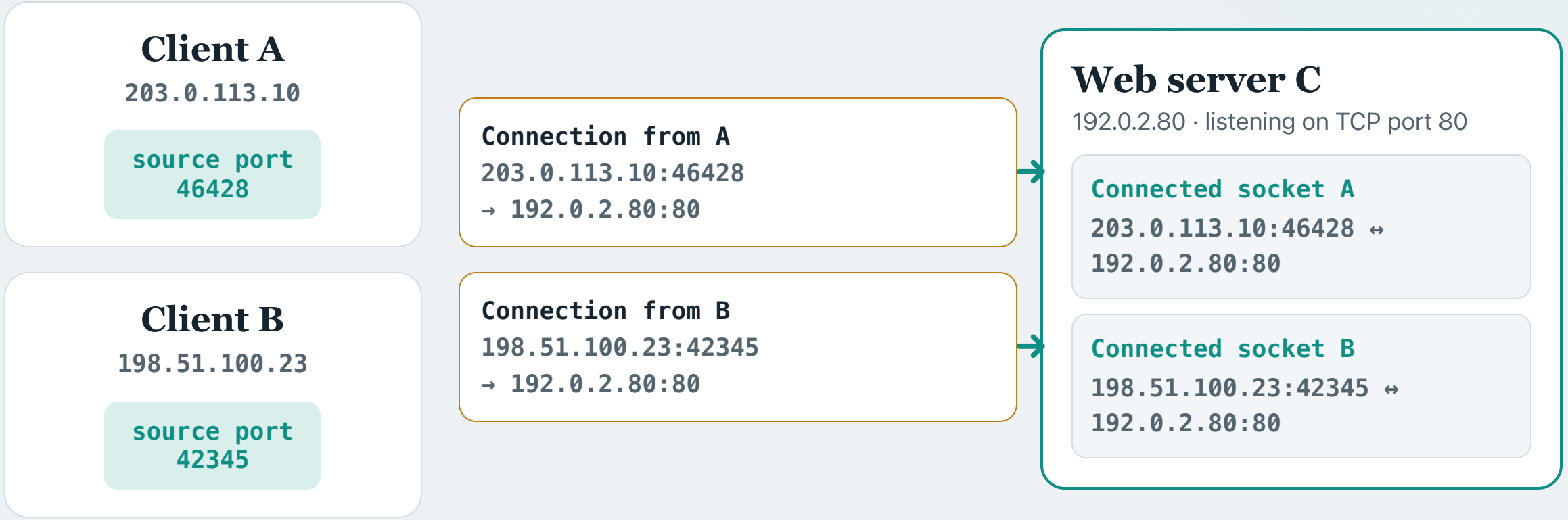
# One connected socket per client

- If several clients want to talk to the same server, each must first complete its own **handshake**.



# One connected socket per client

- If several clients want to talk to the same server, each must first complete its own **handshake**.
- After the handshake, each client has its own **connection** and **connected socket** on the server.
- Compare with **UDP**: being connectionless, it lets many clients share a single socket.



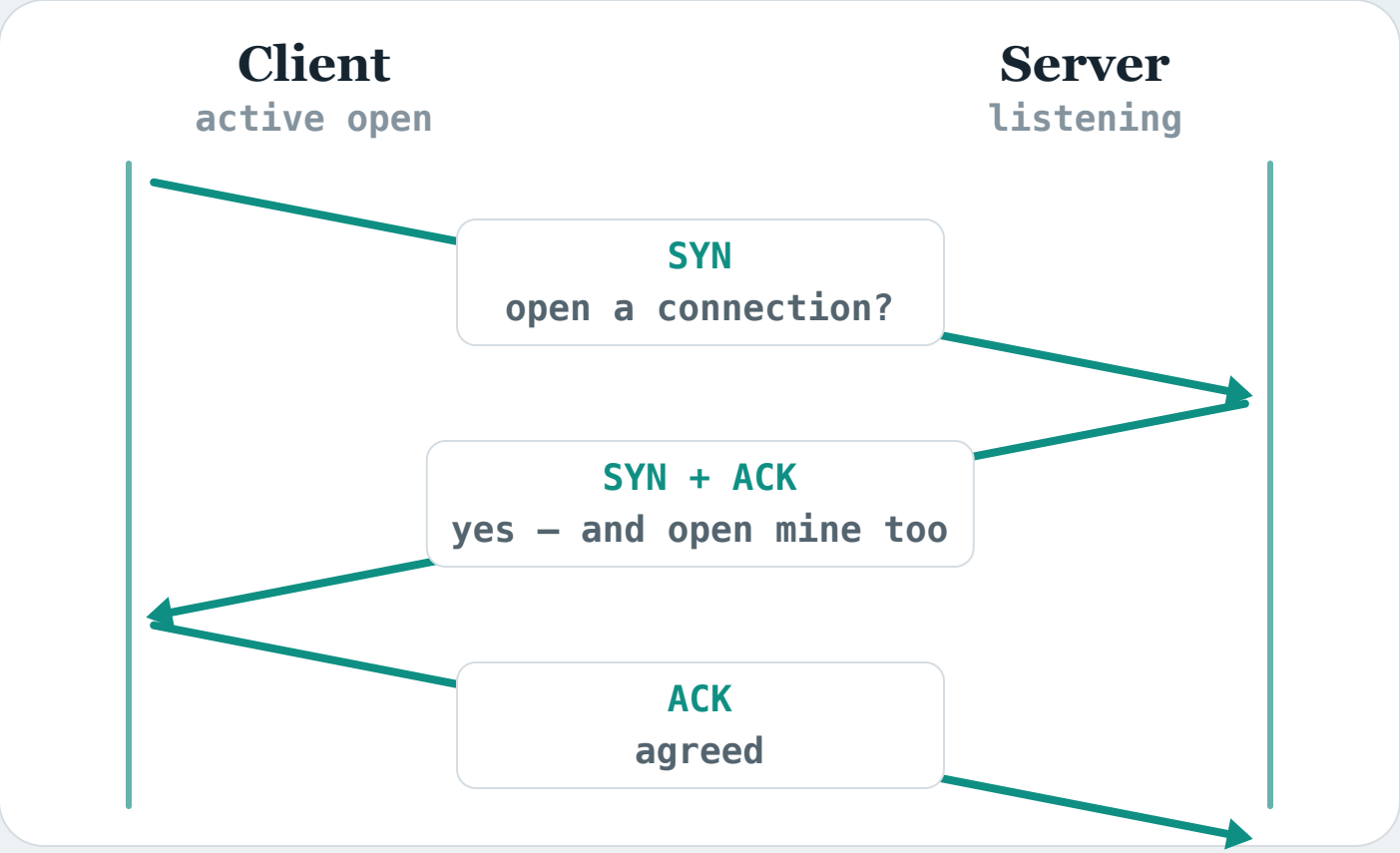
# TCP establishes a connection with a three-way handshake

**1 · SYN.** The client sends a segment with the **SYN** flag set, asking to open a conversation.

**2 · SYN + ACK.** The server agrees: in one segment it sets its own **SYN** flag and an **ACK** for the client's request.

**3 · ACK.** The client acknowledges the server's half, and the connection is open.

**Now that both sides have heard from each other, they can start sending packets to one another.**



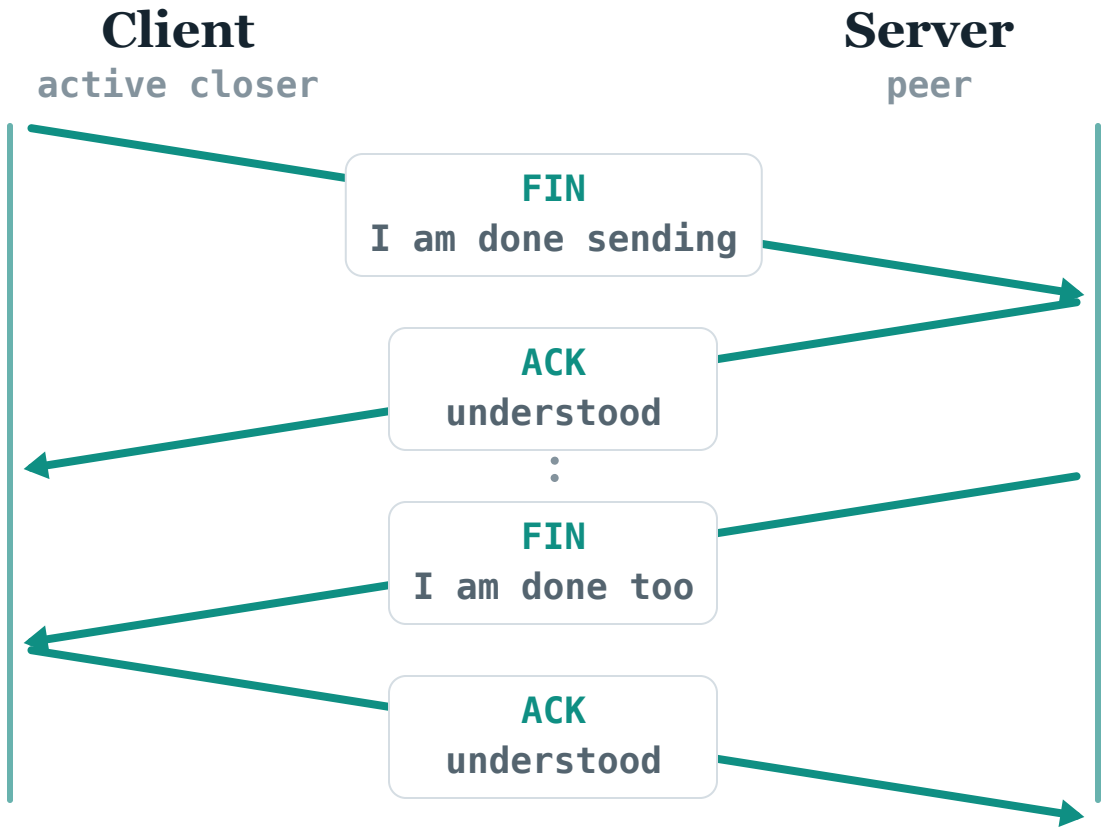
# How TCP closes a connection

**1 · FIN.** The client sets the **FIN** flag to say it has nothing more to send.

**2 · ACK.** The server acknowledges. But its own direction stays open, so it may still have things to send.

**3 · FIN.** When the server has finished too, it sets its own **FIN** flag.

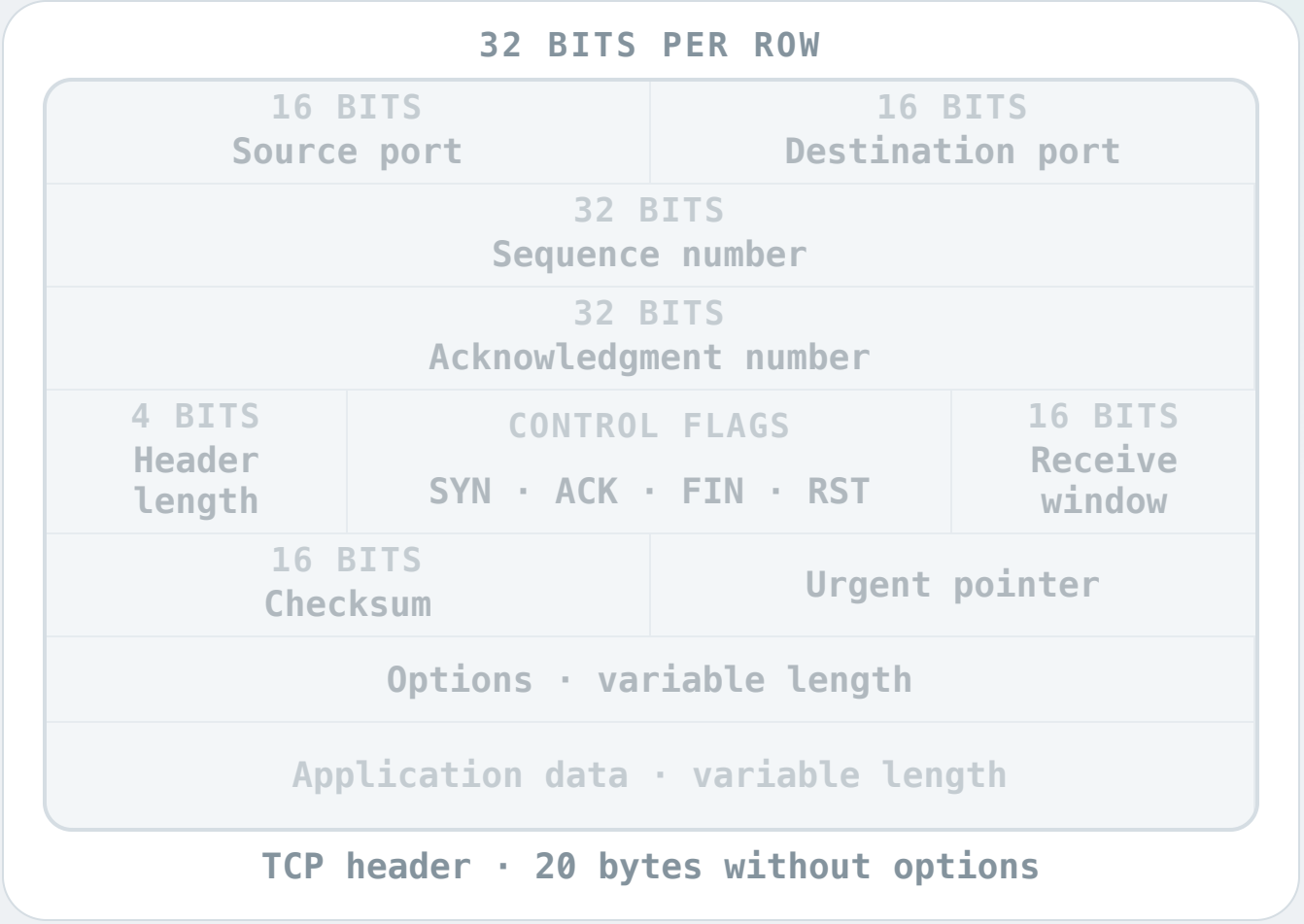
**4 · ACK.** The client acknowledges, and the connection is closed.



A COMPACT BREAKDOWN OF THE TCP HEADER

# Key TCP header fields

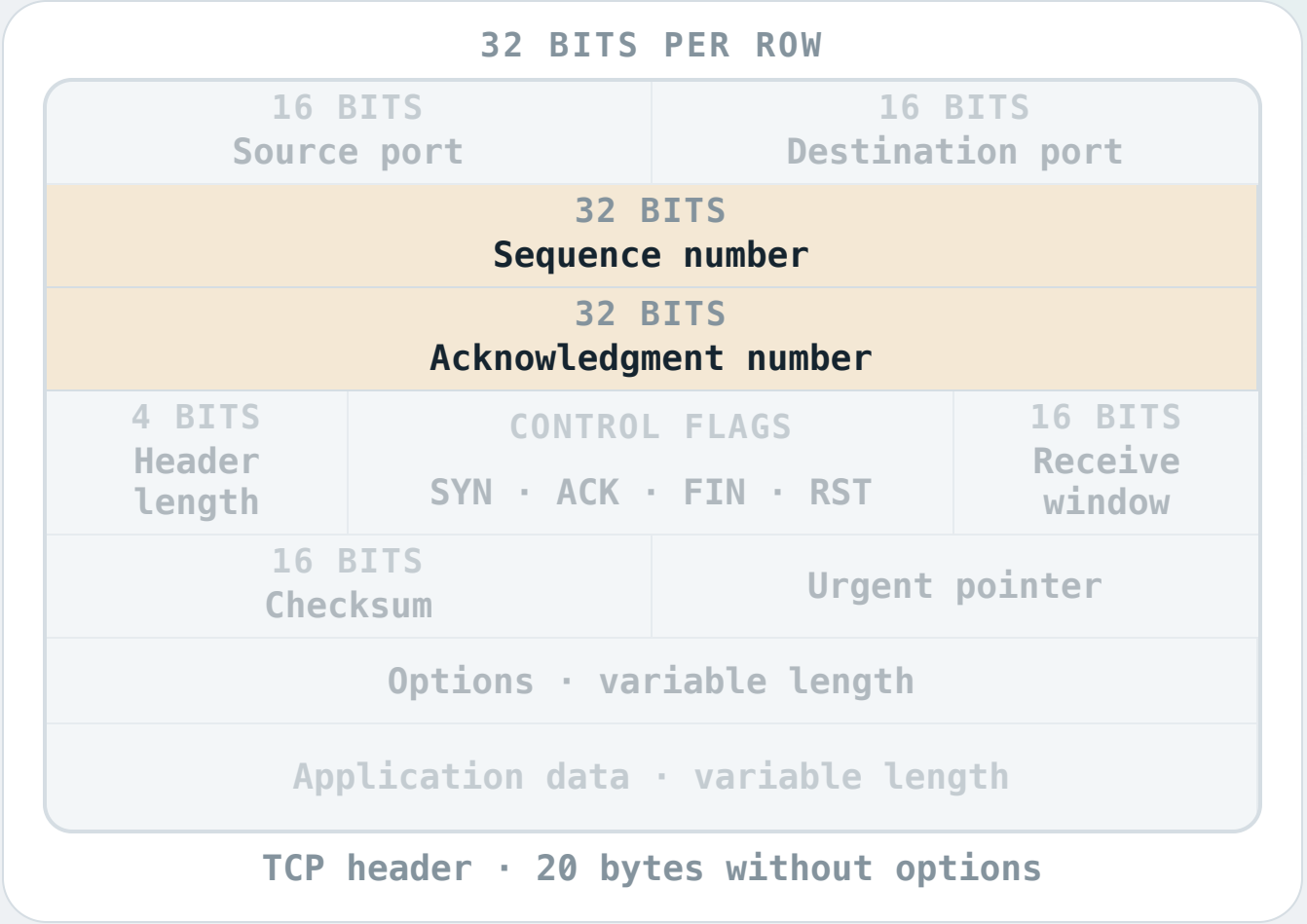
> **20 bytes, or more.** Without options a TCP header is 20 bytes, and options can make it longer.



A COMPACT BREAKDOWN OF THE TCP HEADER

# Key TCP header fields

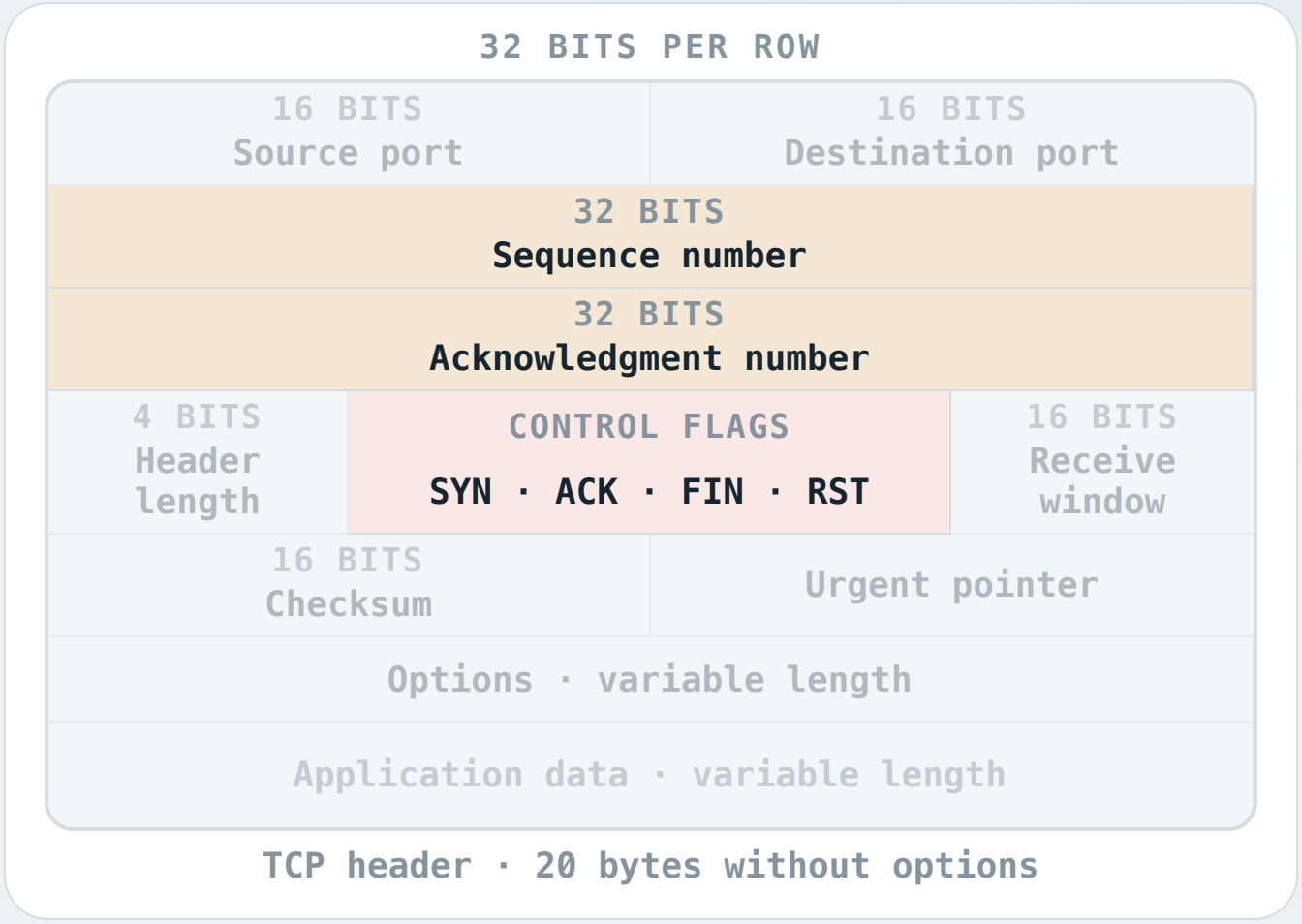
**Sequence and acknowledgment numbers.**
They identify the byte sent and the next byte expected. We come back to them next lecture.



A COMPACT BREAKDOWN OF THE TCP HEADER

# Key TCP header fields

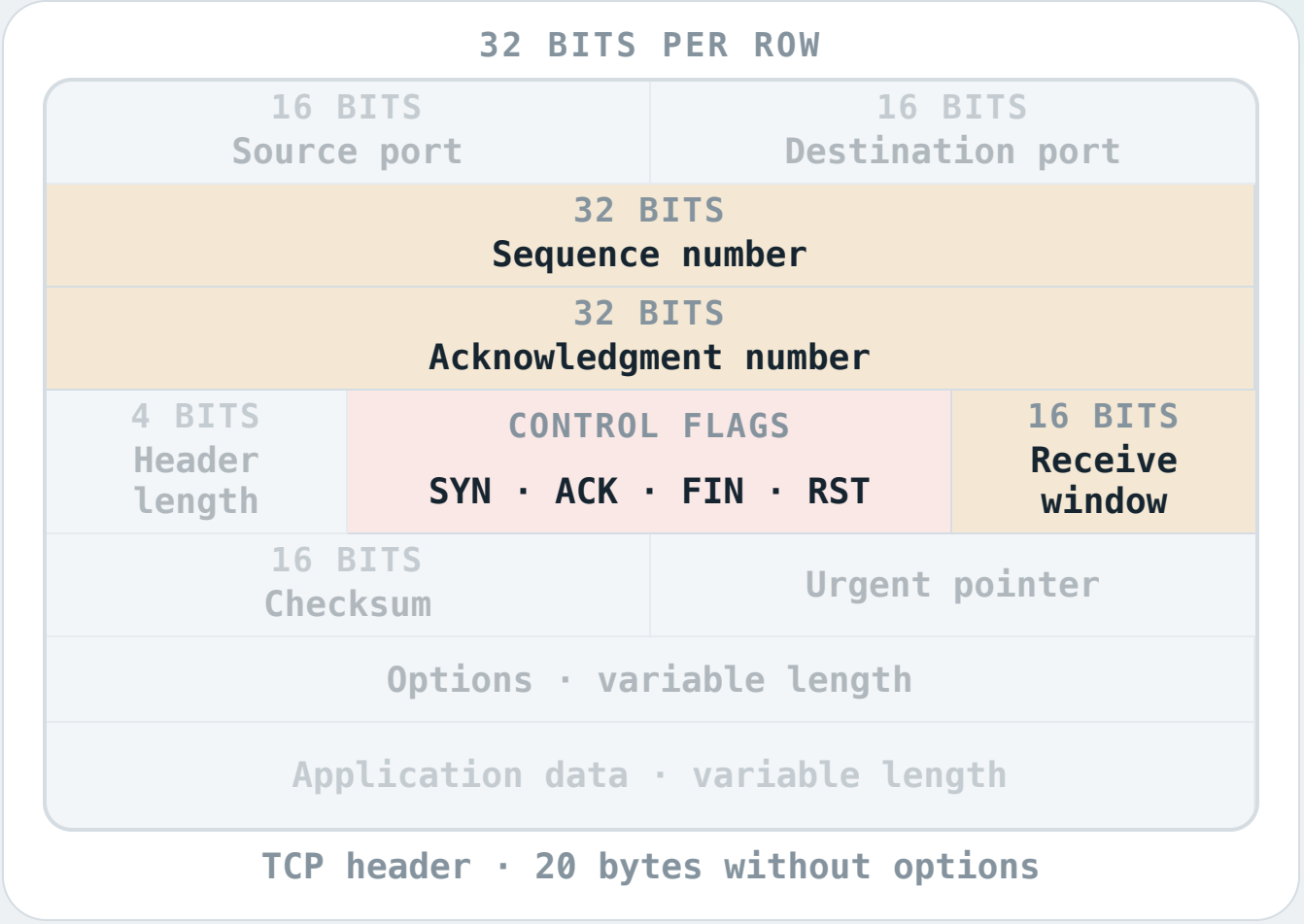
**Control flags.** Each is a one-bit field; a segment sets the relevant bit to 1 to signal SYN, ACK, FIN or RST.



A COMPACT BREAKDOWN OF THE TCP HEADER

# Key TCP header fields

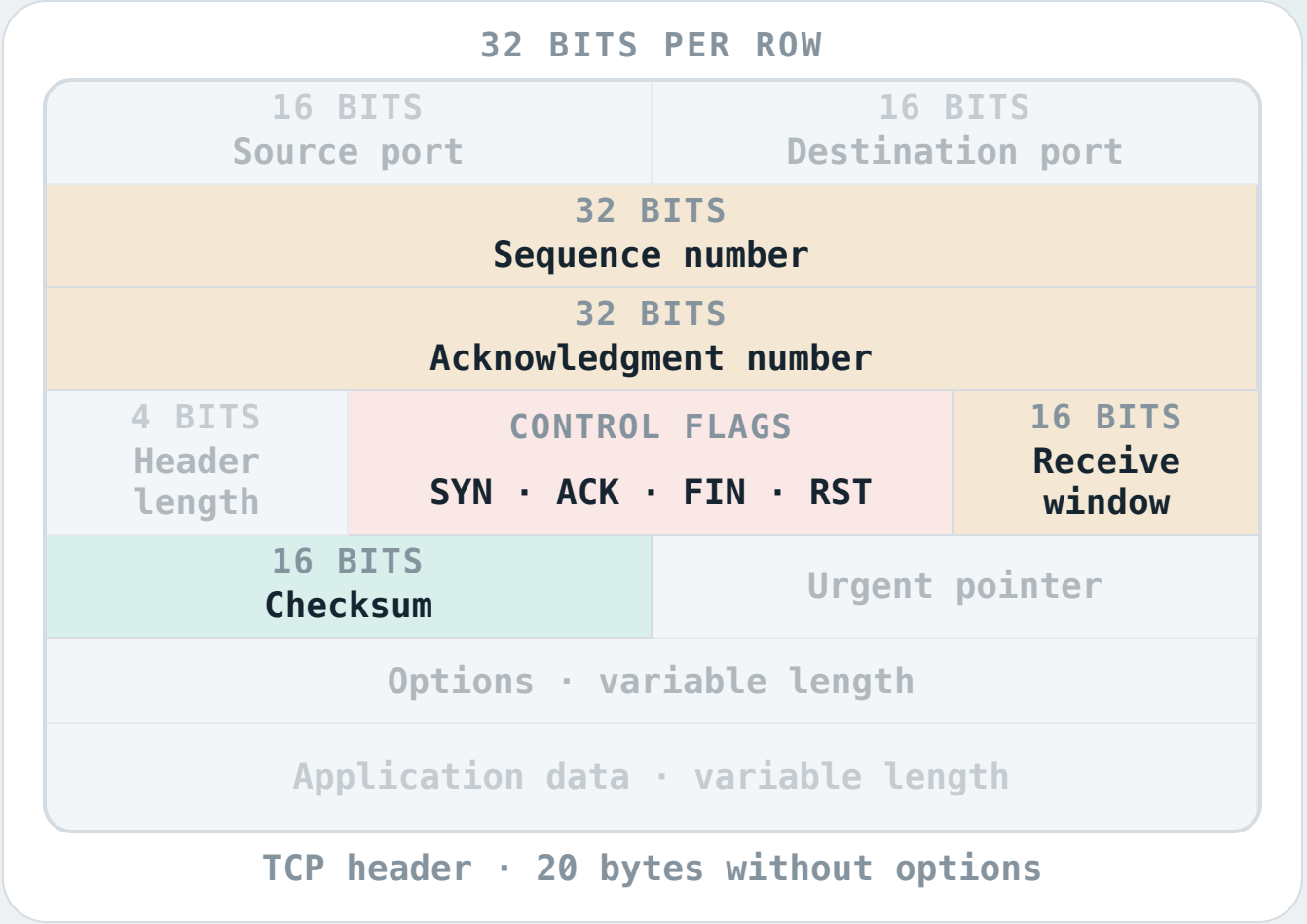
> **Receive window.** We come back to this when we reach flow control.



A COMPACT BREAKDOWN OF THE TCP HEADER

# Key TCP header fields

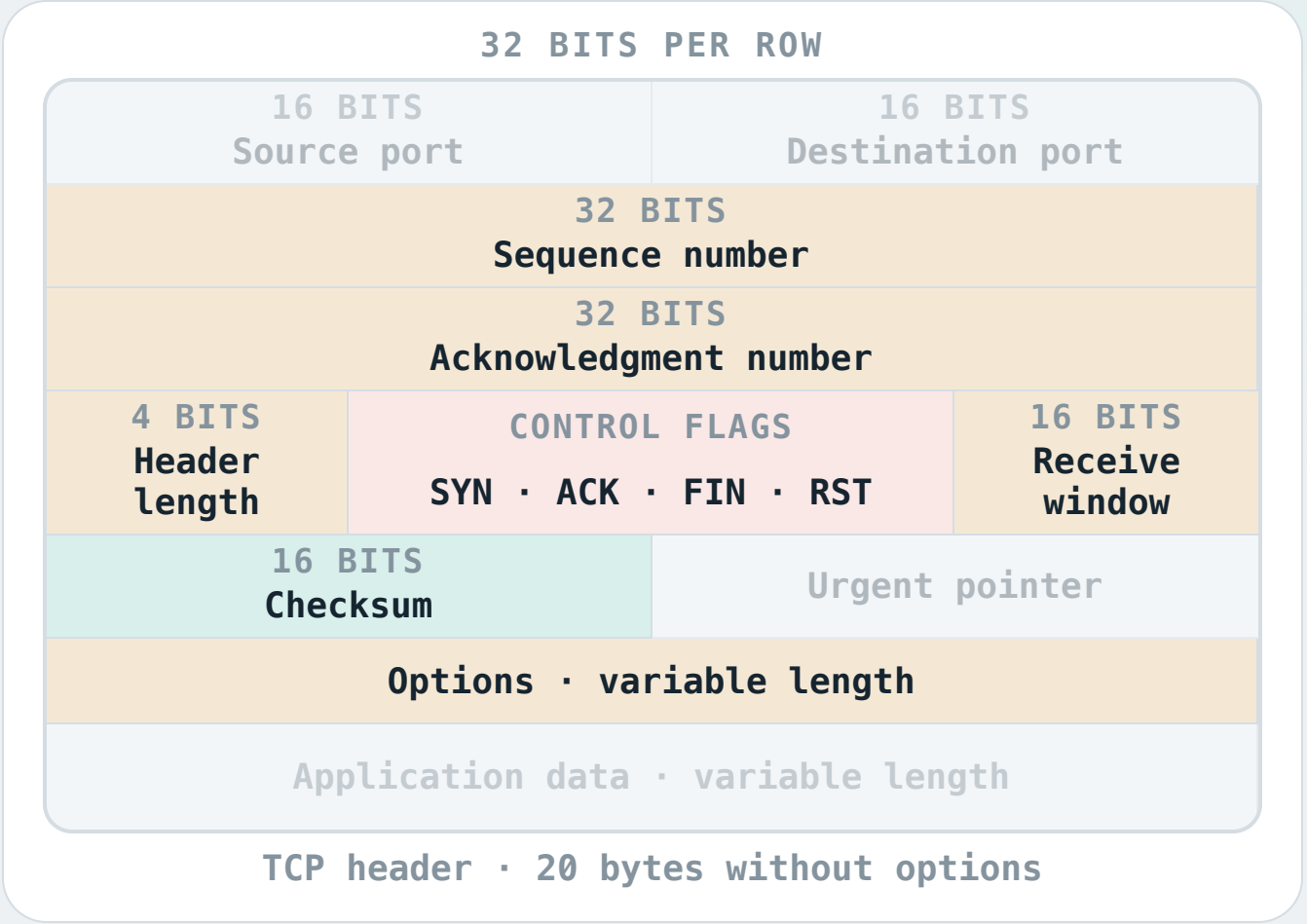
**Checksum.** It detects corruption in the TCP header and payload.



A COMPACT BREAKDOWN OF THE TCP HEADER

# Key TCP header fields

> **Header length and options.** Header length shows where the data begins; options carry extra capabilities such as MSS.



# What we have covered so far

- The transport layer gives two processes on different hosts a logical **end-to-end channel**, hiding the network below.
- A process sends and receives through a **socket**, which is bound to a **port number** on its host.
- IP only reaches the host, so the transport layer **multiplexes** data from many sockets into segments, and **demultiplexes** each arriving segment to the right socket.
  - UDP finds the socket by **destination IP address and port**; TCP by **four values** — source and destination IP addresses and ports.
- **UDP** is unreliable and connectionless: no setup, an 8-byte header (ports, length, checksum). Many senders can share one UDP socket.
- **TCP** is reliable and connection-oriented: a **three-way handshake** (SYN, SYN+ACK, ACK) opens a connection, and FIN/ACK in each direction closes it.
  - Each client gets its own **connected socket** on the server.
  - The TCP header (20+ bytes) adds sequence and ACK numbers, flags, a receive window and a checksum.

# The RDT used by TCP — an overview

TCP uses a reliable data transfer protocol on top of IP to provide reliable, in-order byte delivery.

Its RDT is inspired by both **Go-Back-N** and **Selective Repeat** — the pipelined protocols we studied in Week 10.

### Pipelining and cumulative acknowledgments

The TCP sender uses a **window** of segments in flight, for **pipelining**.

The TCP receiver **keeps a buffer**, as in Selective Repeat, and acknowledges with **cumulative ACKs**, as in Go-Back-N.

But unlike Go-Back-N — where the ACK names the last in-order segment received — TCP's ACK names the **next byte expected**.

### Loss recovery

Retransmission happens when the **timer times out**.

It also happens when **three duplicate ACKs** arrive (**fast retransmit**).

# TCP sequence numbers count bytes

### Byte-level addressing

In the RDT protocols from Week 10, each sequence number advanced by one **packet** — between consecutive sender messages, the sequence number advanced by one.

In TCP, sequence numbers address individual **bytes** — so between consecutive sender segments, the sequence number advances by the **number of bytes** in the segment.

### Maximum Segment Size (MSS)

The largest payload a single TCP segment can carry — negotiated during the handshake. Typical value: **1460 bytes** on Ethernet (1500 MTU − 20 IP − 20 TCP).

### SYN and FIN consume a sequence number

SYN and FIN each consume exactly one sequence number, even though they carry no application data.

**Why do they need a sequence number?**

So that they can be **reliably acknowledged**: the ACK for a SYN or FIN names its sequence number + 1, and an unacknowledged SYN or FIN is retransmitted.

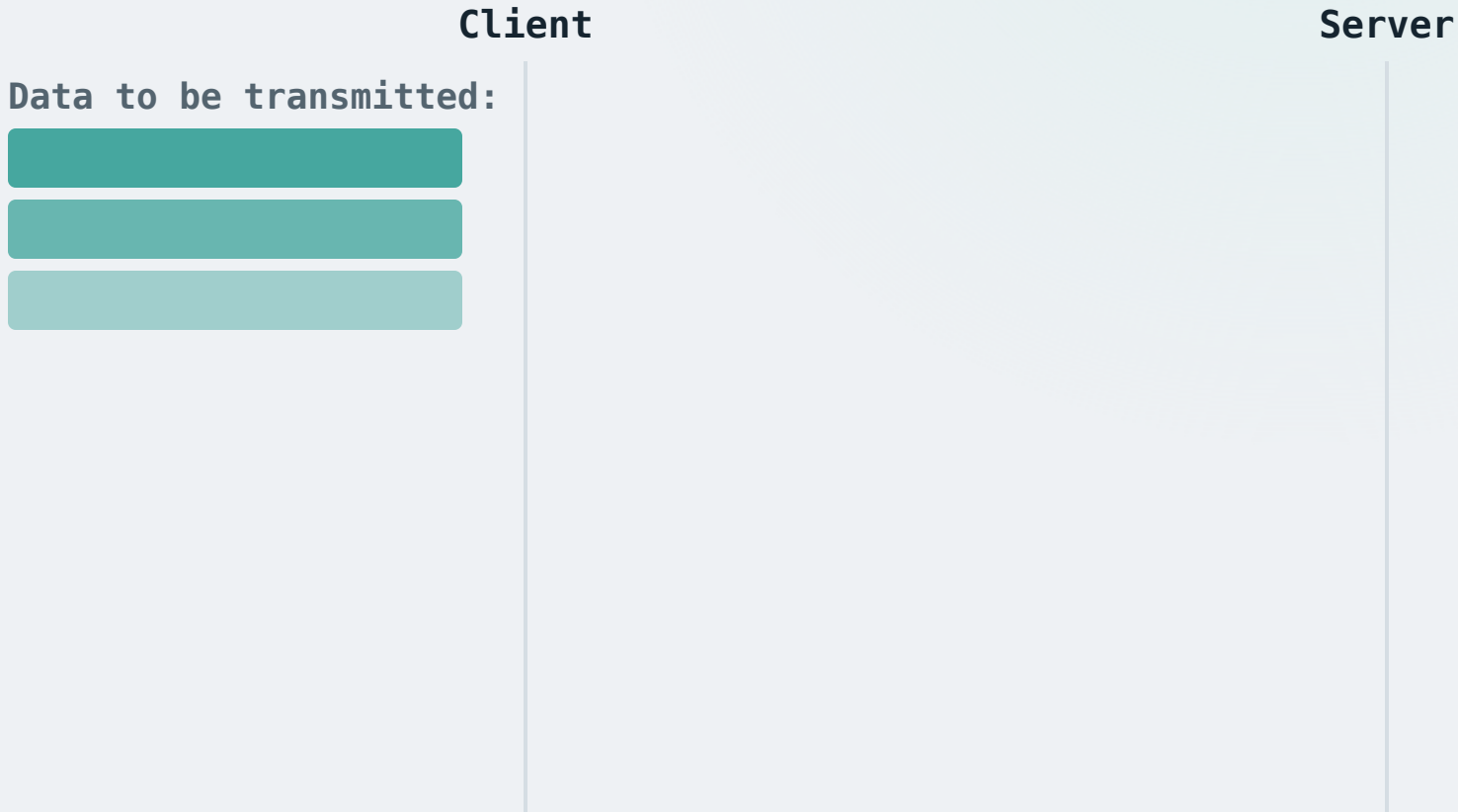
MSS = 1000 BYTES · 3000 BYTES TO SEND

# Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

**The setup**

With MSS = 1000 bytes, the 3000 bytes of data fill **three segments**.



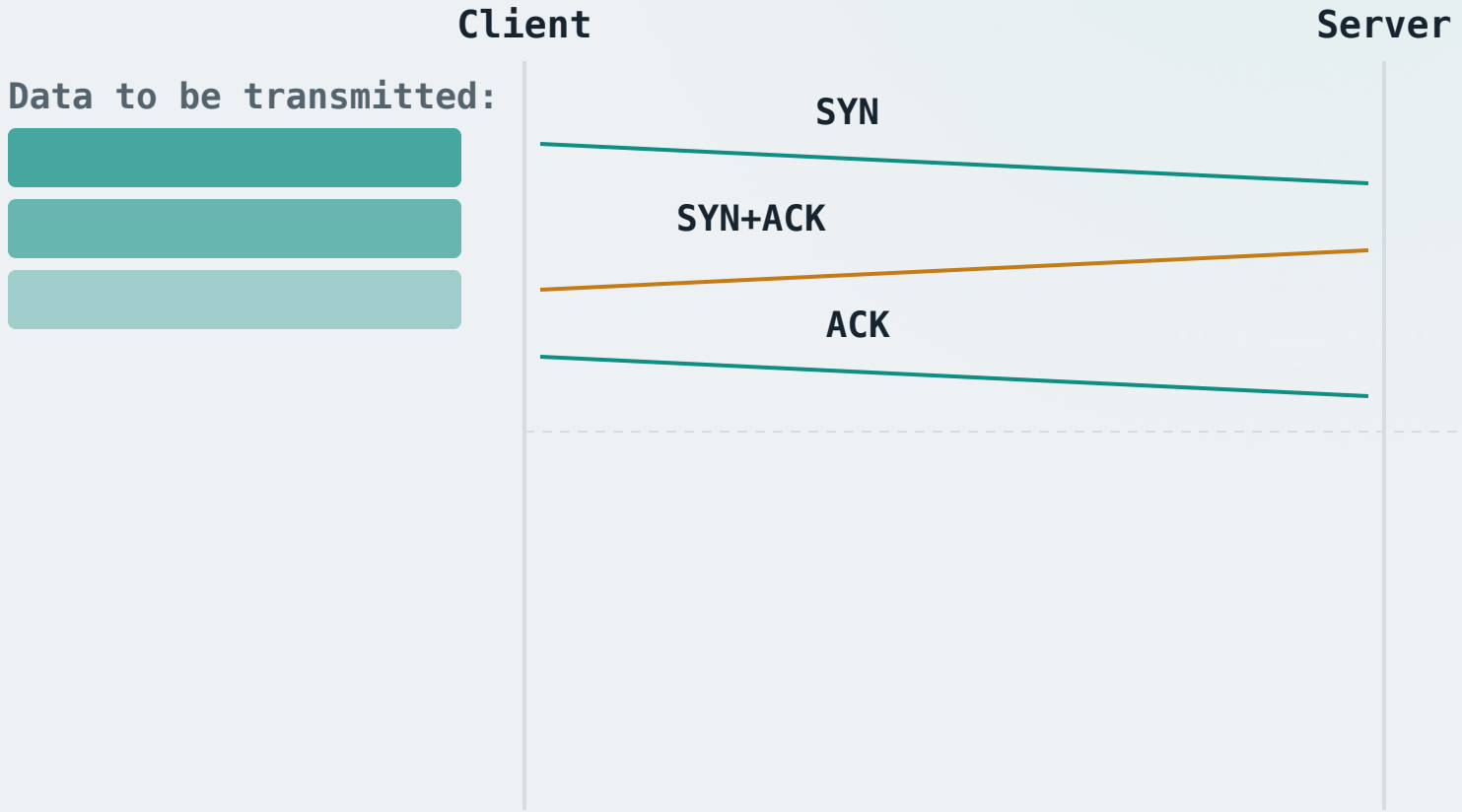
MSS = 1000 BYTES · 3000 BYTES TO SEND

# Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

### Three-way handshake

First, the client and server perform a three-way handshake to open the TCP connection.



MSS = 1000 BYTES · 3000 BYTES TO SEND

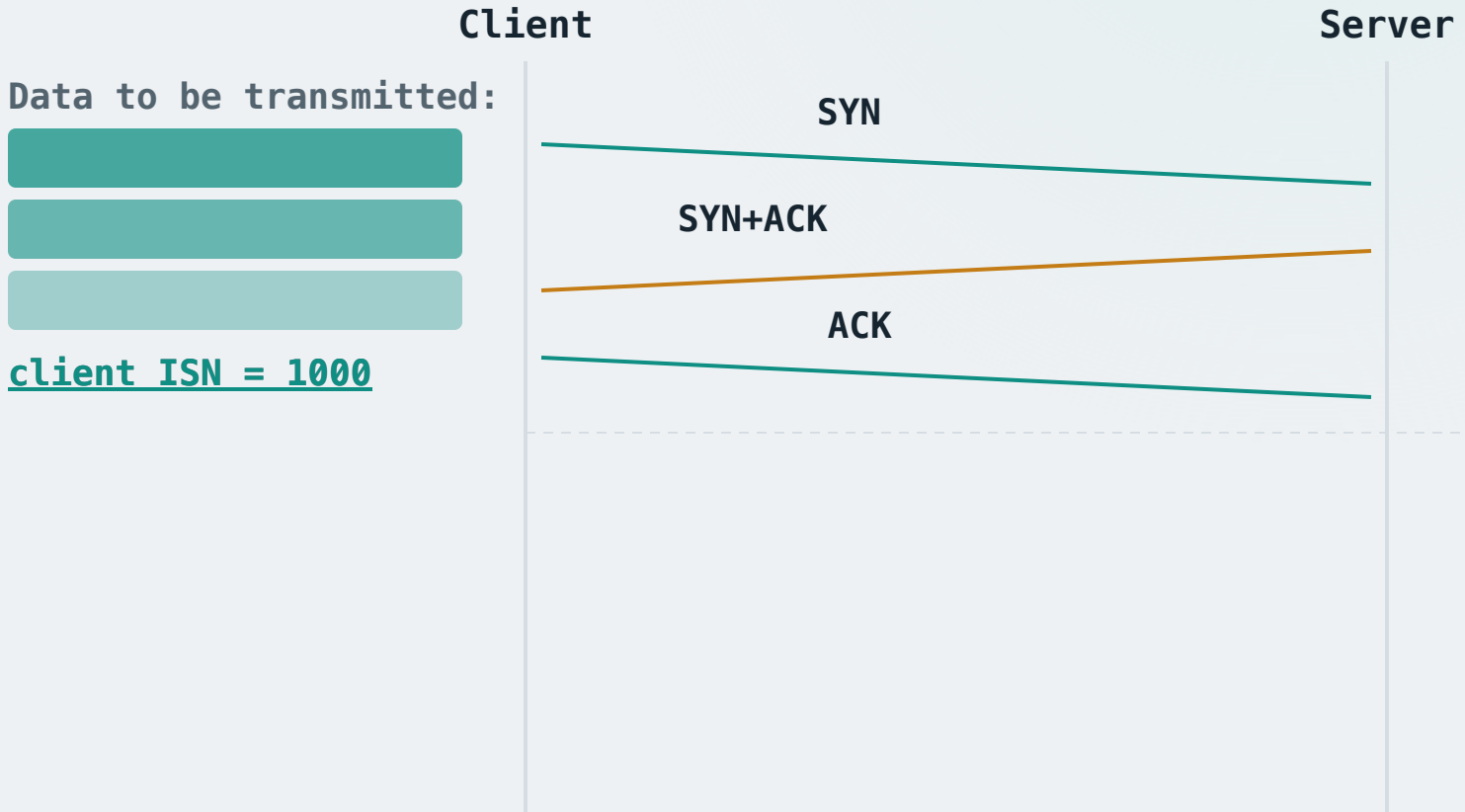
# Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

### Three-way handshake

**Each of these messages carries a sequence number. Where does it start?**

Each side picks an **initial sequence number (ISN)**: the number its first segment carries, typically chosen at random. Say the client's ISN is **1000**.



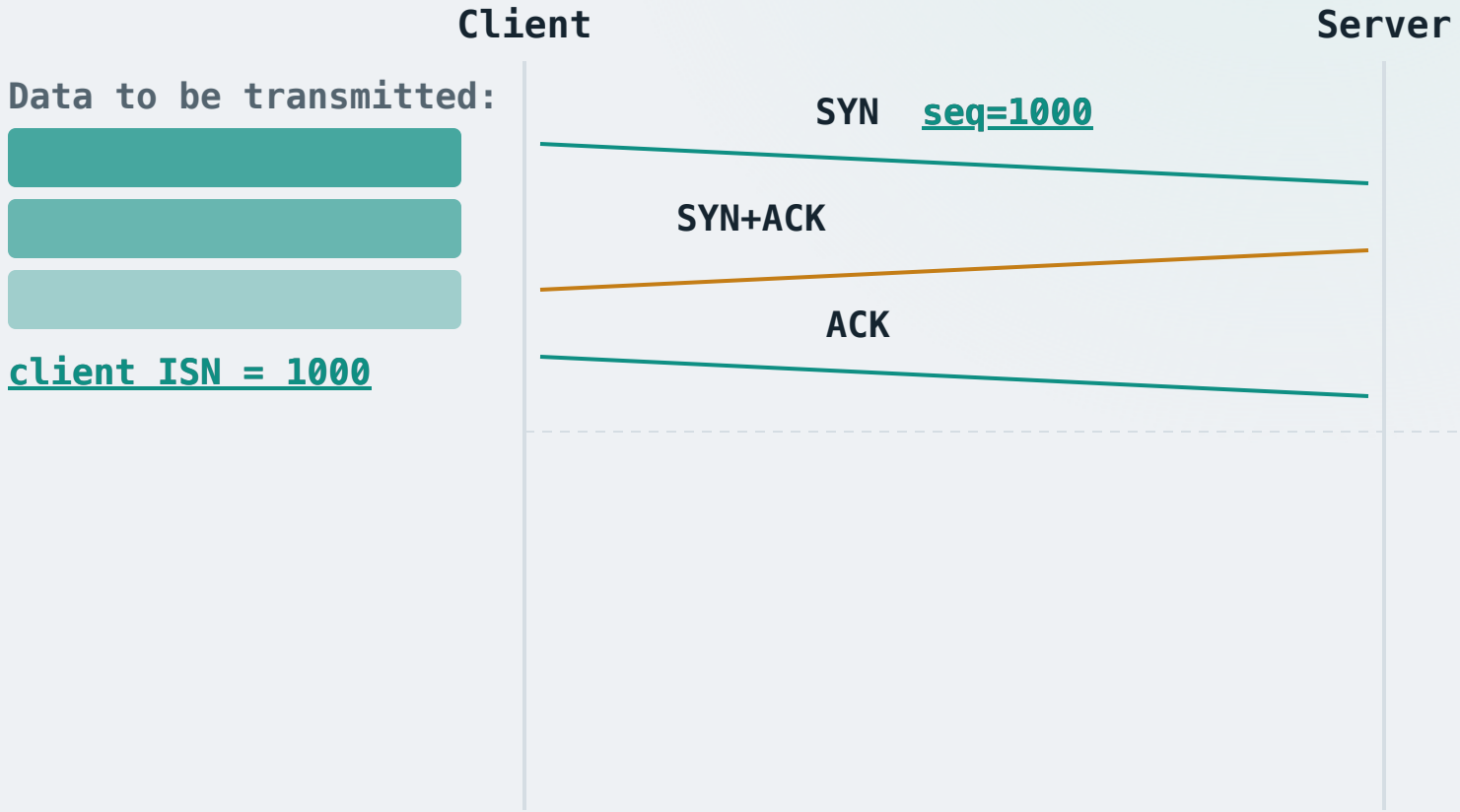
MSS = 1000 BYTES · 3000 BYTES TO SEND

# Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

**Three-way handshake**

The client's **SYN** carries its ISN: **seq = 1000**.



MSS = 1000 BYTES · 3000 BYTES TO SEND

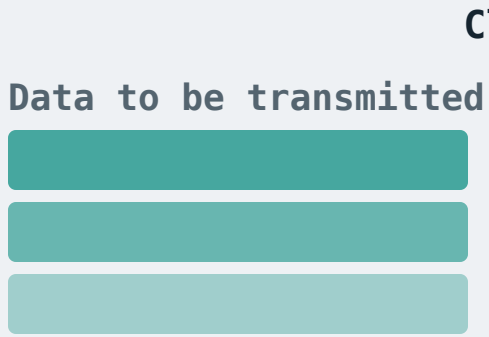
# Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

**Three-way handshake**

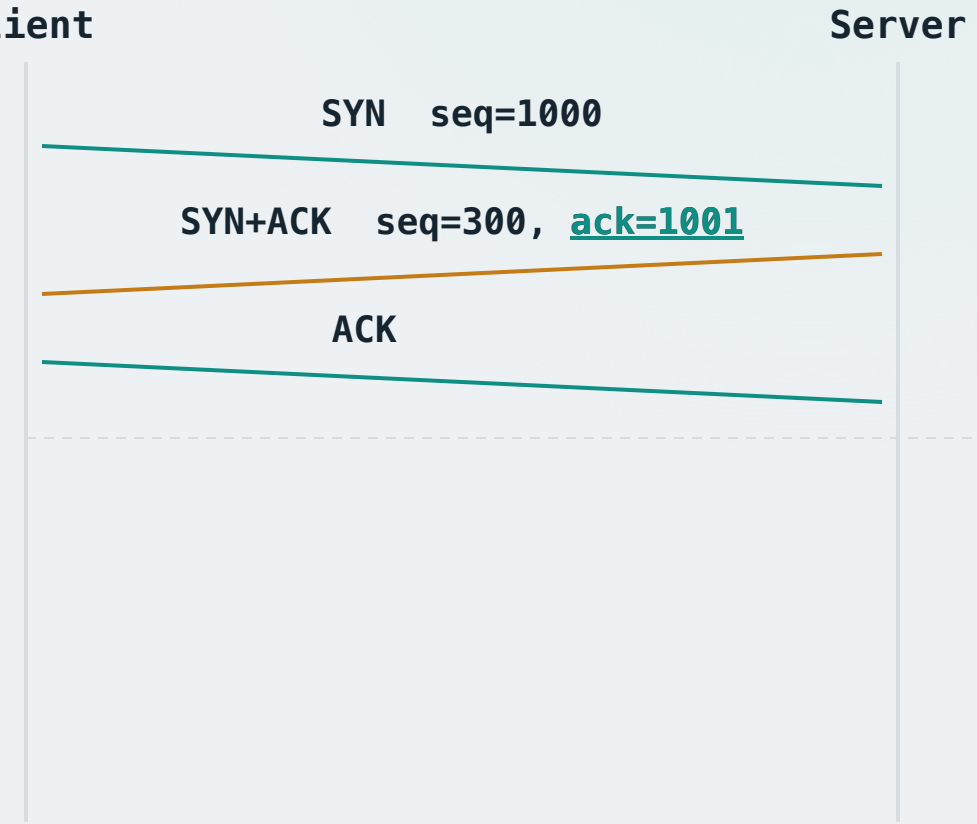
The server replies **SYN+ACK**, carrying its own ISN, **seq = 300**, and **ack = 1001**.

**1001, not 1000**: an ACK names the **next** sequence number expected, not the last one received.



Data to be transmitted:

client ISN = 1000
server ISN = 300



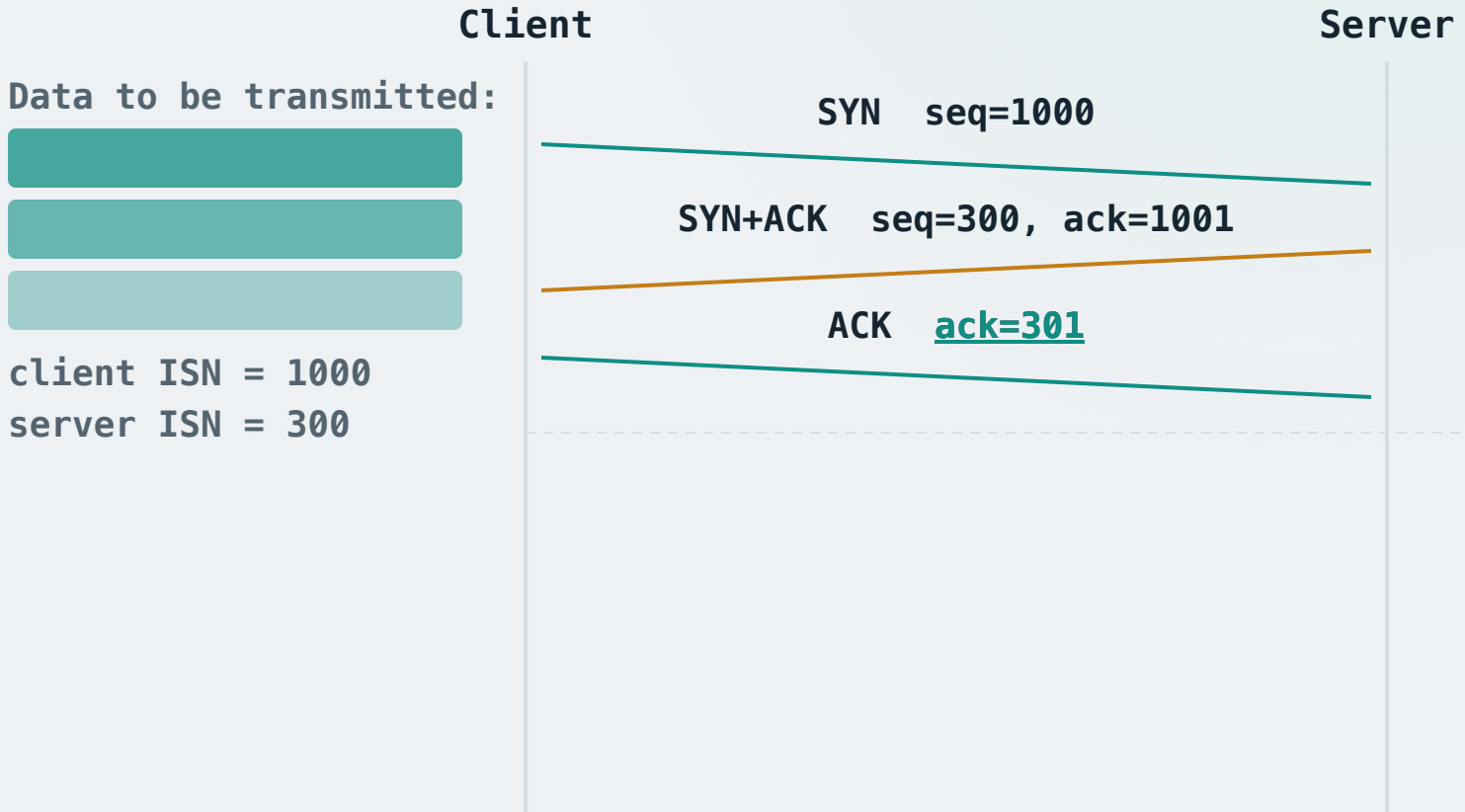
MSS = 1000 BYTES · 3000 BYTES TO SEND

# Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

**Three-way handshake**

The client acknowledges the server's ISN with **ack = 301**. The three-way handshake is complete.



MSS = 1000 BYTES · 3000 BYTES TO SEND

# Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

### Three-way handshake

The client's next sequence number is **1001**, so its first data byte is byte 1001 — exactly what the server's **ack = 1001** says it expects next.

Data to be transmitted:

- 1001 – 2000
- 2001 – 3000
- 3001 – 4000

client ISN = 1000
server ISN = 300



MSS = 1000 BYTES · 3000 BYTES TO SEND

# Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

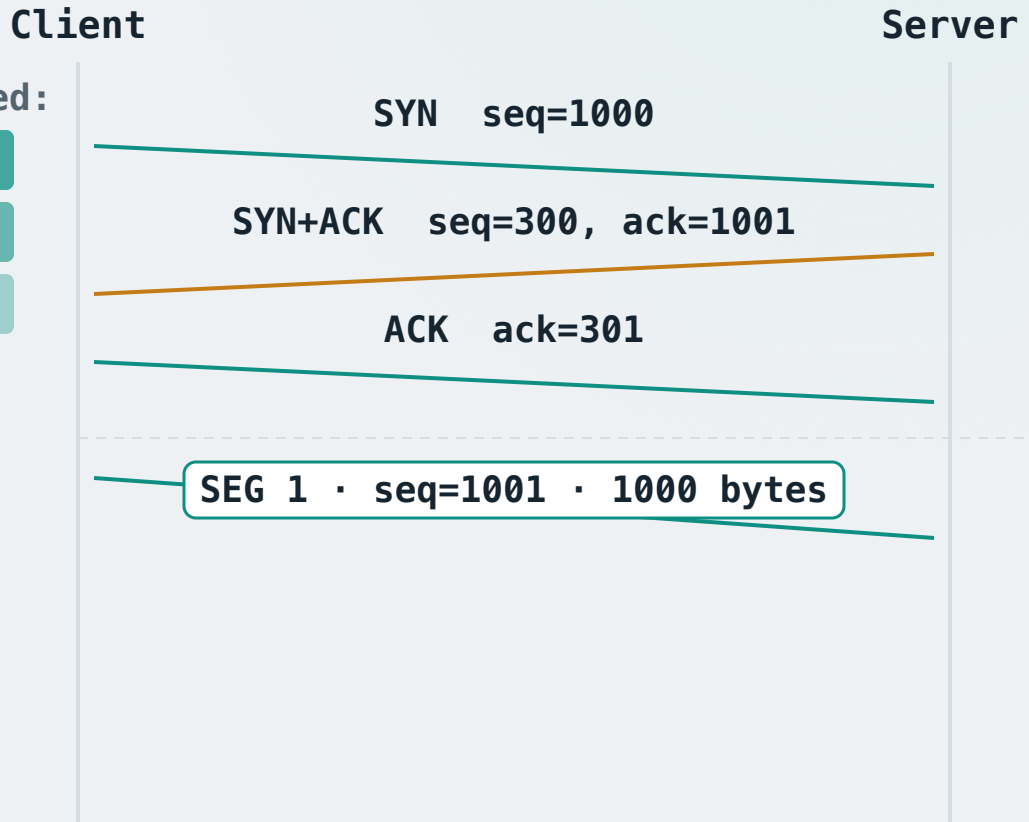
**Transmit segment 1**

The client sends the first 1000 bytes (seq = 1001).

Data to be transmitted:

- 1001 – 2000
- 2001 – 3000
- 3001 – 4000

client ISN = 1000
server ISN = 300



MSS = 1000 BYTES · 3000 BYTES TO SEND

# Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

**Transmit segment 1**

The client sends the first 1000 bytes (seq = 1001).

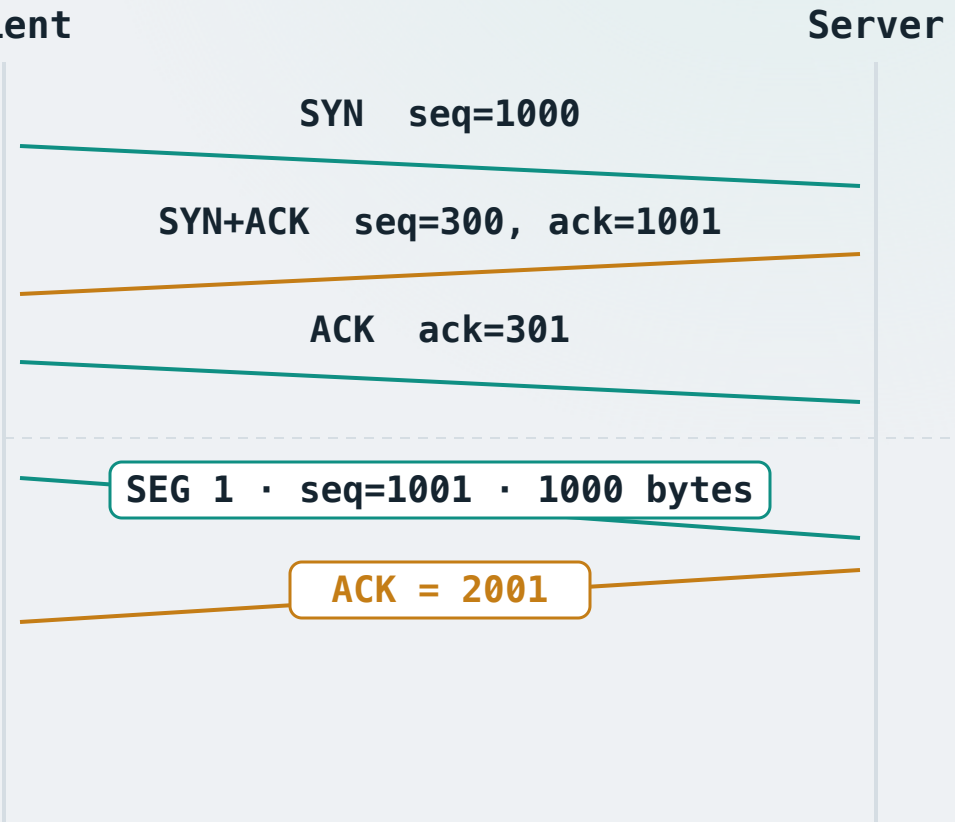
The server cumulatively acknowledges with **ACK = 2001** — it has every byte up to 2000 and expects byte 2001 next.

Data to be transmitted:

2001 – 3000

3001 – 4000

client ISN = 1000
server ISN = 300



MSS = 1000 BYTES · 3000 BYTES TO SEND

# Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

**Transmit segment 2**

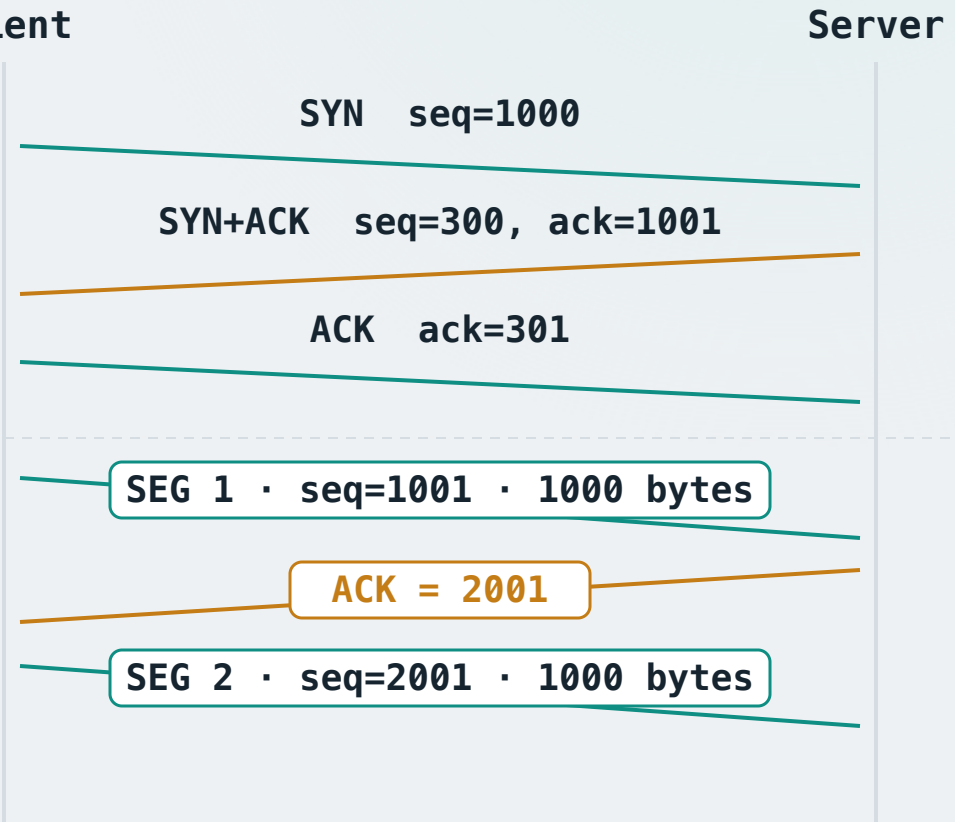
The client sends the next 1000 bytes (seq = 2001).

Data to be transmitted:

2001 – 3000

3001 – 4000

client ISN = 1000
server ISN = 300



MSS = 1000 BYTES · 3000 BYTES TO SEND

# Sequence and acknowledgment in action

The client wants to transmit 3000 bytes of application data over a TCP connection.

**Transmit segment 2**

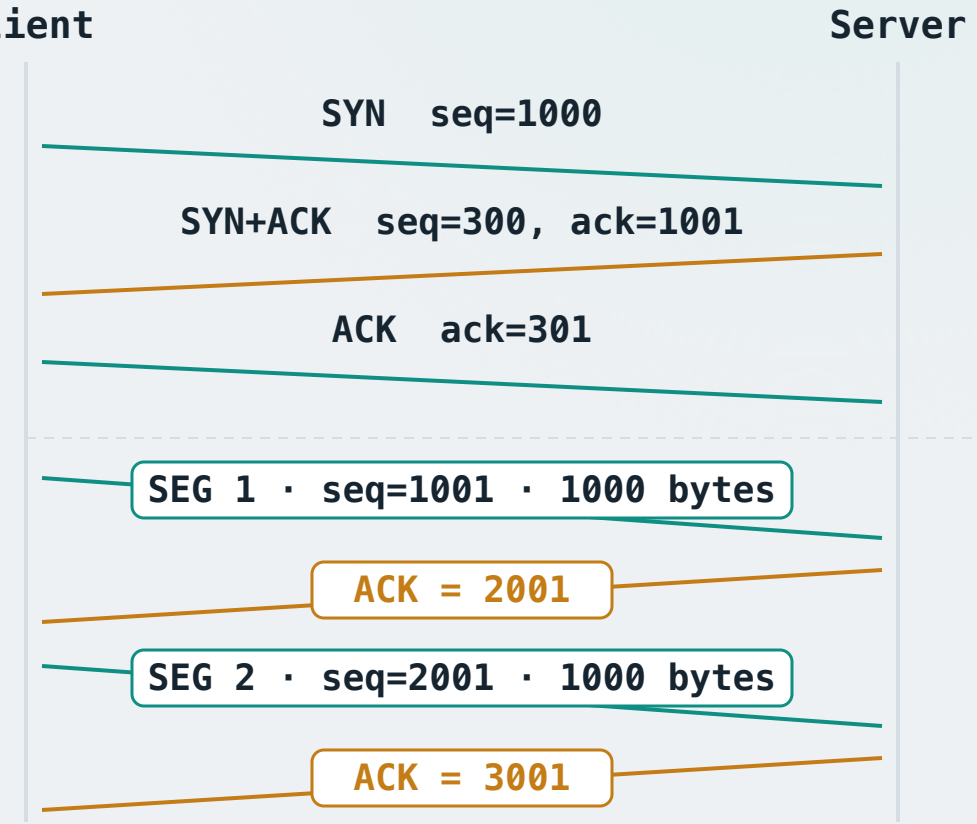
The client sends the next 1000 bytes (seq = 2001).

The server cumulatively acknowledges with **ACK = 3001** — confirming every byte up to 3000.

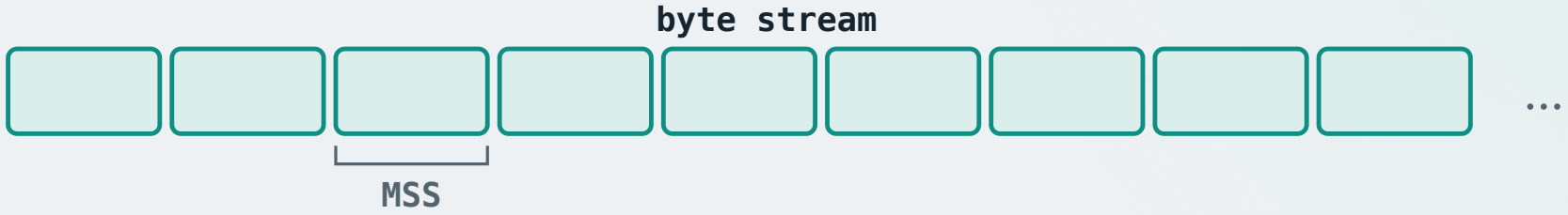
Data to be transmitted:

3001 – 4000

client ISN = 1000
server ISN = 300



# The TCP sender rules: overview




**Data from application**

Divide the byte stream into segments, typically **1 MSS** bytes each.

# The TCP sender rules: overview

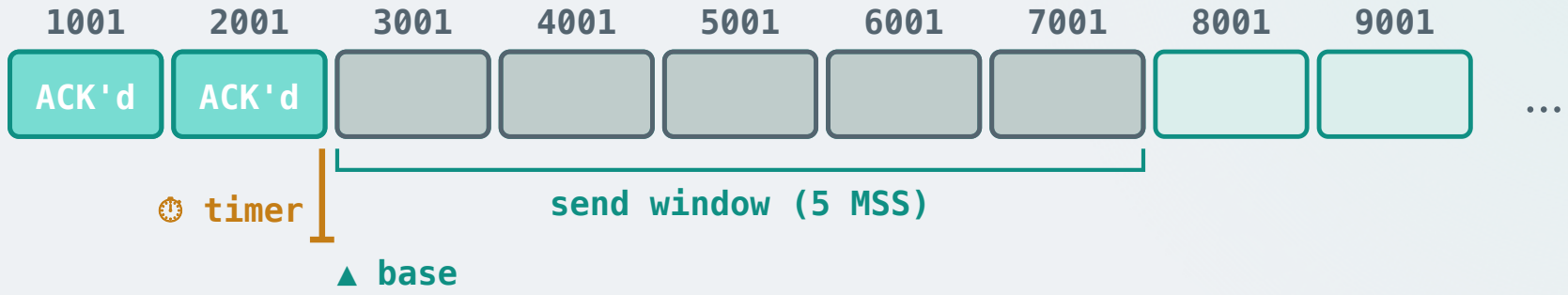



**Data from application**

Divide the byte stream into segments, typically **1 MSS** bytes each.

Number each segment by its first byte, counting on from the **initial sequence number (ISN)** chosen in the handshake.
*e.g. ISN = 1000 and 1 MSS = 1000 bytes, so the segments start at 1001, 2001, 3001, ...*

# The TCP sender rules: overview




## Send window

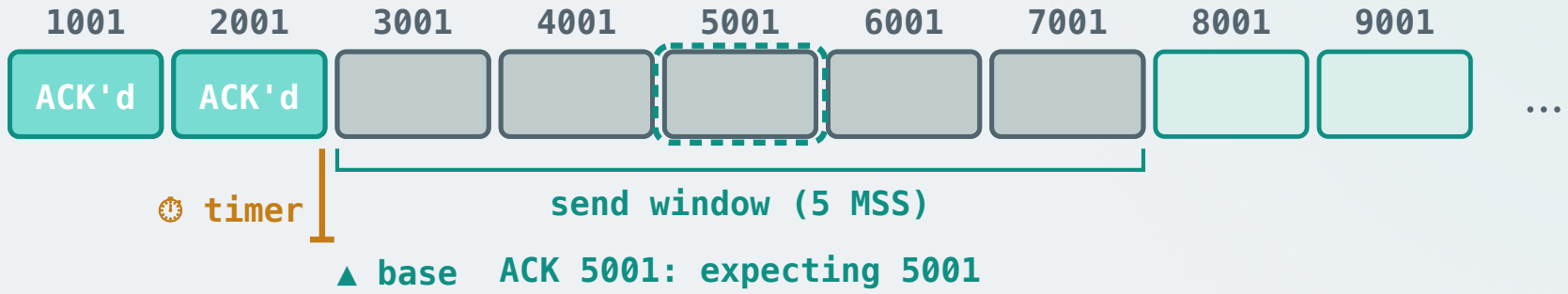
As in any pipelined RDT, the sender keeps a **send window**.
*e.g. a window of 5 MSS, i.e. 5 segments.*

The **window base** is the oldest unacknowledged segment.

Suppose the first two segments have already been acknowledged, so the window base starts at the third segment, seq 3001. The sender may send anything within the window, so segments 3001–7001 go out.

Then start **one timer**, if none is running. As in **Go-Back-N**, this single timer runs for the oldest unacknowledged segment, at the window base, and covers the entire window.

# The TCP sender rules: overview

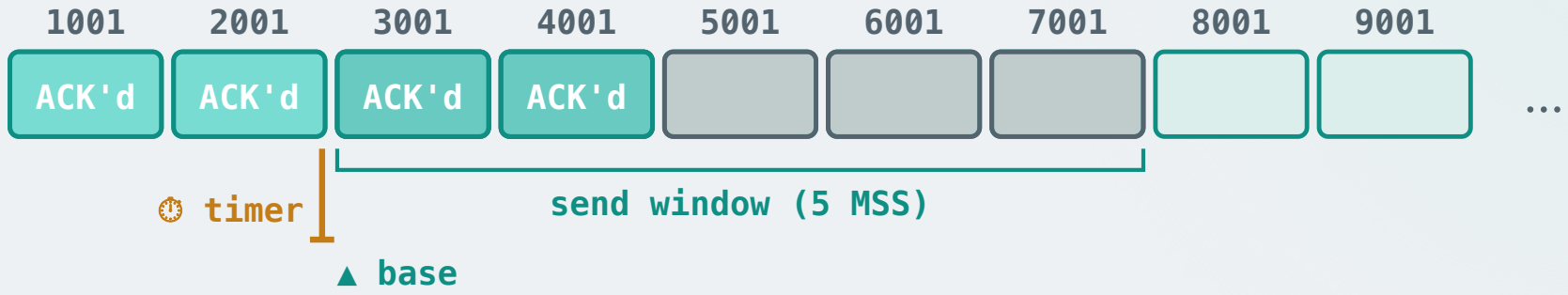



### ACK received

A cumulative ACK slides the window forward.

*e.g. the sender receives ACK 5001: the receiver is expecting the segment starting at 5001.*

# The TCP sender rules: overview



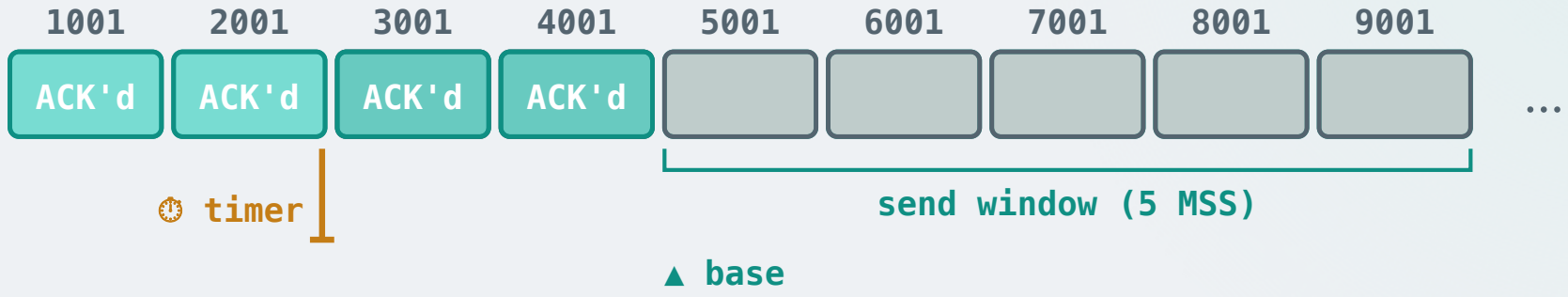

### ACK received

A cumulative ACK slides the window forward.

*e.g. the sender receives ACK 5001: the receiver is expecting the segment starting at 5001.*

*Since ACKs are cumulative, segs 3001 and 4001 are both acknowledged.*

# The TCP sender rules: overview




**ACK received**

A cumulative ACK slides the window forward.

*e.g. the sender receives ACK 5001: the receiver is expecting the segment starting at 5001.*

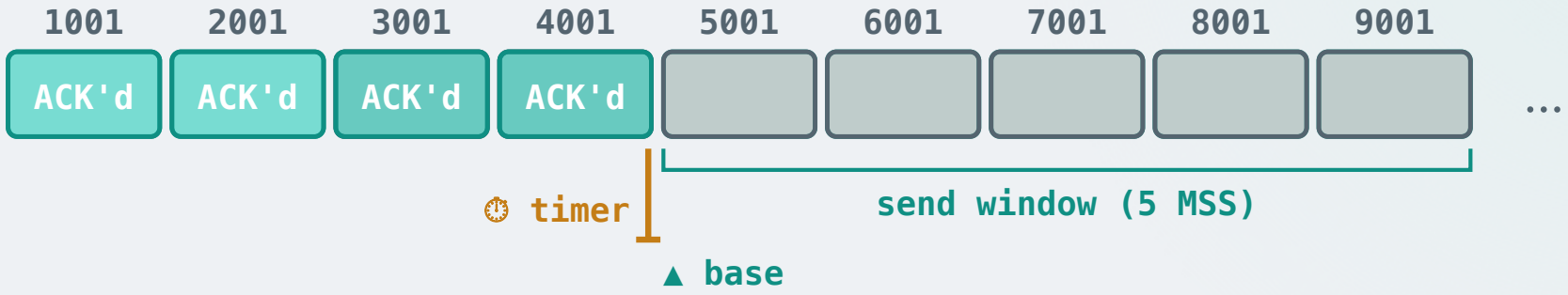
*Since ACKs are cumulative, segs 3001 and 4001 are both acknowledged.*

*So the window base moves to 5001, and the window now covers 5001–9001.*

*Segs 8001 and 9001 can now go out.*

Restart the timer if unacknowledged data remains.

# The TCP sender rules: overview




**ACK received**

A cumulative ACK slides the window forward.

*e.g. the sender receives ACK 5001: the receiver is expecting the segment starting at 5001.*

*Since ACKs are cumulative, segs 3001 and 4001 are both acknowledged.*

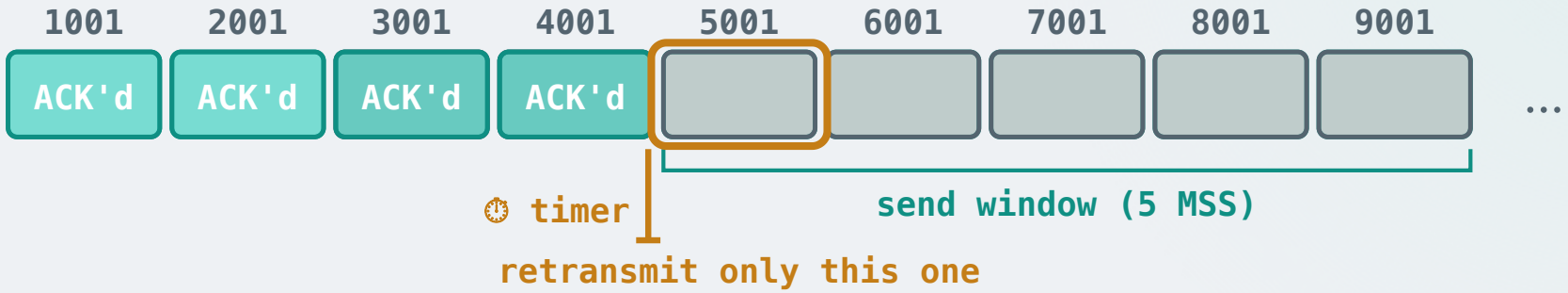
*So the window base moves to 5001, and the window now covers 5001–9001.*

*Segs 8001 and 9001 can now go out.*

Restart the timer if unacknowledged data remains.

*In this case, the timer restarts on 5001.*

# The TCP sender rules: overview



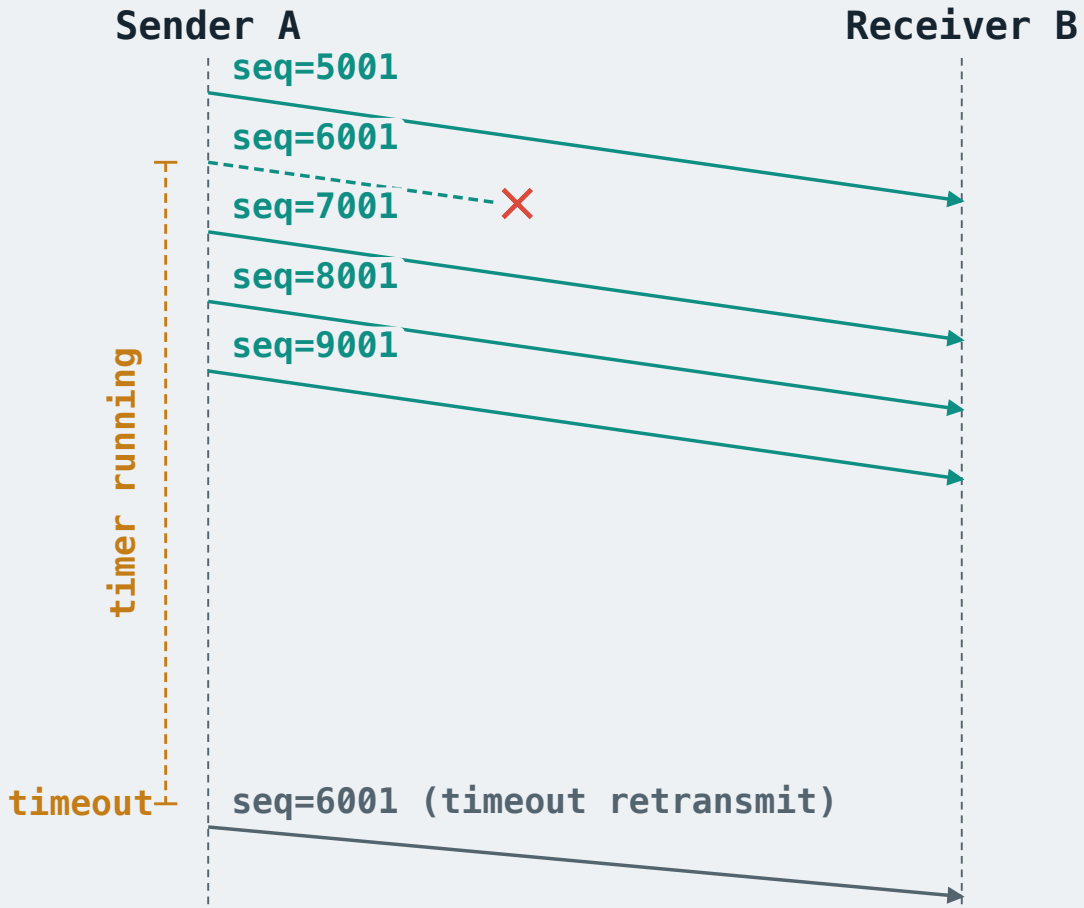

### Timeout

Retransmit only the **oldest unacknowledged segment**, then restart the timer.

This is different from **Go-Back-N**. Recall that in GBN, the sender retransmits *every* in-flight packet in the window; TCP retransmits only *one*.

*e.g. here GBN would retransmit all five segs, 5001–9001; TCP retransmits only seg 5001.*

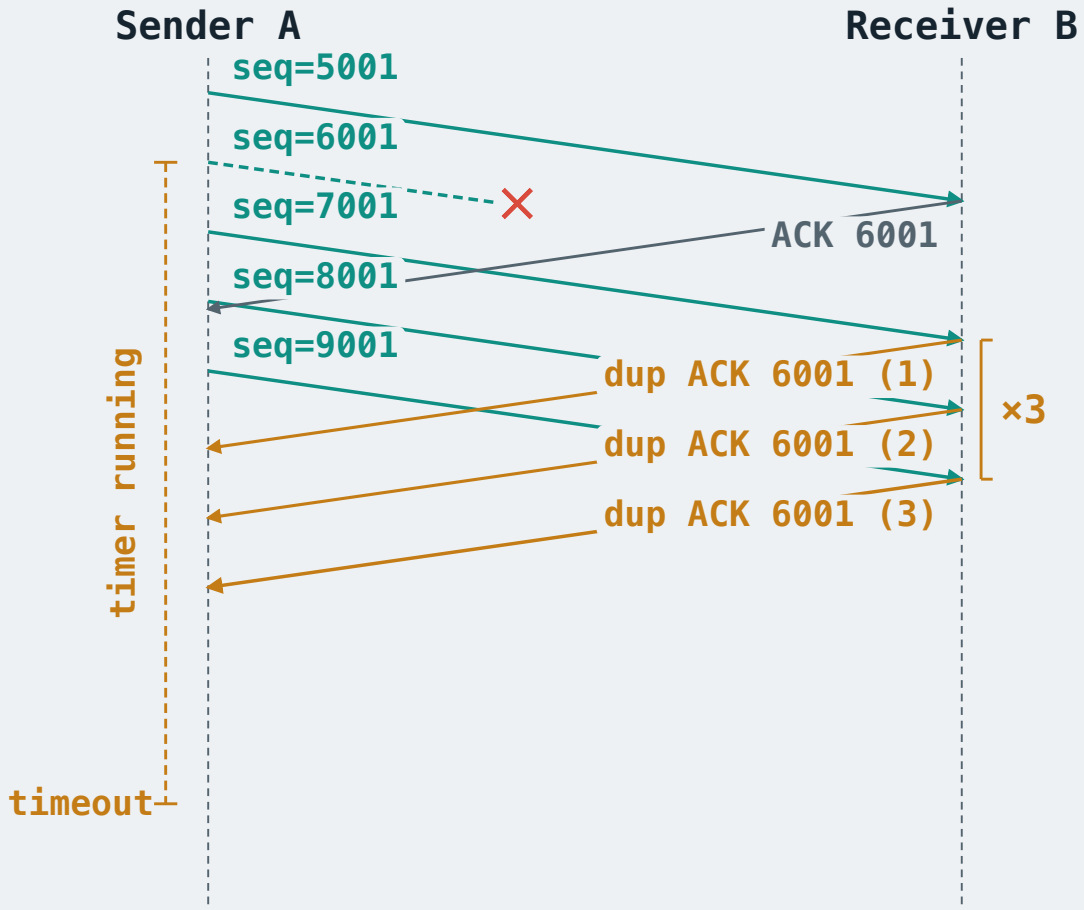
# TCP fast retransmit




**The problem**

Timeout period is relatively long. Suppose seg 6001 is lost — in Go-Back-N, the sender must wait until timeout before it can retransmit. This can take a long time.

# TCP fast retransmit




**Duplicate ACKs — an early signal**

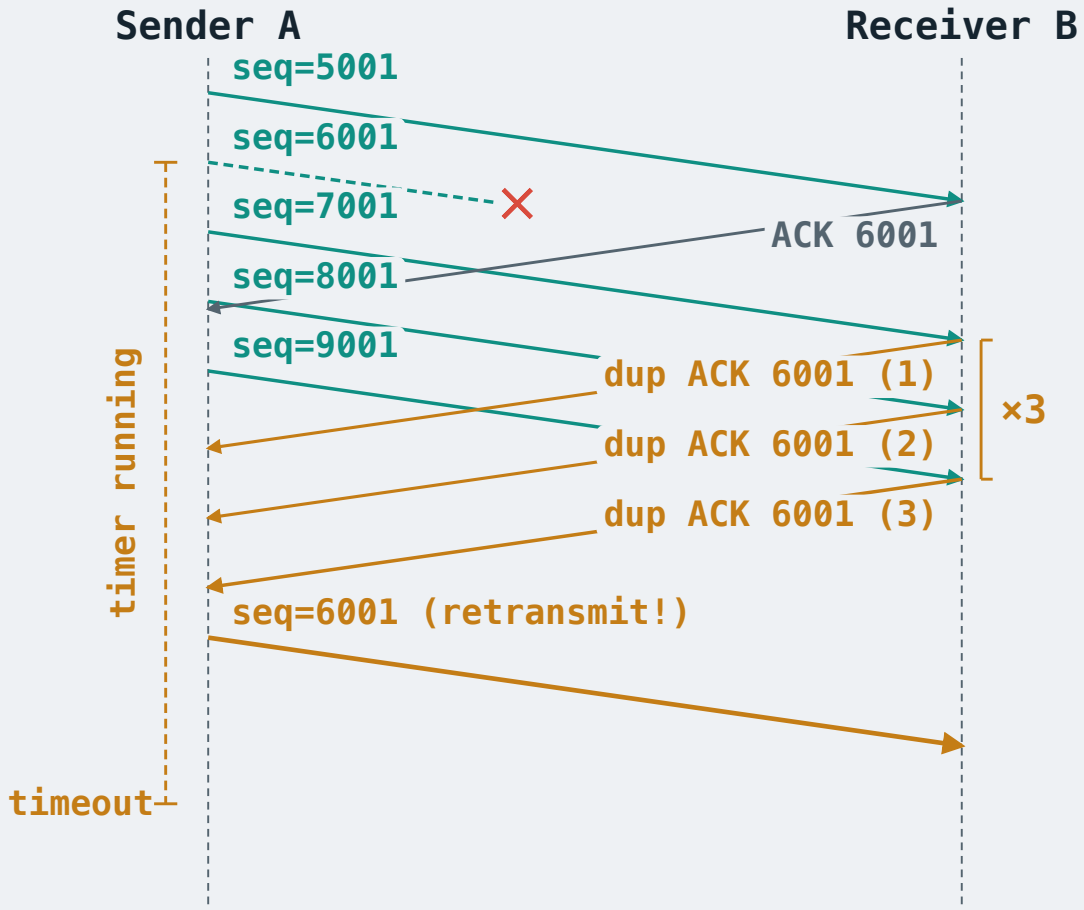
The receiver's ACK names the next byte it expects, e.g. ACK 6001 after seg 5001.

When 7001, 8001 and 9001 arrive but 6001 is missing, each triggers another **duplicate ACK 6001**.

The ACK is stuck: it does not advance, so the receiver is still waiting for the same segment.

That is already an *early indication of loss*, before the timer fires.

# TCP fast retransmit




**Fast retransmit**

After **3 duplicate ACKs**, retransmit the missing segment immediately — without waiting for the timeout.

# TCP receiver rules: overview

**Like Go-Back-N: cumulative ACK**

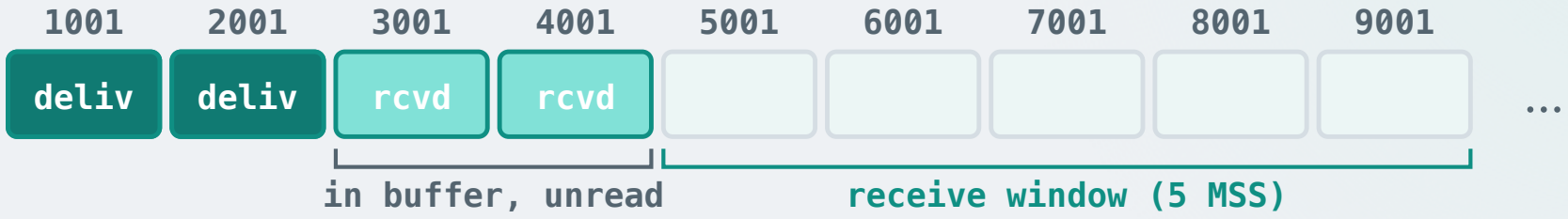
The receiver sends **cumulative ACKs**: each ACK names the next byte it expects, confirming every byte before it.

**Like Selective Repeat: receiver buffer**

The receiver **buffers out-of-order segments** within its receive window instead of discarding them.

Once the missing segment arrives and fills the gap, the buffered data is delivered to the application in order.

# The TCP receiver rules: an example




### The setup

The receiver keeps a **receive window**, e.g. 5 MSS. It is at most the size of the receive buffer, but not always equal to it. We return to this in flow control.

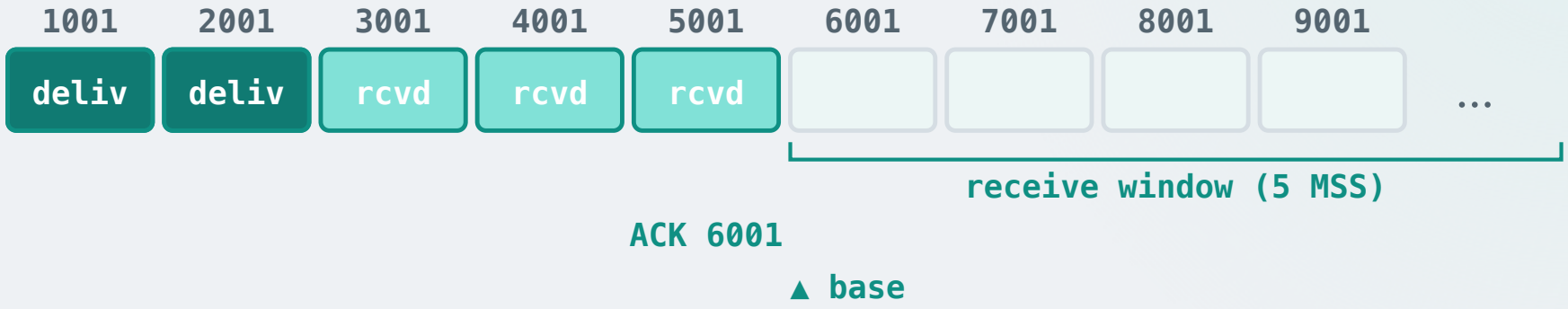
In Selective Repeat (Module 4), the window base `rcv_base` was the first packet **not yet delivered** to the application.

In TCP, the base is instead the **next byte the receiver expects**. Here every byte up to 5000 has been received in order, so the base is **seq 5001**.

**Why are these not the same?**

Received bytes can wait in the receive buffer, because the application may read them slowly: received in order does not mean delivered. We return to this in flow control.

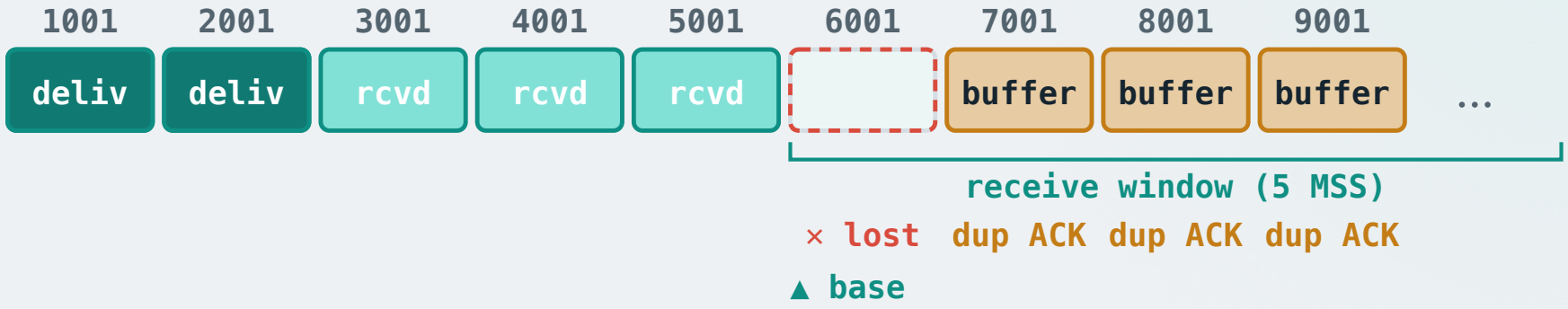
# The TCP receiver rules: an example




### In-order segment arrives

Seg 5001 arrives in order. It is now ready for the application to read, and the receiver sends **ACK 6001**: the next byte it expects. The window base moves to 6001.

# The TCP receiver rules: an example




### Out-of-order segments arrive

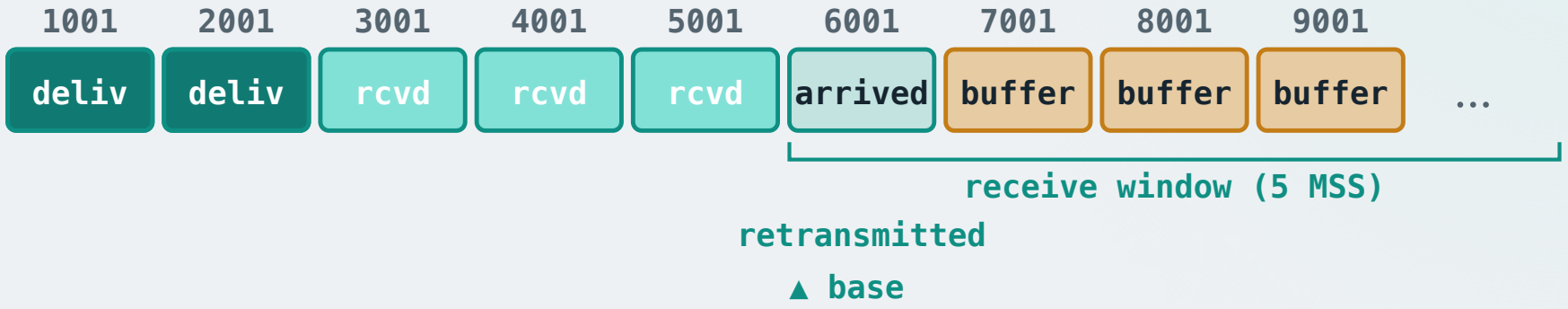
Seg 6001 is lost on the way.

Seg 7001 arrives out of order but **within the receive window**, so the receiver **buffers** it rather than discarding it.

It immediately sends a **duplicate ACK 6001**: it is still expecting byte 6001.

Segs 8001 and 9001 are buffered too, each triggering another duplicate ACK 6001. After three duplicates, the sender fast-retransmits seg 6001.

# The TCP receiver rules: an example




**The gap is filled**

The retransmitted seg 6001 arrives and fills the gap.

# The TCP receiver rules: an example




**The gap is filled**

The retransmitted seg 6001 arrives and fills the gap.

Bytes 6001–10000 are now in order and ready for the application to read, and the receiver immediately sends one cumulative **ACK 10001**. The window base moves to 10001.

# The TCP receiver rules: an example




### Further improvements

RFC 1122 and RFC 5681 add rules for generating ACKs, e.g. **delayed ACKs**, to avoid sending too many.

TCP can also negotiate **selective ACK (SACK)** [RFC 2018]: the receiver reports exactly which byte ranges it holds, so the sender retransmits only the missing pieces, much like Selective Repeat.

# What we have covered so far

- TCP's RDT mixes **Go-Back-N** and **Selective Repeat**: a pipelined sender window, cumulative ACKs, and a receiver buffer for out-of-order segments.
- **Sequence numbers count bytes**: each segment is numbered by its first byte, counting on from the **ISN** chosen in the three-way handshake. SYN and FIN each use one number.
  - An **ACK names the next byte expected**, e.g. ACK 1001 for a SYN with seq 1000.
- **Sender**: sends anything within its window, with **one timer** for the oldest unacknowledged segment; a cumulative ACK slides the window forward.
  - On **timeout**, it retransmits only the oldest unacknowledged segment, not the whole window.
  - On **three duplicate ACKs**, it **fast-retransmits** that segment without waiting for the timeout.
- **Receiver**: its window base is the next byte expected. It buffers out-of-order segments within the window.

# TCP timeout: the challenge

### The timer should track RTT

The sender's retransmit timer is set to roughly the **round-trip time** —

just long enough for a packet to receive its ACK. Too short → spurious retransmissions waste bandwidth. Too long → slow recovery when a segment really is lost.

### The problem: RTT is not a constant number in the real world

Network load, queueing, and path changes cause RTT to fluctuate — sometimes dramatically.

A fixed timeout value cannot adapt.

This is why TCP **estimates RTT dynamically**, in real time, and updates its RTT estimate continuously.

# TCP RTT estimation

**SampleRTT**

EstimatedRTT is derived by taking many individual RTT samples.

Each sample is measured from when a segment is transmitted until its ACK is received.

Retransmitted segments are **excluded**: the ACK could be for the original send or the retransmit — the sender cannot tell, so the measurement would be unreliable.

# TCP RTT estimation

## SampleRTT

### EstimatedRTT — exponential weighted moving average

SampleRTT varies noticeably from measurement to measurement. So how does TCP smooth out these variations and derive EstimatedRTT from the samples?

TCP uses a smoothing technique that computes a smooth, adaptive estimate from the samples.

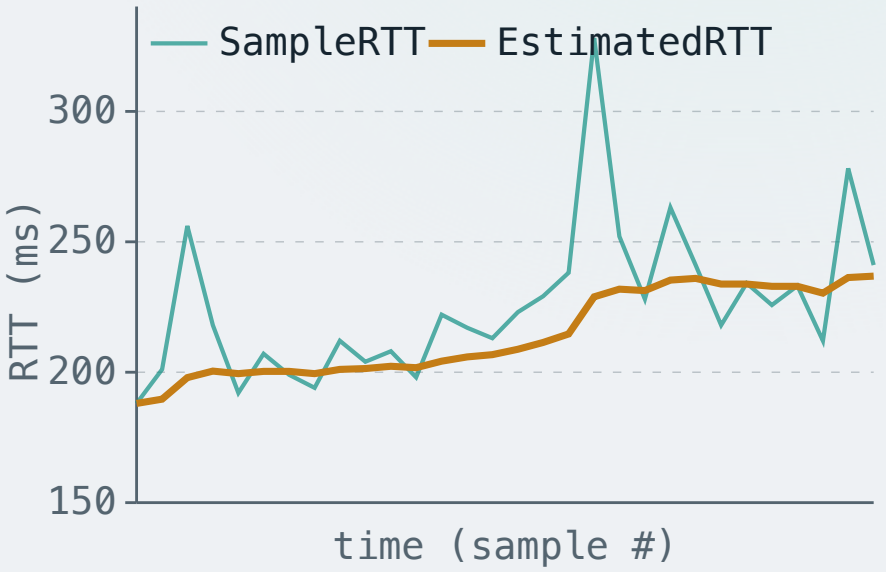
$$\text{EstimatedRTT} = (1-\alpha)\cdot\text{EstimatedRTT} + \alpha\cdot\text{SampleRTT}$$

Typical value: **$\alpha = 0.125$**. Unrolling this one step at a time:

$$\text{EstimatedRTT} = \alpha\cdot\text{Sample}_n + \alpha(1-\alpha)\cdot\text{Sample}_{n-1} + \alpha(1-\alpha)^2\cdot\text{Sample}_{n-2} + \ldots$$

Each step further back adds another factor of (1–α) = 0.875 — so older samples count exponentially less.

This is why it is known as an **exponential weighted moving average**: it smooths out SampleRTT to produce EstimatedRTT.



# TCP timeout interval

You might be tempted to simply set the timeout interval to EstimatedRTT.

In practice, this turns out to be too aggressive: due to exponential averaging, EstimatedRTT takes some time to adapt to the real-time RTT.

So when the real RTT suddenly spikes — say the network becomes congested — EstimatedRTT has not caught up yet, and the timer fires too early.

**We therefore need a safety margin for RTT variability. How do we choose it?**

### DevRTT — tracking variability

We also maintain **DevRTT** — a tracker for how much SampleRTT deviates from EstimatedRTT.

$$\text{DevRTT} = (1-\beta)\cdot\text{DevRTT} + \beta\cdot|\text{SampleRTT} - \text{EstimatedRTT}|$$

Typical value: $\beta = 0.25$.

### TimeoutInterval

This is how TCP sets the timeout interval, with DevRTT providing the extra safety margin:

$$\textbf{TimeoutInterval} = \textbf{EstimatedRTT} + 4\cdot\textbf{DevRTT}$$

Empirically, this prevents the timer from firing too early in the case of a sudden spike in RTT.

MATCHING THE SENDER TO THE RECEIVER AND THE NETWORK

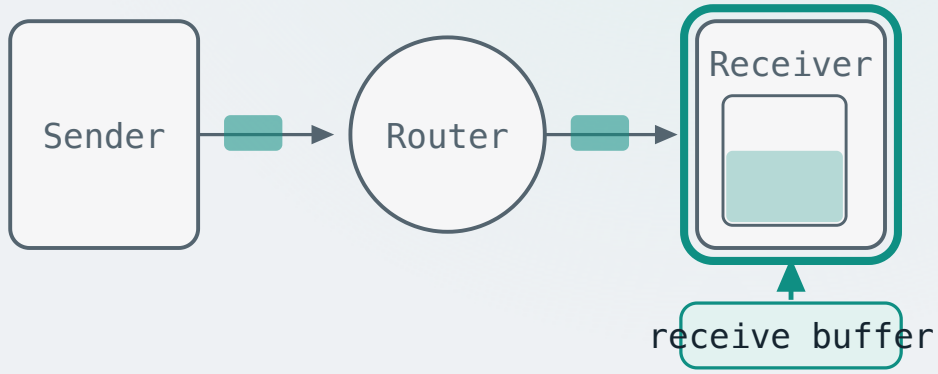
Concept 3 of 3

# TCP Flow Control and Congestion Control

# Flow control vs congestion control

What are flow control and congestion control? Consider a sender transmitting data to a receiver. The router in the middle stands for the network in between.

Ideally, the sender would send data as fast as it likes. In practice it cannot: there are **two bottlenecks** that could occur.




### Flow control

In our previous discussions, we always assumed that the application consumes data as soon as it arrives in order at the receiver.

In reality, the application may read slowly, so the receiver must hold incoming data and feed it to the application gradually. This is why the receiver maintains a **receive buffer**.

This buffer has a limited size: if the sender transmits too fast, it overflows and segments are lost.

Flow control prevents this by matching the sender's transmission rate to the speed at which the receiver's application reads, so the buffer does not overflow.

# Flow control vs congestion control

What are flow control and congestion control? Consider a sender transmitting data to a receiver. The router in the middle stands for the network in between.

Ideally, the sender would send data as fast as it likes. In practice it cannot: there are **two bottlenecks** that could occur.

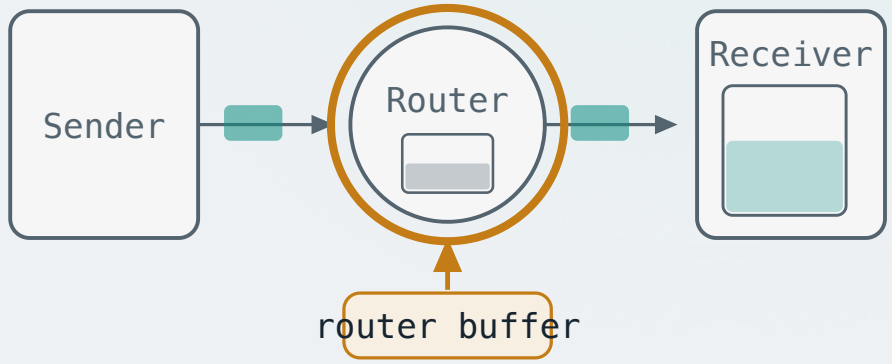
### Congestion control

We have already seen that datagrams travel through the Internet across a series of routers.

A router takes time to process each incoming datagram and cannot always forward it immediately, so it keeps a **router buffer**.

If too many senders transmit simultaneously, the buffer of some router along the route fills up and segments are dropped: the network is **congested**.

Congestion control prevents this by limiting each sender's rate to match what the network can carry.



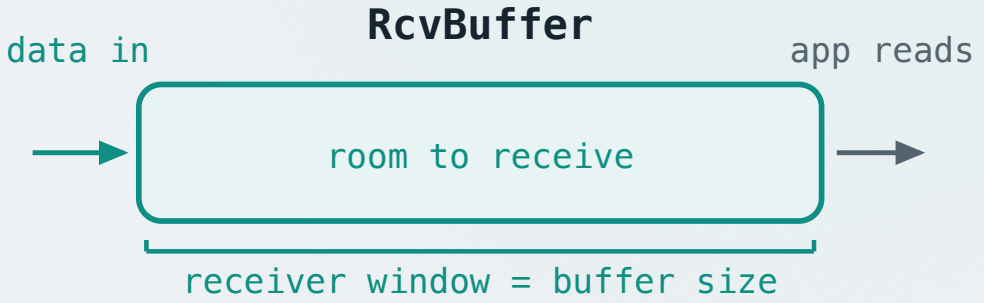
# TCP flow control: the receive window

### The receive buffer and floating window

The receiver allocates a fixed-size receive buffer (**RcvBuffer**) for the connection. Incoming data sits there until the application reads it.

Recall that in RDT, the **receiver window** is the range of **out-of-order** packets the receiver is willing to buffer.

In our previous RDT discussions, we assumed an **ideal scenario** where the application consumes data as soon as it arrives **in order**. The buffer was therefore always fully available for **out-of-order** packets, so we always set **receiver window = buffer size**.



# TCP flow control: the receive window

### The receive buffer and floating window

The receiver allocates a fixed-size receive buffer (**RcvBuffer**) for the connection. Incoming data sits there until the application reads it.

Recall that in RDT, the **receiver window** is the range of **out-of-order** packets the receiver is willing to buffer.

In our previous RDT discussions, we assumed an **ideal scenario** where the application consumes data as soon as it arrives **in order**. The buffer was therefore always fully available for **out-of-order** packets, so we always set **receiver window = buffer size**.

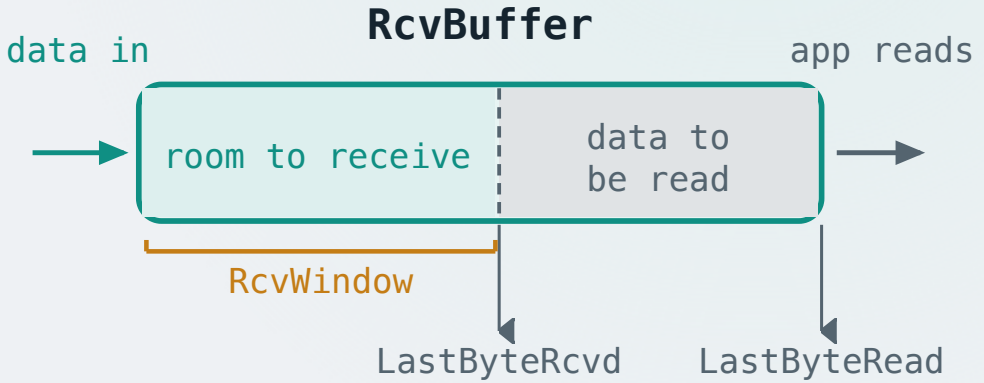
In practice, however, the application reads at its own pace — so the buffer may contain unread data that reduces the available space.

The unread portion is then [LastByteRcvd – LastByteRead].

Thus the receiver window **floats**:

```
RcvWindow = RcvBuffer – [LastByteRcvd – LastByteRead]
```

*e.g. RcvBuffer = 4000 bytes, LastByteRead = 2000, LastByteRcvd = 3500 → RcvWindow = 4000 – (3500 – 2000) = **2500** bytes.*




**LastByteRcvd**: the sequence number of the last byte received.

**LastByteRead**: the sequence number of the last byte the application has read.

FROM RECEIVER TO SENDER

# Advertising and enforcing the receive window

### Advertising the window

How does the TCP receiver advertise its current window to the sender?

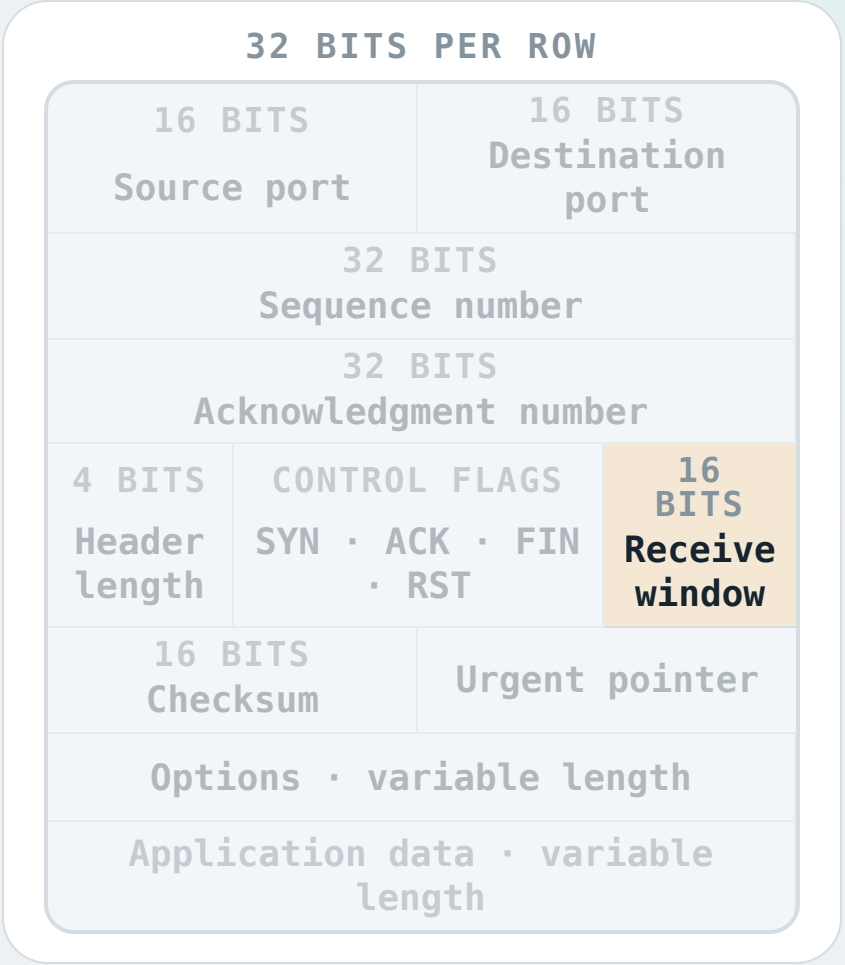
In TCP, the receiver piggybacks RcvWindow on every segment it sends, carried in the **Receive Window** field of the TCP header.

### The TCP sender's rule

**`data in flight ≤ RcvWindow`**

Recall that the data in flight is the range [**base**, **nextseqnum**), so the sender keeps `nextseqnum − base ≤ RcvWindow`.

Whereas in our previous RDT discussions we assumed the sender window was a fixed number **N**, in flow control the TCP sender's cap is the **RcvWindow** the receiver advertises, which changes over time.



# A corner case: the zero window

**The problem**

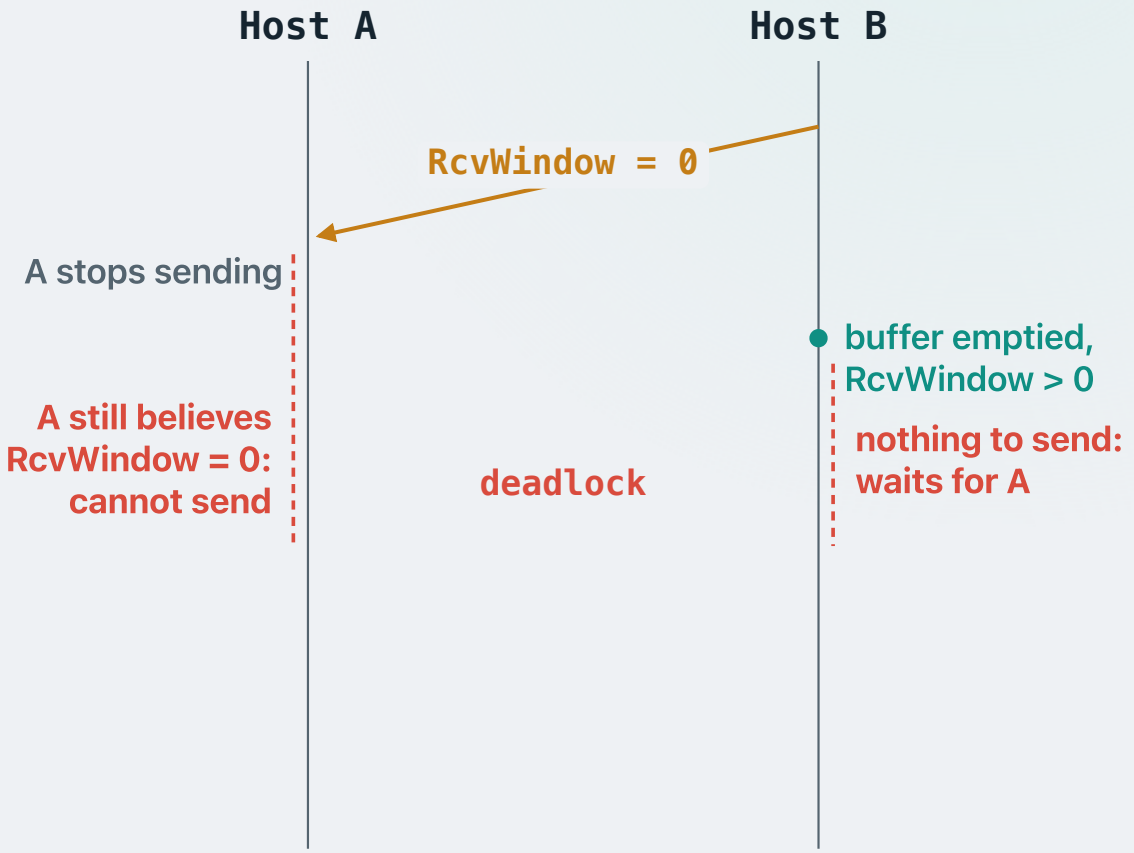
Suppose B's receive buffer fills up, so B advertises **RcvWindow = 0**. A must stop sending.

Later, B's application reads the buffered data. The buffer empties, so B's window is open again: RcvWindow > 0.

But A has not heard about it. Its latest information is still RcvWindow = 0, so A still cannot send.

And B cannot tell A: B only sends a segment (an ACK) in reply to one from A.

A waits for B, B waits for A — a **deadlock**.



# A corner case: the zero window

### The problem

Suppose B's receive buffer fills up, so B advertises **RcvWindow = 0**. A must stop sending.

Later, B's application reads the buffered data. The buffer empties, so B's window is open again: RcvWindow > 0.

But A has not heard about it. Its latest information is still RcvWindow = 0, so A still cannot send.

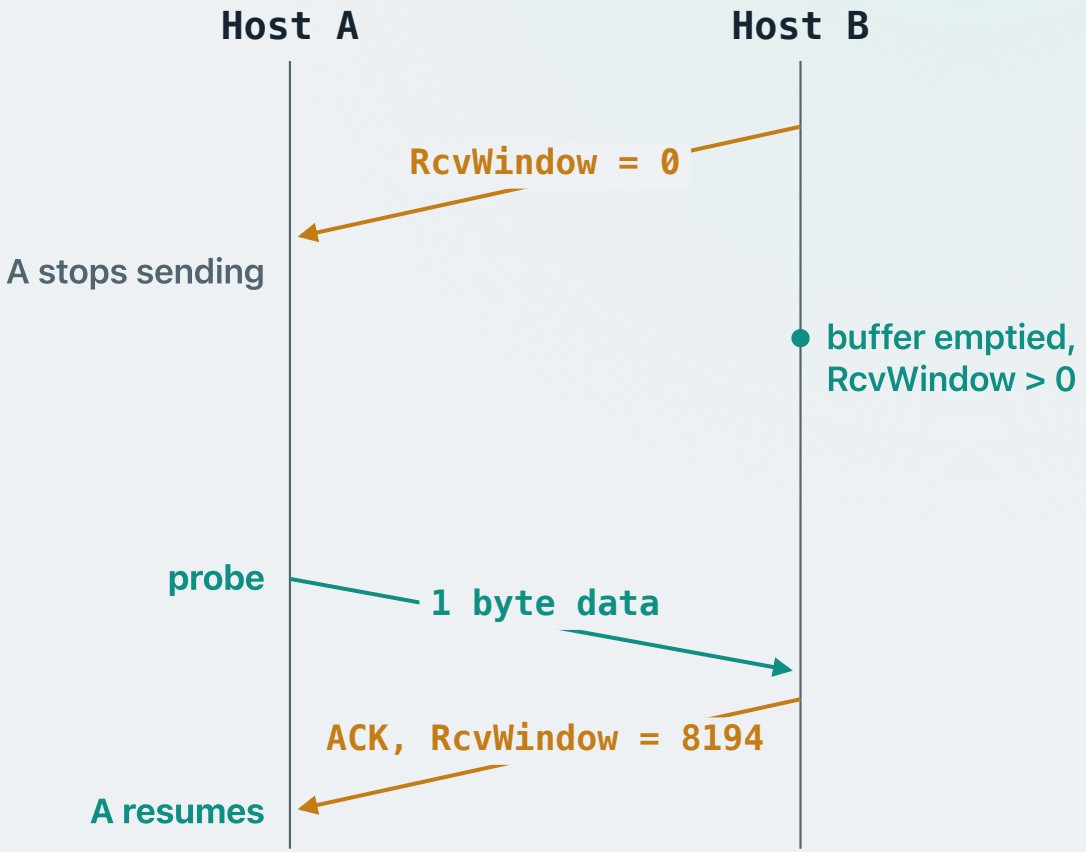
And B cannot tell A: B only sends a segment (an ACK) in reply to one from A.

A waits for B, B waits for A — a **deadlock**.

### The solution

While RcvWindow = 0, A keeps sending tiny segments carrying just **1 byte** of data.

B must acknowledge each one, and every ACK carries B's current RcvWindow — so as soon as space frees up, A learns it and resumes sending.



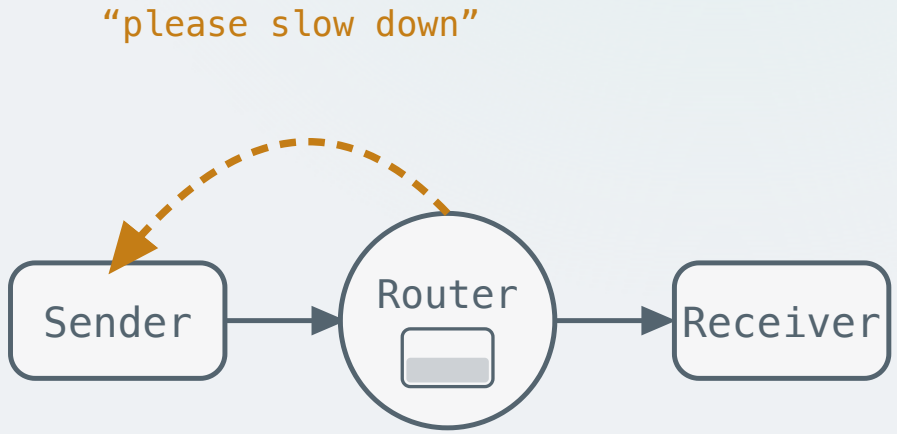
# Approaches to congestion control

Recall the problem: when too many senders transmit at once, router buffers fill up and segments are dropped, so each sender must limit its rate to what the network can carry.

**Network-assisted**

Routers explicitly notify senders to slow down.

This requires the routers along the path to cooperate — e.g. a router sets a congestion bit in the IP packet header, or states the rate it can support.



# Approaches to congestion control

Recall the problem: when too many senders transmit at once, router buffers fill up and segments are dropped, so each sender must limit its rate to what the network can carry.

### Network-assisted

Routers explicitly notify senders to slow down.

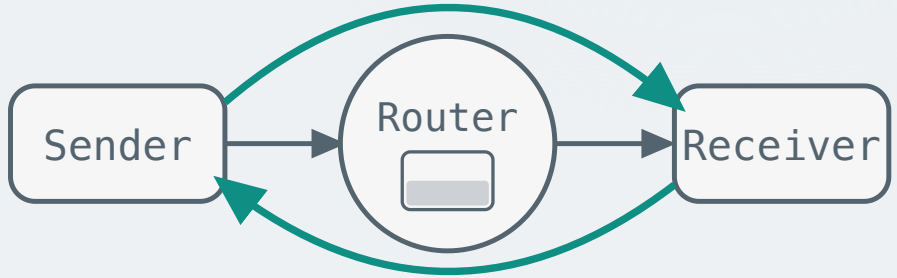
This requires the routers along the path to cooperate — e.g. a router sets a congestion bit in the IP packet header, or states the rate it can support.

### End-to-end

No explicit feedback from the network: the sender infers congestion from signals it can observe itself.

e.g. a **timeout**, or **duplicate ACKs**. TCP treats these as **loss events**.

TCP uses the **end-to-end** approach — congestion is inferred from **loss events** in the network.



# TCP congestion detection and action: an overview

### The congestion window

Flow control already caps the data in flight at RcvWindow.

Congestion control adds a second cap on the same quantity: the **congestion window (CongWin)**, which the sender sets itself and adjusts up and down with the level of congestion.

```
nextseqnum − base
≤ min(CongWin, RcvWindow)
```

The data in flight must respect both caps.



# TCP congestion detection and action: an overview

### The congestion window

```
nextseqnum − base
≤ min(CongWin, RcvWindow)
```

### Sending

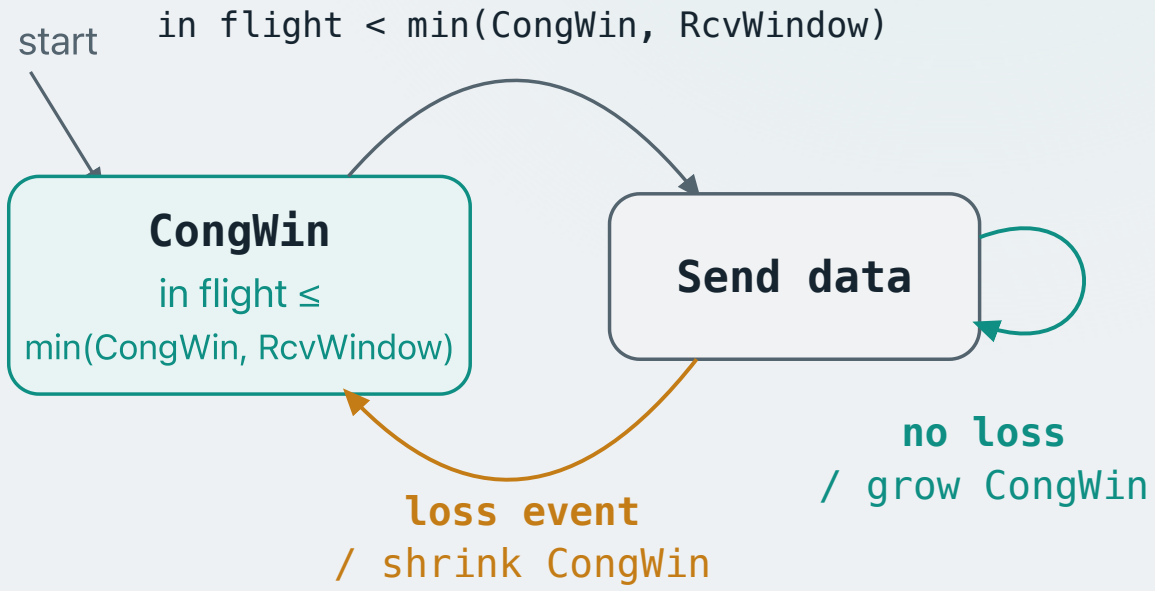
As long as there is room in the window (data in flight < min(CongWin, RcvWindow)), the sender can send more data.

### Detection and action

The sender observes a **loss event**, e.g. three duplicate ACKs or a timeout.

On a loss event, the sender **shrinks** CongWin.

If no loss event is observed, the sender slowly **increases** CongWin.



# Setting the TCP sending rate

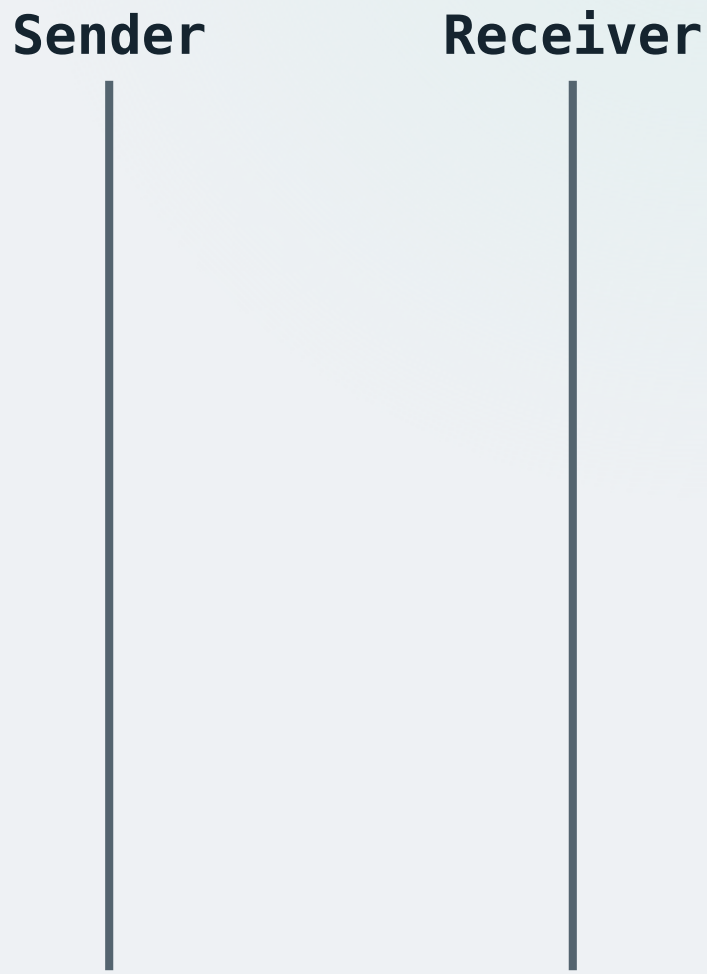
### What the sender controls

The previous frame gives a discrete rule: keep data in flight ≤ CongWin. But how does the sender actually obey that rule?

In practice, the TCP sender can control its **sending rate** to obey the cap. Since the TCP sender sends data as a byte stream, the sending rate is measured in **bytes per second**.

If the sending rate is too high, the data in flight exceeds CongWin.

**So, given a fixed CongWin, what sending rate — how many bytes per second — should the sender use to obey the in-flight constraint?**



# Setting the TCP sending rate

## What the sender controls

### The missing ingredient: RTT

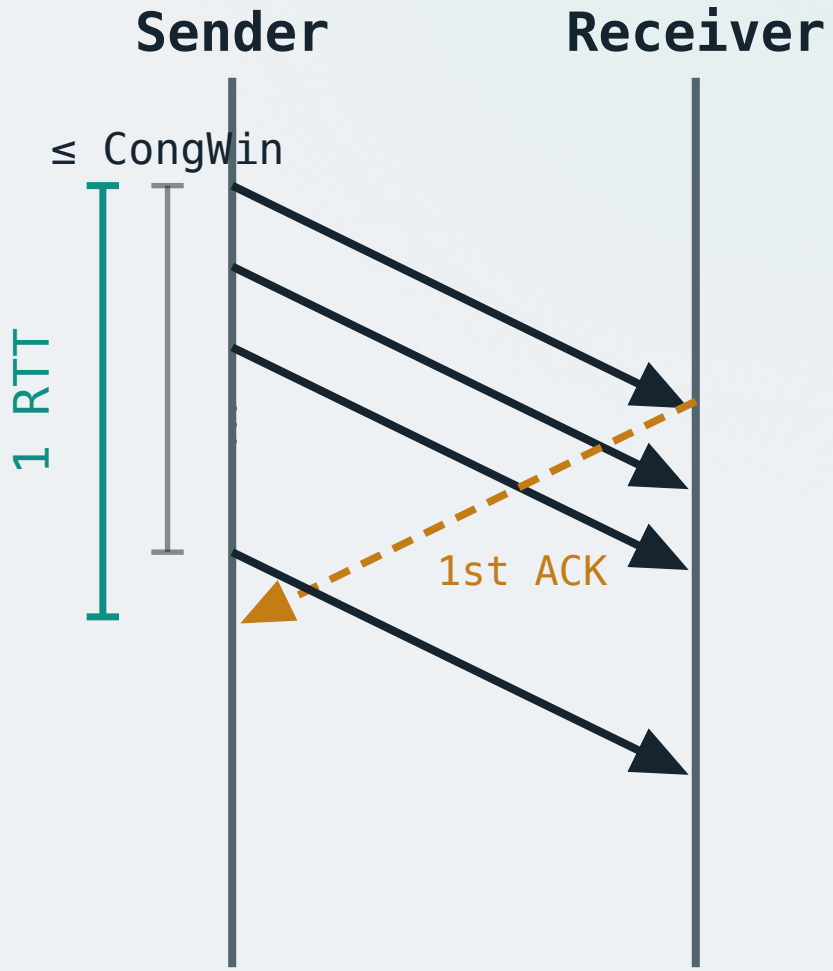
Recall: the sender transmits segments back-to-back — all remain in flight until the first ACK returns, which takes one RTT.

So within one RTT, everything sent is simultaneously in flight. Since at most CongWin bytes can be in flight, **the sender can send at most CongWin bytes per RTT.**

### The sending rate

So the sender should send CongWin bytes per RTT, i.e. set its sending rate to:

```
sending rate ≈ CongWin / RTT  bytes/sec
```



# Setting the congestion window

**How does the TCP sender decide how to update its congestion window?**

It depends on where the connection is in its life. There are two cases, and each gets a different treatment.

### Case 1: just after the connection is set up

The three-way handshake has just finished. The sender knows nothing yet about how much the network can carry, so it starts by setting CongWin small and increases it very quickly.

This phase is called **slow start**.

### Case 2: a while after the connection is set up

CongWin has already grown large, close to what the network can carry. Now the sender should grow it carefully, to avoid causing congestion.

This phase is called **congestion avoidance**.

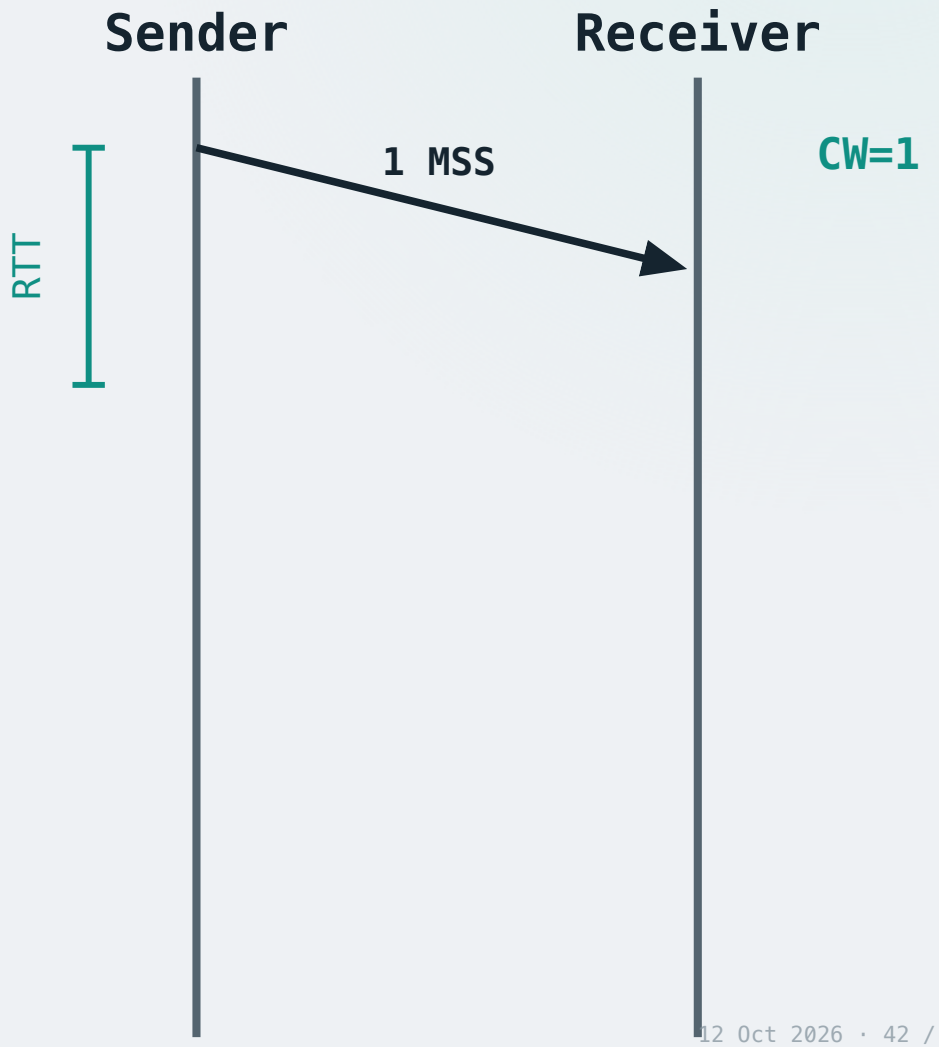
# TCP · slow start

### Starting conservatively

When a connection begins, the sender knows nothing about the path — start conservatively: **CongWin = 1 MSS**.

Since CongWin = 1 MSS, the in-flight constraint means the sender may only send **1 MSS per RTT**.

Each segment carries exactly 1 MSS of data. Send one segment, wait for its ACK.



# TCP · slow start

### Growing the window

Each ACK confirms the network handled the segment — likely no congestion yet.

**SLOW-START RULE**

**For every ACK received: CongWin += 1 MSS**

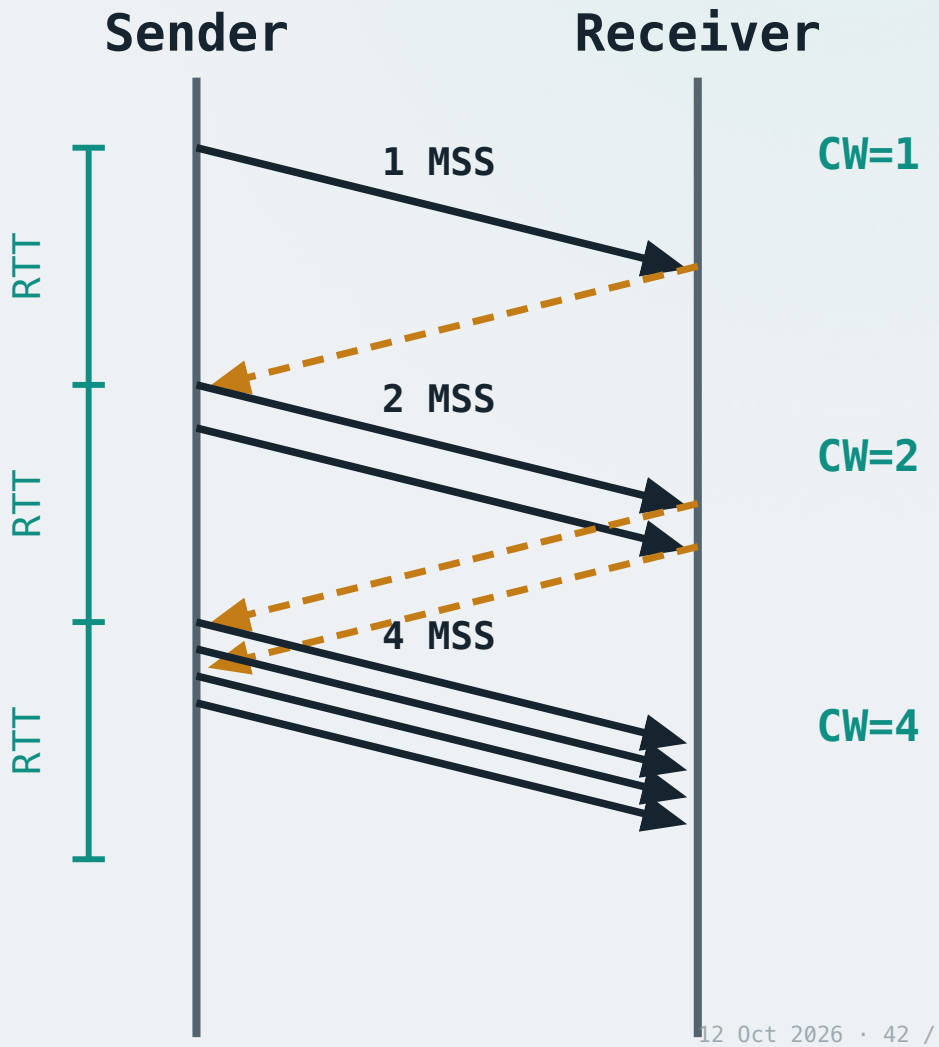
CongWin increases to 2 — in the next RTT the sender can send **2 MSS**.

Two more ACKs received — each adds 1 MSS. CongWin increases to 4, and in the next RTT we can send **4 MSS**.

The pattern: the growth rate depends on the current window size.

**SLOW-START EMPIRICAL RULE**

**CongWin doubles every RTT — 1, 2, 4, 8, 16… — exponential growth**

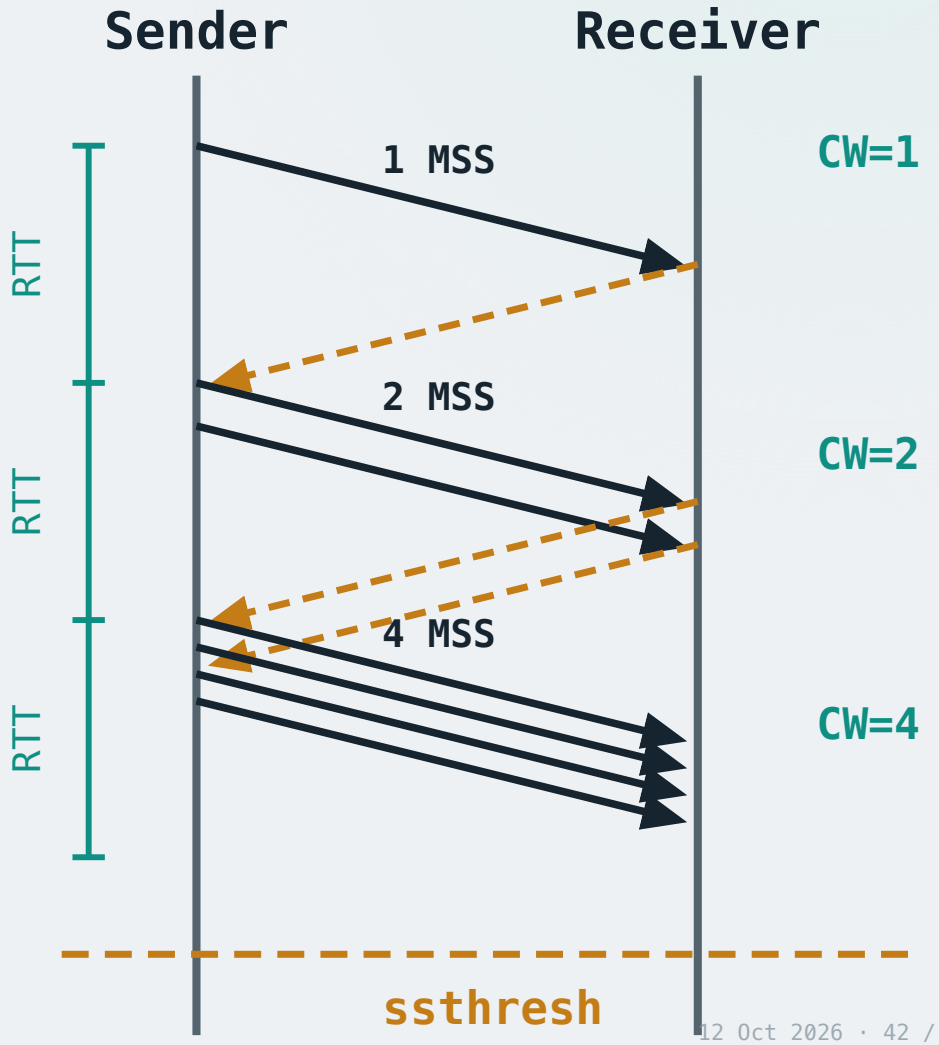


# TCP · slow start

### The threshold

Exponential growth can't last forever. A threshold called **ssthresh** (slow-start threshold) marks where slow start ends.

When CongWin ≥ ssthresh → switch to **congestion avoidance** (next frame).



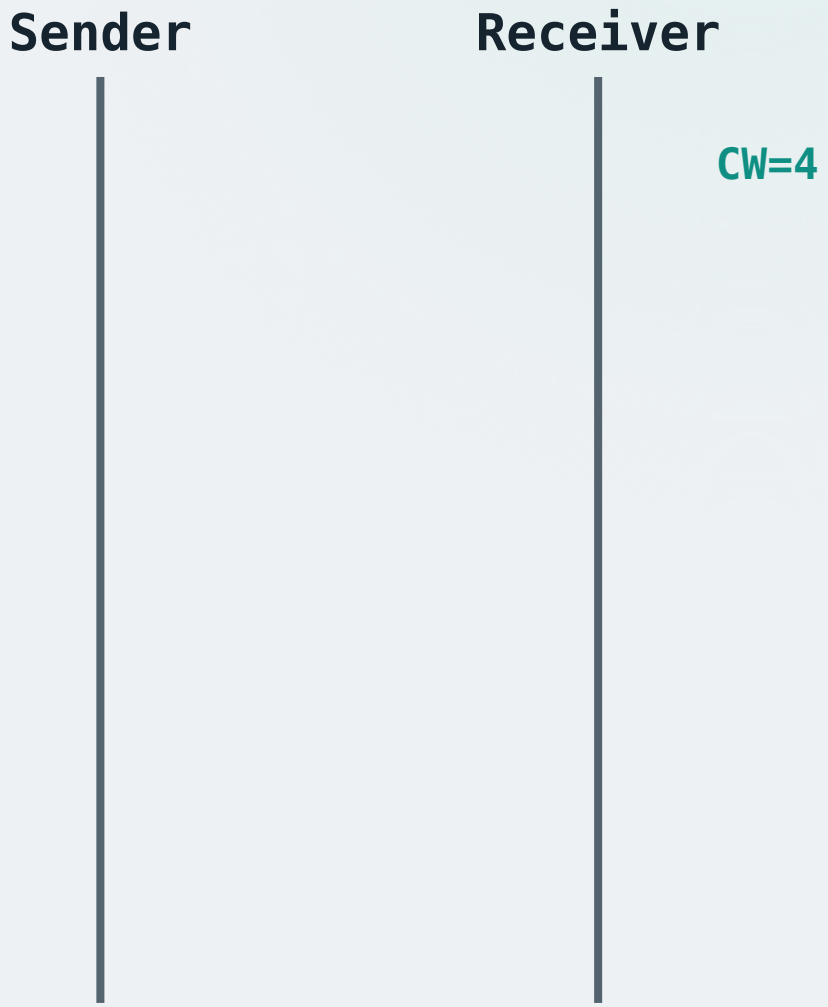
# TCP · congestion avoidance

### Additive increase

Once CongWin ≥ ssthresh, growth switches from exponential to linear.

```
ADDITIVE INCREASE
If no loss detected: CongWin += 1 MSS per RTT
```

**Example.** Suppose **ssthresh = 4 MSS**, and CongWin has just reached it: CongWin = 4 MSS.



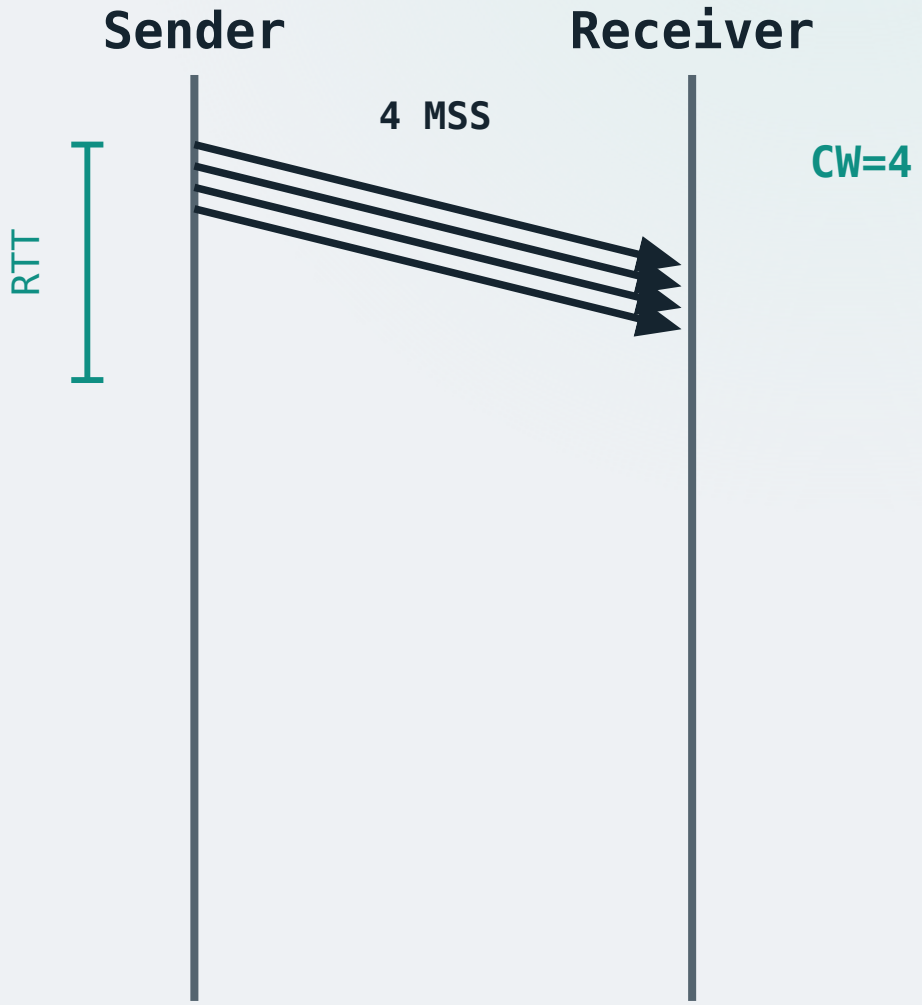
# TCP · congestion avoidance

### Additive increase

Once CongWin ≥ ssthresh, growth switches from exponential to linear.

```
ADDITIVE INCREASE
If no loss detected: CongWin += 1 MSS per RTT
```

In the first RTT, the sender sends **4 segments**.



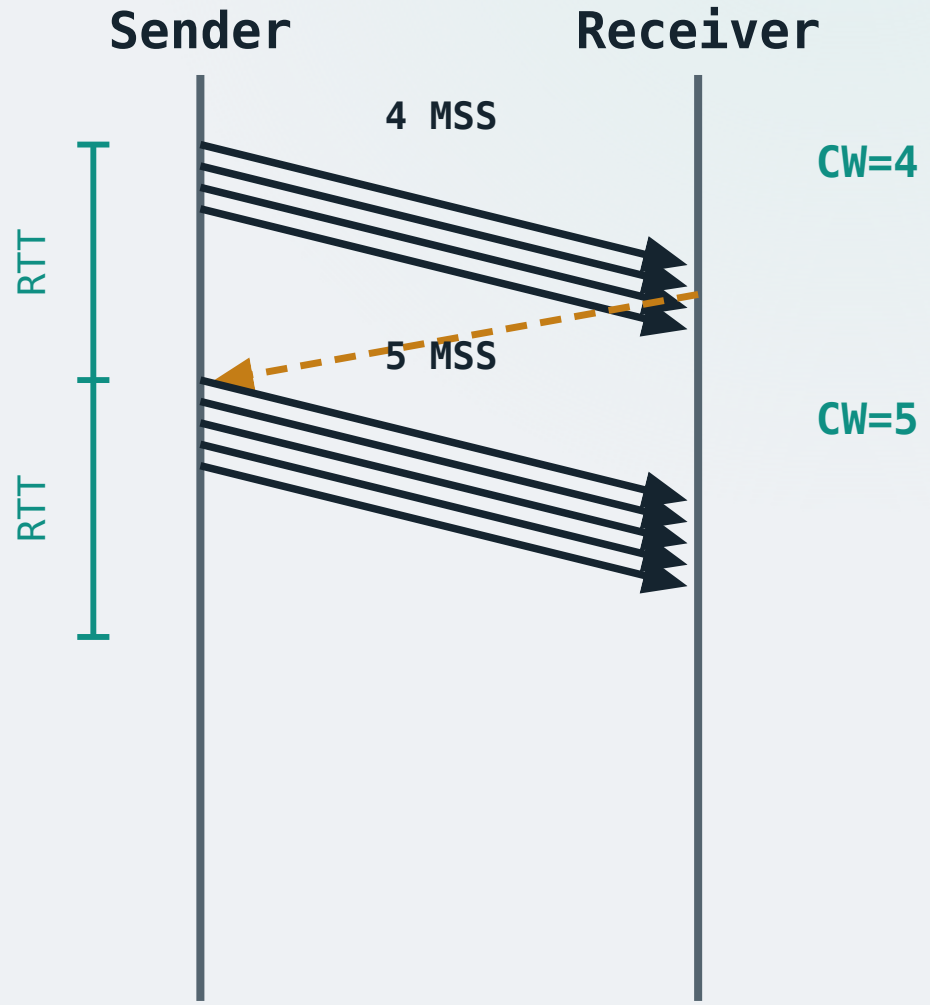
# TCP · congestion avoidance

### Additive increase

Once CongWin ≥ ssthresh, growth switches from exponential to linear.

```
ADDITIVE INCREASE
If no loss detected: CongWin += 1 MSS per RTT
```

Once all 4 are acknowledged, CongWin grows by just **1 MSS** — not doubled to 8 — so CongWin = 5, and the next RTT carries 5 segments.



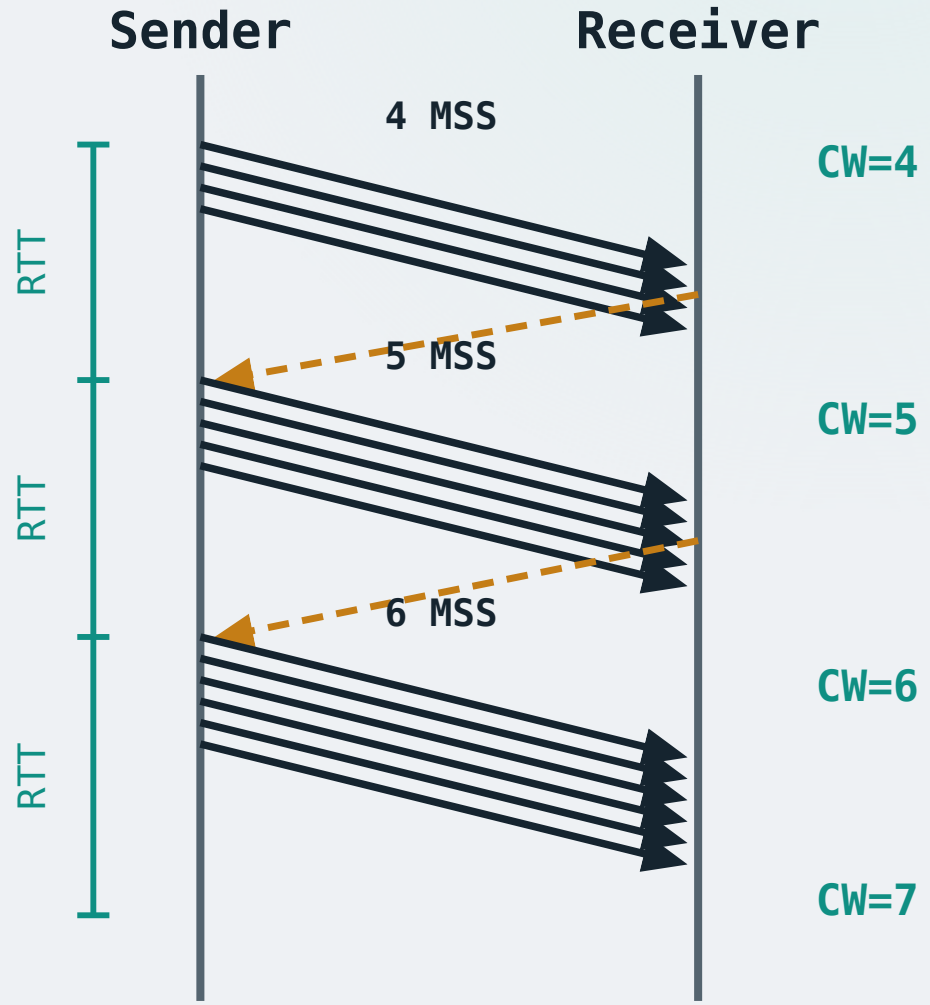
# TCP · congestion avoidance

### Additive increase

Once CongWin ≥ ssthresh, growth switches from exponential to linear.

```
ADDITIVE INCREASE
If no loss detected: CongWin += 1 MSS per RTT
```

And so on: after those 5 are acknowledged, CongWin = 6, then 7, … one MSS per RTT.



# TCP · congestion avoidance

### Additive increase

Once CongWin ≥ ssthresh, growth switches from exponential to linear.

```
ADDITIVE INCREASE
If no loss detected: CongWin += 1 MSS per RTT
```

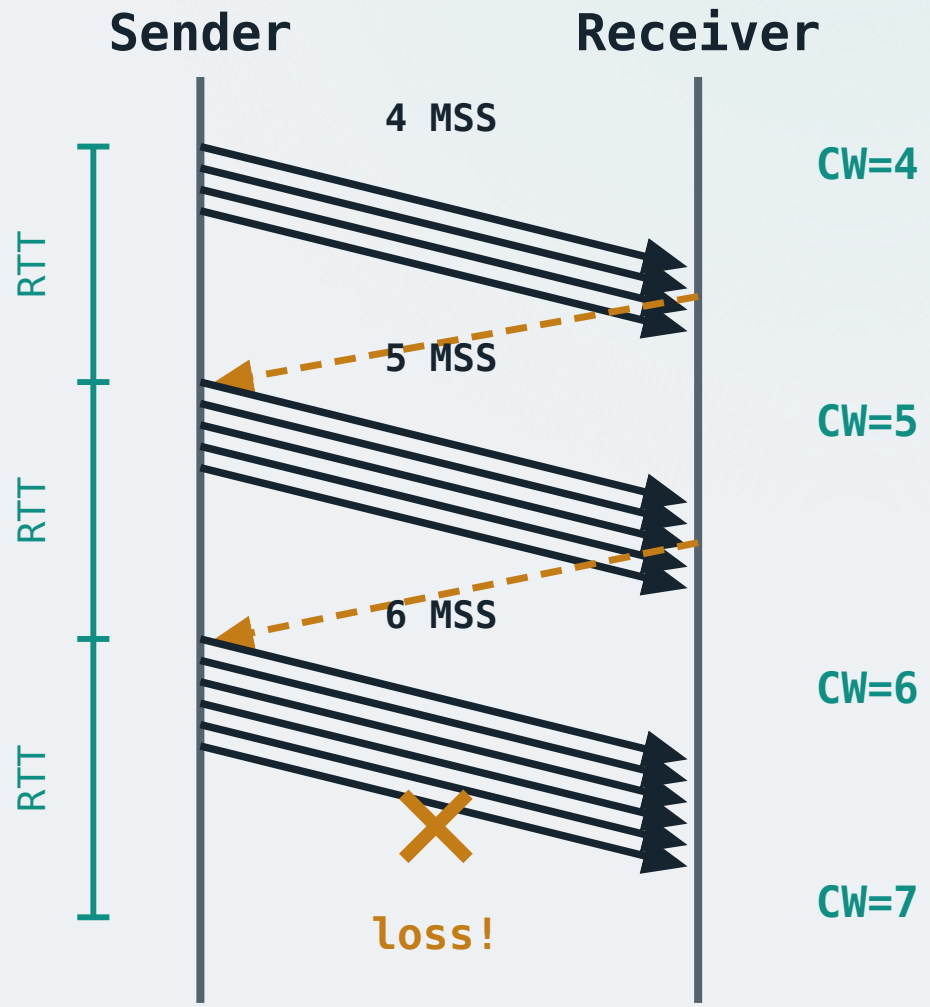
We keep doing additive increase, but this cannot last forever: eventually the network becomes congested again, and the sender hits a **loss event**.

**What should the sender do then?**

### Multiplicative decrease

```
MULTIPLICATIVE DECREASE
Upon loss event: CongWin = CongWin / 2
```

Together, this is known as the **AIMD** rule — Additive Increase, Multiplicative Decrease.



# Why AIMD?

Why this particular combination? Let’s consider some alternative designs to AIMD. Here are a few options.

**Multiplicative Increase, Additive Decrease (MIAD)**

Multiplicative increase overshoots capacity aggressively, while additive decrease backs off too slowly — congestion persists.

# Why AIMD?

Why this particular combination? Let's consider some alternative designs to AIMD. Here are a few options.

### Multiplicative Increase, Additive Decrease (MIAD)

### Additive Increase, Additive Decrease (AIAD)

Does AIAD control congestion correctly? For a single flow, yes.

But here is what goes wrong: a different issue, **fairness**. The goal of fairness is that every flow sharing a bottleneck link should converge to an equal share of bandwidth.

> Suppose two flows share a 10 Mbps link: Flow A = 8 Mbps, Flow B = 2 Mbps. This is not fair — a good congestion control design should correct the imbalance while also preventing congestion.
>
> Additive increase: both add 1 Mbps → A = 9 Mbps, B = 3 Mbps. Total exceeds capacity → loss.
>
> Additive decrease: both subtract 1 Mbps → A = 8 Mbps, B = 2 Mbps. Back to the start — the unfairness never corrects.

**AIAD does not solve the fairness problem.**

### Additive Increase, Multiplicative Decrease (AIMD)

Additive increase probes gently; multiplicative decrease reacts sharply. Converges to both **efficiency** and **fairness**.

*Take-home question: using the same example (A = 8 Mbps, B = 2 Mbps), why does multiplicative decrease close the gap where additive decrease could not?*

# TCP · reacting to loss

### What a loss event changes

We said TCP cuts CongWin in half on a loss. That is not the whole truth: there are two kinds of loss event.

What every loss event changes is the **slow-start threshold**, our empirical estimate of the network's capacity. It is set to half the CongWin at which the loss occurred:

```
ON ANY LOSS EVENT
ssthresh = CongWin / 2
```

What happens to CongWin itself depends on the loss signal. There are two cases.

# TCP · reacting to loss

### What a loss event changes

What every loss event changes is the **slow-start threshold**, our empirical estimate of the network's capacity. It is set to half the CongWin at which the loss occurred:

```
ON ANY LOSS EVENT
ssthresh = CongWin / 2
```

### Case 1: Timeout

A timeout signals **severe** congestion: the sender hears nothing back at all.

So be conservative and start from scratch: reset **CongWin = 1 MSS** and enter **slow start**.

### Case 2: Three duplicate ACKs

Three duplicate ACKs signal a **milder** loss: the receiver is still replying, so some data was lost but packets are still getting through.

So the sender enters **fast recovery**, and eventually CongWin is cut only to **ssthresh**, half its old value.

*This is the TCP Reno design. In TCP Tahoe, there is no distinction: both cases reset CongWin = 1 MSS and enter slow start.*

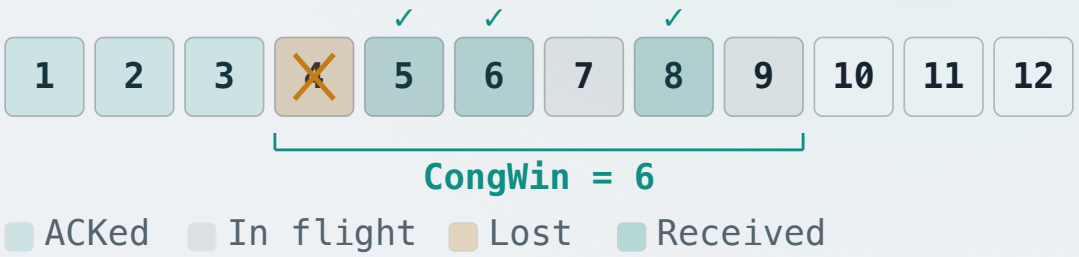
# TCP · fast recovery

Suppose CongWin = 6. Segments 1–3 have been acknowledged, and the sender has more data to transmit.

The sender fills the congestion window: segments 4–9 are now in flight.

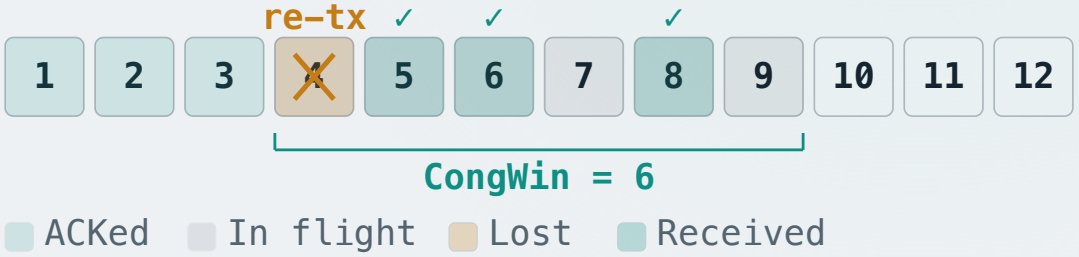
The sender then receives three dup ACKs for segment 4. What does this mean?

Segment 4 is likely **lost** during transmission. But importantly, three ACKs means three of the in-flight packets 5–9 reached the receiver — suppose those are segments 5, 6, 8.



# TCP · fast recovery

> What should the sender do now?
>
> **Fast retransmit**: immediately resend segment 4.
>
> The sender enters **fast recovery**.



# TCP · fast recovery

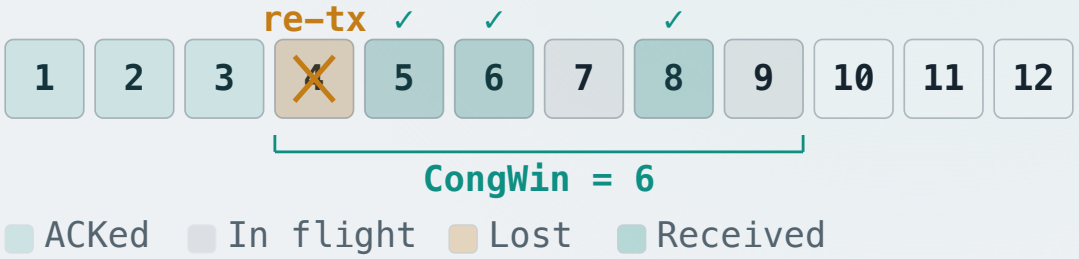
**FAST RECOVERY — ENTER**

ssthresh = CongWin / 2 = **3**

CongWin = ssthresh + **3** = **6**

**Why inflate by 3?**

- CongWin limits how many segments can be in flight. If we simply apply AIMD and set CongWin = ssthresh = 6/2 = 3, the window starts at base 4 and covers segments 4, 5, 6.
- However, by observing 3 dup ACKs, we know 3 segments among 5–9 are no longer in flight. Segments **5, 6, 8** are confirmed received and do not occupy in-flight slots.
- The only truly in-flight segments are **4, 7, 9**. This justifies inflating CongWin to 6 = ssthresh (3) + confirmed received (3), reflecting that only 3 segments are truly in flight.

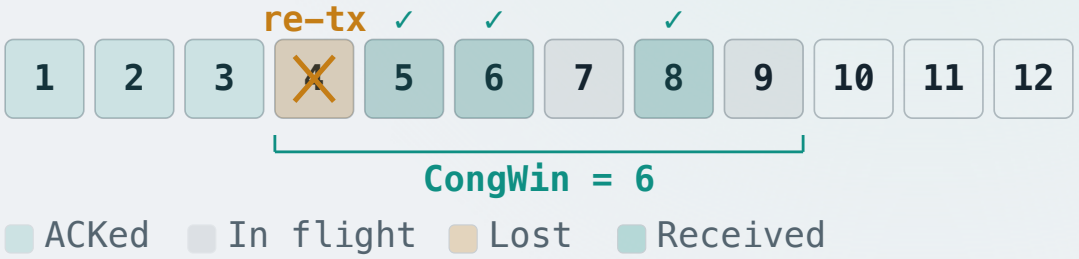


# TCP · fast recovery

**FAST RECOVERY — PER DUP ACK**

**For each additional dup ACK: CongWin += 1 MSS**

For the same reason, each additional dup ACK signifies one less segment is truly in flight — the window can grow by 1.



# TCP · fast recovery

**FAST RECOVERY — PER DUP ACK**

**For each additional dup ACK: CongWin += 1 MSS**

For the same reason, each additional dup ACK signifies one less segment is truly in flight — the window can grow by 1.

Segment 7 received, yet segment 4 still pending (another dup ACK for 4): **CongWin = 7** — can now send segment 10.



# TCP · fast recovery

**FAST RECOVERY — PER DUP ACK**

**For each additional dup ACK: CongWin += 1 MSS**

For the same reason, each additional dup ACK signifies one less segment is truly in flight — the window can grow by 1.

Segment 7 received, yet segment 4 still pending (another dup ACK for 4): **CongWin = 7** — can now send segment 10.

Segment 9 received, yet segment 4 still pending (another dup ACK for 4): **CongWin = 8** — can now send segment 11.



# TCP · fast recovery exit

Now suppose the retransmitted segment 4 has been received, so the receiver sends a cumulative ACK for everything through segment 9 — it sends ACK 10.

**FAST RECOVERY — EXIT RULE**

On new ACK:

- **Set CongWin = ssthresh**
- **Enter congestion avoidance**



# TCP · fast recovery exit

Now suppose the retransmitted segment 4 has been received, so the receiver sends a cumulative ACK for everything through segment 9 — it sends ACK 10.

**FAST RECOVERY — EXIT RULE**

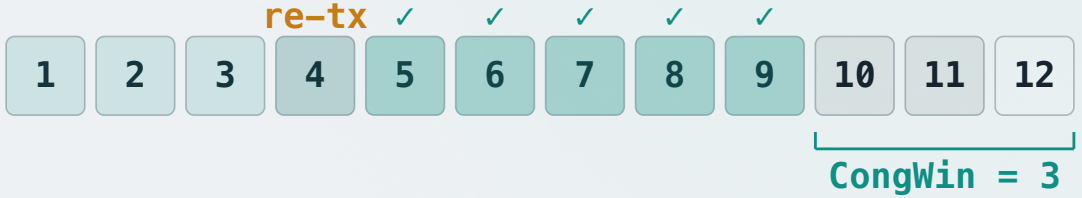
On new ACK:

- **Set CongWin = ssthresh**
- **Enter congestion avoidance**

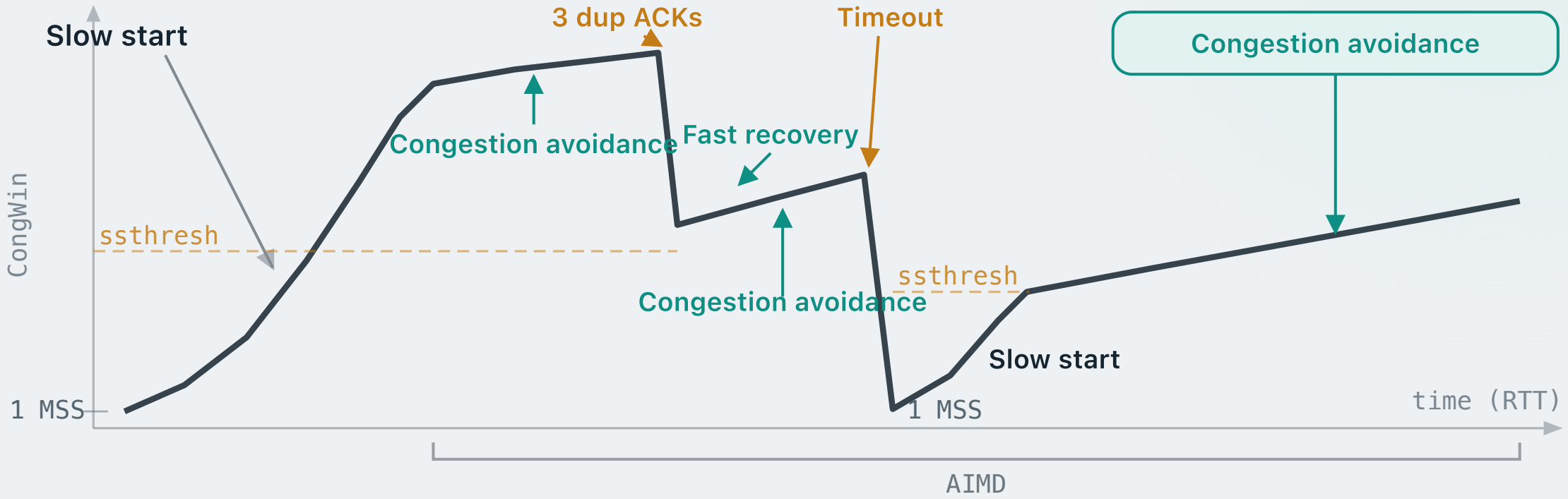
The sender receives ACK 10. Base advances to segment 10.

**CongWin = ssthresh = 3** — the inflated window deflates back to the halved value.

Enter **congestion avoidance**. Fast recovery is complete.



# TCP · congestion control overview



# Flow and congestion control — summary

**Flow control**: the receiver advertises **RcvWindow** in every segment, so a fast sender cannot overflow a slow receiver's buffer.

**Congestion control**: the TCP sender maintains a **CongWin**, and keeps its data in flight within both caps: **`nextseqnum − base ≤ min(CongWin, RcvWindow)`**.

TCP begins in **slow start**, doubling CongWin each RTT until ssthresh, then grows linearly in **congestion avoidance**.

Three dup ACKs trigger **fast recovery** — ssthresh and CongWin halve. A **timeout** drops CongWin to 1 MSS and restarts slow start.

# Acknowledgements

Lecture notes are developed based on slides from the following sources:

- Dr Mengmeng Ge's lecture notes for COSC264, University of Canterbury, 2024
- Assoc Prof Dan Kim's lecture notes for COSC264, University of Canterbury, 2017
- Prof Aleksandar Kuzmanovic's lecture notes for CS340, Northwestern University, 2005 — `https://users.cs.northwestern.edu/~akuzma/classes/CS340-w05/lecture_notes.htm`
- Dr Barry Wu's lecture notes for COSC264, University of Canterbury

# References

James F. Kurose & Keith W. Ross, *Computer Networking: A Top-Down Approach Featuring the Internet*, Addison Wesley, 3rd edition, 2005.

Chapter 3 Transport Layer (3.1 – 3.3, 3.5 – 3.7)

William Stallings, *Data and Computer Communications*, Pearson, 10th edition, 2013.

Chapter 15 Transport Protocols

Andrew S. Tanenbaum & David J. Wetherall, *Computer Networks*, Prentice Hall, 5th edition, 2010.

Chapter 6 The Transport Layer (6.2, 6.3, 6.5)